1. Create a GUI (Dark Green Theme) in Python in green with graphs and histograms with CSV files pip install matplotlib, numpy, pandas, tkinter, scipy, seaborn, statsmodels, plotly, scikit-learn - determination of toxins of rare species of ditome algae

I'll create a comprehensive GUI application for analyzing toxins in rare diatom algae species with a dark green theme. This will include data loading, visualization, statistical analysis, and machine learning capabilities.

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from matplotlib.figure import Figure
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, roc_curve,
auc
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')

# Set dark green theme
GREEN_THEME = {
    'bg_dark': '#0A1F0A',
    'bg_medium': '#1A3D1A',
    'bg_light': '#2A5C2A',
    'accent': '#4CAF50',
    'accent_light': '#81C784',
    'text': '#E8F5E9',
    'graph_bg': '#0A1F0A',
    'grid_color': '#2A5C2A',
    'toxic': '#FF5252',
    'warning': '#FF9800',
    'safe': '#4CAF50'
}

class DiatomToxinAnalyzer:
    def __init__(self, root):
        self.root = root
        self.root.title("Rare Diatom Algae Toxin Analyzer")
        self.root.geometry("1600x1000")
        self.root.configure(bg=GREEN_THEME['bg_dark'])
```

```
        # Initialize variables
        self.data = None
        self.current_visualization = None
        self.ml_model = None
        self.scaler = StandardScaler()
        self.label_encoder = LabelEncoder()

        # Create sample data
        self.sample_data = self.create_sample_data()

        # Set up UI
        self.setup_ui()

        # Load sample data initially
        self.load_sample_data()

    def create_sample_data(self):
        """Create realistic sample data for rare diatom algae toxin analysis"""
        np.random.seed(42)

        # Rare diatom species
        species_list = [
            'Pseudo-nitzschia australis',
            'Pseudo-nitzschia seriata',
            'Fragilariopsis cylindrus',
            'Thalassiosira antarctica',
            'Chaetoceros socialis',
            'Corethron pennatum',
            'Rhizosolenia hebetata',
            'Asterionellopsis glacialis',
            'Bacteriastrum furcatum',
            'Ditylum brightwellii'
        ]

        # Toxin types
        toxin_types = ['Domoic Acid', 'Okadaic Acid', 'Saxitoxin', 'Yessotoxin',
                       'Pectenotoxin', 'Azaspiracid', 'Brevetoxin', 'None']

        # Geographic locations
        locations = ['North Atlantic', 'South Atlantic', 'North Pacific', 'South Pacific',
                     'Arctic Ocean', 'Southern Ocean', 'Mediterranean Sea', 'Baltic Sea']

        # Create sample data
        n_samples = 500
        data = {
            'Sample_ID': [f'DIA_{i:04d}' for i in range(1, n_samples + 1)],
            'Collection_Date': pd.date_range('2023-01-01', periods=n_samples, freq='D').strftime('%Y-
%m-%d'),
            'Species': np.random.choice(species_list, n_samples),
            'Location': np.random.choice(locations, n_samples),
            'Water_Temperature_C': np.random.uniform(2, 28, n_samples),
            'Salinity_ppt': np.random.uniform(28, 36, n_samples),
```

```
        'pH': np.random.uniform(7.5, 8.5, n_samples),
        'Nitrate_uM': np.random.uniform(0.1, 10, n_samples),
        'Phosphate_uM': np.random.uniform(0.01, 2, n_samples),
        'Silicate_uM': np.random.uniform(1, 20, n_samples),
        'Chlorophyll_a_ugL': np.random.uniform(0.1, 15, n_samples),
        'Cell_Density_cellsmL': np.random.uniform(100, 10000, n_samples),
        'Primary_Toxin': np.random.choice(toxin_types, n_samples, p=[0.25, 0.15, 0.12, 0.1, 0.08,
0.08, 0.07, 0.15]),
        'Toxin_Concentration_ugL': np.zeros(n_samples),
        'Toxicity_Level': np.zeros(n_samples)  # 0: Non-toxic, 1: Low, 2: Medium, 3: High
    }

    df = pd.DataFrame(data)

    # Generate realistic toxin concentrations based on species and conditions
    for idx, row in df.iterrows():
        toxin_type = row['Primary_Toxin']
        species = row['Species']

        # Base concentrations by toxin type
        if toxin_type == 'Domoic Acid':
            base_conc = np.random.lognormal(1.5, 0.8)
        elif toxin_type == 'Okadaic Acid':
            base_conc = np.random.lognormal(1.2, 0.7)
        elif toxin_type == 'Saxitoxin':
            base_conc = np.random.lognormal(0.8, 0.6)
        elif toxin_type == 'Yessotoxin':
            base_conc = np.random.lognormal(1.0, 0.5)
        elif toxin_type == 'Pectenotoxin':
            base_conc = np.random.lognormal(0.7, 0.4)
        elif toxin_type == 'Azaspiracid':
            base_conc = np.random.lognormal(0.6, 0.4)
        elif toxin_type == 'Brevetoxin':
            base_conc = np.random.lognormal(0.9, 0.5)
        else:
            base_conc = 0.0

        # Species-specific adjustments
        if 'Pseudo-nitzschia' in species:
            base_conc *= np.random.uniform(1.5, 3.0)
        elif 'Fragilariopsis' in species:
            base_conc *= np.random.uniform(1.2, 2.0)

        # Environmental factors
        if row['Water_Temperature_C'] > 15:
            base_conc *= np.random.uniform(1.1, 1.5)
        if row['Nitrate_uM'] > 5:
            base_conc *= np.random.uniform(1.1, 1.4)

        df.at[idx, 'Toxin_Concentration_ugL'] = round(base_conc, 3)

        # Assign toxicity level
```

```
            if base_conc == 0:
                df.at[idx, 'Toxicity_Level'] = 0
            elif base_conc < 1:
                df.at[idx, 'Toxicity_Level'] = 1
            elif base_conc < 5:
                df.at[idx, 'Toxicity_Level'] = 2
            else:
                df.at[idx, 'Toxicity_Level'] = 3

        return df

    def setup_ui(self):
        # Create main container
        main_container = tk.Frame(self.root, bg=GREEN_THEME['bg_dark'])
        main_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Header
        header_frame = tk.Frame(main_container, bg=GREEN_THEME['bg_dark'])
        header_frame.pack(fill=tk.X, pady=(0, 10))

        title_label = tk.Label(
            header_frame,
            text="🔬 RARE DIATOM ALGAE TOXIN ANALYSIS SYSTEM",
            font=('Arial', 24, 'bold'),
            fg=GREEN_THEME['accent_light'],
            bg=GREEN_THEME['bg_dark']
        )
        title_label.pack()

        subtitle_label = tk.Label(
            header_frame,
            text="Advanced Tool for Detection and Analysis of Marine Biotoxins",
            font=('Arial', 12),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_dark']
        )
        subtitle_label.pack()

        # Create main content area with notebook
        self.notebook = ttk.Notebook(main_container)
        self.notebook.pack(fill=tk.BOTH, expand=True)

        # Create tabs
        self.tab_dashboard = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])
        self.tab_data = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])
        self.tab_visualize = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])
        self.tab_statistics = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])
        self.tab_ml = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])
        self.tab_report = tk.Frame(self.notebook, bg=GREEN_THEME['graph_bg'])

        self.notebook.add(self.tab_dashboard, text="📊 Dashboard")
        self.notebook.add(self.tab_data, text="📁 Data Management")
```

```
    self.notebook.add(self.tab_visualize, text="📈 Visualization")
    self.notebook.add(self.tab_statistics, text="📊 Statistics")
    self.notebook.add(self.tab_ml, text="🤖 Machine Learning")
    self.notebook.add(self.tab_report, text="📄 Report")

    # Setup each tab
    self.setup_dashboard_tab()
    self.setup_data_tab()
    self.setup_visualization_tab()
    self.setup_statistics_tab()
    self.setup_ml_tab()
    self.setup_report_tab()

    # Status bar
    self.status_bar = tk.Label(
        main_container,
        text="Ready | Loaded sample data with 500 rare diatom samples",
        font=('Arial', 9),
        fg=GREEN_THEME['accent_light'],
        bg=GREEN_THEME['bg_medium'],
        relief=tk.SUNKEN,
        anchor=tk.W
    )
    self.status_bar.pack(fill=tk.X, pady=(10, 0))

def setup_dashboard_tab(self):
    # Dashboard tab with overview
    dashboard_container = tk.Frame(self.tab_dashboard, bg=GREEN_THEME['graph_bg'])
    dashboard_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Metrics row
    metrics_frame = tk.Frame(dashboard_container, bg=GREEN_THEME['bg_medium'])
    metrics_frame.pack(fill=tk.X, pady=(0, 20))

    metrics = [
        ("Total Samples", "500", "#4CAF50"),
        ("Toxic Samples", "45%", "#FF5252"),
        ("Species Count", "10", "#81C784"),
        ("Avg Toxin Conc", "3.2 µg/L", "#FF9800"),
        ("High Risk", "18%", "#F44336"),
        ("Locations", "8", "#2196F3")
    ]

    for i, (label, value, color) in enumerate(metrics):
        metric_card = tk.Frame(metrics_frame, bg=color, relief=tk.RAISED, borderwidth=3)
        metric_card.grid(row=0, column=i, padx=5, pady=5, sticky='nsew')

        value_label = tk.Label(
            metric_card,

          text=value,
            font=('Arial', 20, 'bold'),
```

```
            fg='white',
            bg=color
        )
        value_label.pack(padx=20, pady=(10, 5))

        label_label = tk.Label(
            metric_card,
            text=label,
            font=('Arial', 10),
            fg='white',
            bg=color
        )
        label_label.pack(padx=20, pady=(0, 10))

        metrics_frame.columnconfigure(i, weight=1)

    # Create dashboard charts
    fig = Figure(figsize=(14, 8), facecolor=GREEN_THEME['graph_bg'])
    gs = fig.add_gridspec(2, 3, hspace=0.3, wspace=0.3)

    # Chart 1: Toxin distribution by species
    ax1 = fig.add_subplot(gs[0, 0])
    if self.data is not None:
        toxic_species = self.data[self.data['Toxicity_Level'] > 0]['Species'].value_counts().head(5)
        bars = ax1.bar(range(len(toxic_species)), toxic_species.values,
                       color=[GREEN_THEME['toxic'], '#FF9800', '#FFC107', '#8BC34A', '#2196F3'])
        ax1.set_xticks(range(len(toxic_species)))
        ax1.set_xticklabels([s[:15] + '...' if len(s) > 15 else s for s in toxic_species.index],
                            rotation=45, ha='right', color=GREEN_THEME['text'])
        ax1.set_title('Top Toxic Species', color=GREEN_THEME['accent_light'], fontsize=11)
        ax1.set_ylabel('Toxic Samples', color=GREEN_THEME['text'])
        ax1.set_facecolor(GREEN_THEME['graph_bg'])
        ax1.tick_params(colors=GREEN_THEME['text'])

        # Add value labels on bars
        for i, v in enumerate(toxic_species.values):
            ax1.text(i, v + 0.5, str(v), ha='center', va='bottom',
                     color=GREEN_THEME['text'], fontsize=9)

    # Chart 2: Toxin concentration histogram
    ax2 = fig.add_subplot(gs[0, 1])
    if self.data is not None:
        toxic_data = self.data[self.data['Toxicity_Level'] > 0]['Toxin_Concentration_ugL']
        ax2.hist(toxic_data, bins=30, color=GREEN_THEME['toxic'], alpha=0.7,
edgecolor='white')
        ax2.set_title('Toxin Concentration Distribution', color=GREEN_THEME['accent_light'],
fontsize=11)
        ax2.set_xlabel('Concentration (µg/L)', color=GREEN_THEME['text'])
        ax2.set_ylabel('Frequency', color=GREEN_THEME['text'])
        ax2.set_facecolor(GREEN_THEME['graph_bg'])
        ax2.tick_params(colors=GREEN_THEME['text'])
```

```
            # Add mean line
            mean_conc = toxic_data.mean()
            ax2.axvline(mean_conc, color=GREEN_THEME['accent_light'], linestyle='--', alpha=0.7)
            ax2.text(mean_conc + 0.1, ax2.get_ylim()[1] * 0.9, f'Mean: {mean_conc:.2f}',
                     color=GREEN_THEME['accent_light'], fontsize=9)

        # Chart 3: Toxin types pie chart
        ax3 = fig.add_subplot(gs[0, 2])
        if self.data is not None:
            toxin_counts = self.data[self.data['Primary_Toxin'] != 'None']
['Primary_Toxin'].value_counts()
            colors = ['#FF5252', '#FF9800', '#FFC107', '#8BC34A', '#2196F3', '#9C27B0', '#795548']
            wedges, texts, autotexts = ax3.pie(toxin_counts.values, labels=toxin_counts.index,
                                               autopct='%1.1f%%', colors=colors, startangle=90)
            ax3.set_title('Toxin Types Distribution', color=GREEN_THEME['accent_light'],
fontsize=11)

            # Style text
            for text in texts:
                text.set_color(GREEN_THEME['text'])
                text.set_fontsize(9)
            for autotext in autotexts:
                autotext.set_color('white')
                autotext.set_fontsize(8)

        # Chart 4: Environmental factors correlation
        ax4 = fig.add_subplot(gs[1, :])
        if self.data is not None:
            env_factors = ['Water_Temperature_C', 'Salinity_ppt', 'pH', 'Nitrate_uM',
                           'Phosphate_uM', 'Silicate_uM', 'Chlorophyll_a_ugL']

            # Calculate correlation
            corr_data = self.data[env_factors + ['Toxin_Concentration_ugL']].corr()

            # Create heatmap
            im = ax4.imshow(corr_data, cmap='RdYlGn', aspect='auto', vmin=-1, vmax=1)
            ax4.set_xticks(range(len(corr_data.columns)))
            ax4.set_yticks(range(len(corr_data.columns)))
            ax4.set_xticklabels([col.replace('_', '\n') for col in corr_data.columns],
                                color=GREEN_THEME['text'], fontsize=8)
            ax4.set_yticklabels([col.replace('_', '\n') for col in corr_data.columns],
                                color=GREEN_THEME['text'], fontsize=8)
            ax4.set_title('Environmental Factors Correlation Matrix',
                          color=GREEN_THEME['accent_light'], fontsize=12, pad=10)

            # Add correlation values
            for i in range(len(corr_data.columns)):
                for j in range(len(corr_data.columns)):
                    value = corr_data.iloc[i, j]
                    ax4.text(j, i, f'{value:.2f}', ha="center", va="center",
                             color="white" if abs(value) > 0.5 else "black", fontsize=7)
```

```
            # Add colorbar
            cbar = plt.colorbar(im, ax=ax4)
            cbar.ax.tick_params(colors=GREEN_THEME['text'])
            cbar.set_label('Correlation Coefficient', color=GREEN_THEME['text'])

        # Format all axes
        for ax in fig.axes:
            ax.set_facecolor(GREEN_THEME['graph_bg'])
            ax.grid(True, alpha=0.2, color=GREEN_THEME['grid_color'])

        plt.tight_layout()

        # Embed chart in dashboard
        canvas = FigureCanvasTkAgg(fig, dashboard_container)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    def setup_data_tab(self):
        # Data management tab
        data_container = tk.Frame(self.tab_data, bg=GREEN_THEME['graph_bg'])
        data_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        # Control panel
        control_panel = tk.Frame(data_container, bg=GREEN_THEME['bg_medium'],
relief=tk.RAISED, borderwidth=2)
        control_panel.pack(fill=tk.X, pady=(0, 20))

        # Data control buttons
        buttons_frame = tk.Frame(control_panel, bg=GREEN_THEME['bg_medium'])
        buttons_frame.pack(pady=10)

        button_configs = [
            ("📁 Load CSV", self.load_csv_data),
            ("💾 Save Data", self.save_data),
            ("🧪 Generate Sample", self.load_sample_data),
            ("📋 Data Info", self.show_data_info),
            ("🧹 Clean Data", self.clean_data),
            ("📤 Export", self.export_data)
        ]

        for i, (text, command) in enumerate(button_configs):
            btn = tk.Button(
                buttons_frame,
                text=text,
                command=command,
                font=('Arial', 10, 'bold'),
                bg=GREEN_THEME['accent'],
                fg=GREEN_THEME['bg_dark'],
                activebackground=GREEN_THEME['accent_light'],
                relief=tk.RAISED,
                borderwidth=2,
                cursor='hand2',
```

```
        width=12
    )
    btn.grid(row=0, column=i, padx=5)

# Data preview area
preview_frame = tk.LabelFrame(
    data_container,
    text="Data Preview",
    font=('Arial', 12, 'bold'),
    fg=GREEN_THEME['accent_light'],
    bg=GREEN_THEME['graph_bg'],
    relief=tk.GROOVE
)
preview_frame.pack(fill=tk.BOTH, expand=True)

# Create treeview for data display
columns = ['Sample_ID', 'Species', 'Primary_Toxin', 'Toxin_Concentration_ugL',
           'Toxicity_Level', 'Water_Temperature_C', 'Location']

# Scrollbars
tree_scroll_y = tk.Scrollbar(preview_frame)
tree_scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

tree_scroll_x = tk.Scrollbar(preview_frame, orient=tk.HORIZONTAL)
tree_scroll_x.pack(side=tk.BOTTOM, fill=tk.X)

# Treeview
self.data_tree = ttk.Treeview(
    preview_frame,
    columns=columns,
    show='headings',
    yscrollcommand=tree_scroll_y.set,
    xscrollcommand=tree_scroll_x.set,
    height=20
)

# Configure treeview style
style = ttk.Style()
style.theme_use('clam')
style.configure("Treeview",
                background=GREEN_THEME['bg_light'],
                foreground=GREEN_THEME['text'],
                fieldbackground=GREEN_THEME['bg_light'],
                rowheight=25)
style.map('Treeview', background=[('selected', GREEN_THEME['accent'])])

# Define headings
column_widths = [100, 150, 120, 150, 100, 150, 120]
for col, width in zip(columns, column_widths):
    self.data_tree.heading(col, text=col.replace('_', ' '))
    self.data_tree.column(col, width=width, anchor='center')
```

```
        # Configure scrollbars
        tree_scroll_y.config(command=self.data_tree.yview)
        tree_scroll_x.config(command=self.data_tree.xview)

        self.data_tree.pack(fill=tk.BOTH, expand=True)

    def setup_visualization_tab(self):
        # Visualization tab
        viz_container = tk.Frame(self.tab_visualize, bg=GREEN_THEME['graph_bg'])
        viz_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        # Control panel
        viz_control = tk.Frame(viz_container, bg=GREEN_THEME['bg_medium'])
        viz_control.pack(fill=tk.X, pady=(0, 20))

        # Visualization type
selection
        tk.Label(
            viz_control,
            text="Visualization Type:",
            font=('Arial', 10, 'bold'),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT, padx=(10, 5))

        self.viz_type = tk.StringVar(value="Histogram")

        viz_options = [
            "Histogram",
            "Scatter Plot",
            "Box Plot",
            "Bar Chart",
            "Heatmap",
            "PCA Analysis",
            "Time Series",
            "3D Scatter"
        ]

        viz_combo = ttk.Combobox(
            viz_control,
            textvariable=self.viz_type,
            values=viz_options,
            state='readonly',
            width=20
        )
        viz_combo.pack(side=tk.LEFT, padx=5)

        # Parameters frame
        param_frame = tk.Frame(viz_control, bg=GREEN_THEME['bg_medium'])
        param_frame.pack(side=tk.LEFT, padx=20)

        tk.Label(
```

```
            param_frame,
            text="X-axis:",
            font=('Arial', 9),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT)

        self.x_var = tk.StringVar(value="Toxin_Concentration_ugL")
        x_combo = ttk.Combobox(
            param_frame,
            textvariable=self.x_var,
            width=20,
            state='readonly'
        )
        x_combo.pack(side=tk.LEFT, padx=(5, 15))

        tk.Label(
            param_frame,
            text="Y-axis:",
            font=('Arial', 9),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT)

        self.y_var = tk.StringVar(value="Water_Temperature_C")
        y_combo = ttk.Combobox(
            param_frame,
            textvariable=self.y_var,
            width=20,
            state='readonly'
        )
        y_combo.pack(side=tk.LEFT, padx=5)

        # Update combobox values when data changes
        def update_viz_columns():
            if self.data is not None:
                numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
                x_combo['values'] = numeric_cols +
list(self.data.select_dtypes(exclude=[np.number]).columns)
                y_combo['values'] = numeric_cols

        # Generate button
        gen_btn = tk.Button(
            viz_control,
            text="Generate Visualization",
            command=lambda: [update_viz_columns(), self.generate_visualization()],
            font=('Arial', 10, 'bold'),
            bg=GREEN_THEME['accent'],
            fg=GREEN_THEME['bg_dark'],
            activebackground=GREEN_THEME['accent_light']
        )
        gen_btn.pack(side=tk.RIGHT, padx=10)
```

```
    # Visualization canvas frame
    self.viz_canvas_frame = tk.Frame(viz_container, bg=GREEN_THEME['graph_bg'])
    self.viz_canvas_frame.pack(fill=tk.BOTH, expand=True)

    # Initial column update
    update_viz_columns()

def setup_statistics_tab(self):
    # Statistics tab
    stats_container = tk.Frame(self.tab_statistics, bg=GREEN_THEME['graph_bg'])
    stats_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

    # Statistics control panel
    stats_control = tk.Frame(stats_container, bg=GREEN_THEME['bg_medium'])
    stats_control.pack(fill=tk.X, pady=(0, 20))

    # Statistics type selection
    tk.Label(
        stats_control,
        text="Analysis Type:",
        font=('Arial', 10, 'bold'),
        fg=GREEN_THEME['text'],
        bg=GREEN_THEME['bg_medium']
    ).pack(side=tk.LEFT, padx=(10, 5))

    self.stats_type = tk.StringVar(value="Descriptive")

    stats_options = [
        "Descriptive",
        "Correlation",
        "T-Test",
        "ANOVA",
        "Regression",
        "Distribution Test"
    ]

    stats_combo = ttk.Combobox(
        stats_control,
        textvariable=self.stats_type,
        values=stats_options,
        state='readonly',
        width=15
    )
    stats_combo.pack(side=tk.LEFT, padx=5)

    # Variable selection
    var_frame = tk.Frame(stats_control, bg=GREEN_THEME['bg_medium'])
    var_frame.pack(side=tk.LEFT, padx=20)

    tk.Label(
        var_frame,
```

```
    text="Variable:",
    font=('Arial', 9),
    fg=GREEN_THEME['text'],
    bg=GREEN_THEME['bg_medium']
).pack(side=tk.LEFT)

self.stats_var = tk.StringVar()
var_combo = ttk.Combobox(
    var_frame,
    textvariable=self.stats_var,
    width=20,
    state='readonly'
)
var_combo.pack(side=tk.LEFT, padx=5)

# Update combobox values
def update_stats_columns():
    if self.data is not None:
        numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
        var_combo['values'] = numeric_cols

# Run analysis button
run_btn = tk.Button(
    stats_control,
    text="Run Analysis",
    command=lambda: [update_stats_columns(), self.run_statistical_analysis()],
    font=('Arial', 10, 'bold'),
    bg=GREEN_THEME['accent'],
    fg=GREEN_THEME['bg_dark'],
    activebackground=GREEN_THEME['accent_light']
)
run_btn.pack(side=tk.RIGHT, padx=10)

# Results display area
results_frame = tk.LabelFrame(
    stats_container,
    text="Analysis Results",
    font=('Arial', 12, 'bold'),
    fg=GREEN_THEME['accent_light'],
    bg=GREEN_THEME['graph_bg'],
    relief=tk.GROOVE
)
results_frame.pack(fill=tk.BOTH, expand=True)

# Text widget for results
self.stats_text = scrolledtext.ScrolledText(
    results_frame,
    bg=GREEN_THEME['bg_light'],
    fg=GREEN_THEME['text'],
    font=('Courier New', 10),
    wrap=tk.WORD
)
```

```
        self.stats_text.pack(fill=tk.BOTH, expand=True)

        # Initial column update
        update_stats_columns()

    def setup_ml_tab(self):
        # Machine Learning tab
        ml_container = tk.Frame(self.tab_ml, bg=GREEN_THEME['graph_bg'])
        ml_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        # ML control panel
        ml_control = tk.Frame(ml_container, bg=GREEN_THEME['bg_medium'])
        ml_control.pack(fill=tk.X, pady=(0, 20))

        # Model selection
        tk.Label(
            ml_control,
            text="ML Model:",
            font=('Arial', 10, 'bold'),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT, padx=(10, 5))

        self.ml_model_type = tk.StringVar(value="Random Forest")

        model_options = ["Random Forest", "Gradient Boosting", "Logistic Regression"]
        model_combo = ttk.Combobox(
            ml_control,
            textvariable=self.ml_model_type,
            values=model_options,
            state='readonly',
            width=15
        )
        model_combo.pack(side=tk.LEFT, padx=5)

        # Target variable
        target_frame = tk.Frame(ml_control, bg=GREEN_THEME['bg_medium'])
        target_frame.pack(side=tk.LEFT, padx=20)

        tk.Label(
            target_frame,
            text="Target:",
            font=('Arial', 9),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT)

        self.target_var = tk.StringVar(value="Toxicity_Level")
        target_combo = ttk.Combobox(
            target_frame,
            textvariable=self.target_var,
            width=15,
```

```
    state='readonly'
)
target_combo.pack(side=tk.LEFT, padx=5)

# Train button
train_btn = tk.Button(
    ml_control,
    text="🚀 Train Model",
    command=self.train_ml_model,
    font=('Arial', 11, 'bold'),
    bg=GREEN_THEME['accent'],
    fg=GREEN_THEME['bg_dark'],
    activebackground=GREEN_THEME['accent_light'],
    relief=tk.RAISED,
    borderwidth=2
)
train_btn.pack(side=tk.RIGHT, padx=10)

# Predict button
predict_btn = tk.Button(
    ml_control,
    text="🔮 Make Prediction",
    command=self.predict_toxicity,
    font=('Arial', 11, 'bold'),
    bg=GREEN_THEME['accent_light'],
    fg=GREEN_THEME['bg_dark'],
    activebackground=GREEN_THEME['accent'],
    relief=tk.RAISED,
    borderwidth=2
)
predict_btn.pack(side=tk.RIGHT, padx=5)

# ML results area (split horizontally)
ml_results_container = tk.Frame(ml_container, bg=GREEN_THEME['graph_bg'])
ml_results_container.pack(fill=tk.BOTH, expand=True)

# Left: Text results
text_frame = tk.Frame(ml_results_container, bg=GREEN_THEME['graph_bg'])
text_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=(0, 10))

tk.Label(
    text_frame,
    text="Model Results",
    font=('Arial', 11, 'bold'),
    fg=GREEN_THEME['accent_light'],
    bg=GREEN_THEME['graph_bg']
).pack(anchor=tk.W)

self.ml_text = scrolledtext.ScrolledText(
    text_frame,
    bg=GREEN_THEME['bg_light'],
    fg=GREEN_THEME['text'],
```

```
            font=('Courier New', 9),
            wrap=tk.WORD
        )
        self.ml_text.pack(fill=tk.BOTH, expand=True)

        # Right:
 Visualization
        viz_frame = tk.Frame(ml_results_container, bg=GREEN_THEME['graph_bg'])
        viz_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=(10, 0))

        tk.Label(
            viz_frame,
            text="Model Visualization",
            font=('Arial', 11, 'bold'),
            fg=GREEN_THEME['accent_light'],
            bg=GREEN_THEME['graph_bg']
        ).pack(anchor=tk.W)

        self.ml_viz_frame = tk.Frame(viz_frame, bg=GREEN_THEME['graph_bg'])
        self.ml_viz_frame.pack(fill=tk.BOTH, expand=True)

        # Update target combobox
        if self.data is not None:
            numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
            target_combo['values'] = numeric_cols

    def setup_report_tab(self):
        # Report tab
        report_container = tk.Frame(self.tab_report, bg=GREEN_THEME['graph_bg'])
        report_container.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)

        # Report control panel
        report_control = tk.Frame(report_container, bg=GREEN_THEME['bg_medium'])
        report_control.pack(fill=tk.X, pady=(0, 20))

        # Report type selection
        tk.Label(
            report_control,
            text="Report Type:",
            font=('Arial', 10, 'bold'),
            fg=GREEN_THEME['text'],
            bg=GREEN_THEME['bg_medium']
        ).pack(side=tk.LEFT, padx=(10, 5))

        self.report_type = tk.StringVar(value="Comprehensive")

        report_options = ["Comprehensive", "Summary", "Species Analysis", "Toxin Profile", "Risk
Assessment"]
        report_combo = ttk.Combobox(
            report_control,
            textvariable=self.report_type,
            values=report_options,
```

```
    state='readonly',
    width=15
)
report_combo.pack(side=tk.LEFT, padx=5)

# Generate report button
gen_btn = tk.Button(
    report_control,
    text="📄 Generate Report",
    command=self.generate_report,
    font=('Arial', 11, 'bold'),
    bg=GREEN_THEME['accent'],
    fg=GREEN_THEME['bg_dark'],
    activebackground=GREEN_THEME['accent_light'],
    relief=tk.RAISED,
    borderwidth=2
)
gen_btn.pack(side=tk.LEFT, padx=20)

# Export button
export_btn = tk.Button(
    report_control,
    text="💾 Export Report",
    command=self.export_report,
    font=('Arial', 11, 'bold'),
    bg=GREEN_THEME['accent_light'],
    fg=GREEN_THEME['bg_dark'],
    activebackground=GREEN_THEME['accent'],
    relief=tk.RAISED,
    borderwidth=2
)
export_btn.pack(side=tk.LEFT)

# Report display area
report_display = tk.LabelFrame(
    report_container,
    text="Generated Report",
    font=('Arial', 12, 'bold'),
    fg=GREEN_THEME['accent_light'],
    bg=GREEN_THEME['graph_bg'],
    relief=tk.GROOVE
)
report_display.pack(fill=tk.BOTH, expand=True)

# Text widget for report
self.report_text = scrolledtext.ScrolledText(
    report_display,
    bg=GREEN_THEME['bg_light'],
    fg=GREEN_THEME['text'],
    font=('Courier New', 10),
    wrap=tk.WORD
)
```

```
        self.report_text.pack(fill=tk.BOTH, expand=True)

        # Generate initial report
        self.generate_report()

    def load_csv_data(self):
        file_path = filedialog.askopenfilename(
            title="Select CSV File",
            filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx;*.xls"), ("All files", "*.*")]
        )

        if file_path:
            try:
                if file_path.endswith(('.xlsx', '.xls')):
                    self.data = pd.read_excel(file_path)
                else:
                    self.data = pd.read_csv(file_path)

                self.update_data_preview()
                self.update_status(f"Loaded {len(self.data)} records from {file_path.split('/')[-1]}")
                messagebox.showinfo("Success", f"Successfully loaded {len(self.data)} records")

            except Exception as e:
                messagebox.showerror("Error", f"Failed to load file: {str(e)}")

    def load_sample_data(self):
        self.data = self.sample_data.copy()
        self.update_data_preview()
        self.update_status("Loaded sample data with 500 rare diatom samples")
        messagebox.showinfo("Sample Data", "Loaded comprehensive sample data for analysis")

    def save_data(self):
        if self.data is None:
            messagebox.showwarning("No Data", "No data to save")
            return

        file_path = filedialog.asksaveasfilename(
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx"), ("All files", "*.*")]
        )

        if file_path:
            try:
                if file_path.endswith('.xlsx'):
                    self.data.to_excel(file_path, index=False)
                else:
                    self.data.to_csv(file_path, index=False)

                self.update_status(f"Data saved to {file_path}")
                messagebox.showinfo("Success", "Data saved successfully")

            except Exception as e:
```

```
            messagebox.showerror("Error", f"Failed to save data: {str(e)}")

    def show_data_info(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        info_window = tk.Toplevel(self.root)
        info_window.title("Data Information")
        info_window.geometry("800x600")
        info_window.configure(bg=GREEN_THEME['bg_dark'])

        # Create text widget
        text_widget = scrolledtext.ScrolledText(
            info_window,
            bg=GREEN_THEME['bg_light'],
            fg=GREEN_THEME['text'],
            font=('Courier New', 10),
            wrap=tk.WORD
        )
        text_widget.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Generate info text
        info = "=" * 80 + "\n"
        info += "DATA INFORMATION REPORT\n"
        info += "=" * 80 + "\n\n"

        info += f"Dataset Shape: {self.data.shape[0]} rows × {self.data.shape[1]} columns\n\n"

        info += "COLUMN INFORMATION:\n"
        info += "-" * 80 + "\n"
        for col in self.data.columns:
            dtype = str(self.data[col].dtype)
            non_null = self.data[col].count()
            total = len(self.data)
            info += f"{col:<25} {dtype:<15} {non_null}/{total} non-null values\n"

        info += "\nDATA TYPES:\n"
        info += "-" * 80 + "\n"
        type_counts = self.data.dtypes.value_counts()
        for dtype, count in type_counts.items():
            info += f"{dtype}: {count} columns\n"

        info += "\nTOXICITY DISTRIBUTION:\n"
        info += "-" * 80 + "\n"
        if 'Toxicity_Level' in self.data.columns:
            toxicity_counts = self.data['Toxicity_Level'].value_counts().sort_index()
            toxicity_labels = ['Non-toxic', 'Low', 'Medium', 'High']
            for level, count in toxicity_counts.items():
                label = toxicity_labels[level] if level < len(toxicity_labels) else f"Level {level}"
                percentage = (count / len(self.data)) * 100
                info += f"{label:<15} {count:<10} ({percentage:.1f}%)\n"
```

```
        info += "\nTOXIN TYPES:\n"
        info += "-" * 80 + "\n"
        if 'Primary_Toxin' in self.data.columns:
            toxin_counts = self.data['Primary_Toxin'].value_counts()
            for toxin, count in toxin_counts.items():
                percentage = (count / len(self.data)) * 100
                info += f"{toxin:<20} {count:<10} ({percentage:.1f}%)\n"

        text_widget.insert(tk.END, info)
        text_widget.configure(state='disabled')

    def clean_data(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        original_shape = self.data.shape

        # Remove duplicates
        self.data = self.data.drop_duplicates()

        # Fill missing numeric values with median
        numeric_cols = self.data.select_dtypes(include=[np.number]).columns
        for col in numeric_cols:
            self.data[col] = self.data[col].fillna(self.data[col].median())

        # Fill missing categorical values with mode
        categorical_cols = self.data.select_dtypes(include=['object']).columns
        for col in categorical_cols:
            if self.data[col].isnull().any():
                mode_val = self.data[col].mode()[0]
                self.data[col] = self.data[col].fillna(mode_val)

        removed_duplicates = original_shape[0] - self.data.shape[0]
        self.update_data_preview()
        self.update_status(f"Data cleaned. Removed {removed_duplicates} duplicates")
        messagebox.showinfo("Data Cleaned", f"Removed {removed_duplicates} duplicate rows")

    def export_data(self):
        self.save_data()

    def update_data_preview(self):
        if self.data is not None:
            # Clear existing items
            for item in self.data_tree.get_children():
                self.data_tree.delete(item)

            # Add new items (limit to 100 for performance)
            preview_data = self.data.head(100)
            for _, row in preview_data.iterrows():
                values = [
```

```
                row.get('Sample_ID', ''),
                row.get('Species', ''),
                row.get('Primary_Toxin', ''),

                f"{row.get('Toxin_Concentration_ugL', 0):.3f}",
                row.get('Toxicity_Level', 0),
                f"{row.get('Water_Temperature_C', 0):.1f}",
                row.get('Location', '')
            ]
            self.data_tree.insert('', tk.END, values=values)

    def generate_visualization(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        # Clear previous visualization
        for widget in self.viz_canvas_frame.winfo_children():
            widget.destroy()

        viz_type = self.viz_type.get()
        x_col = self.x_var.get()
        y_col = self.y_var.get()

        try:
            fig = Figure(figsize=(12, 8), facecolor=GREEN_THEME['graph_bg'])

            if viz_type == "Histogram":
                ax = fig.add_subplot(111)
                if x_col in self.data.columns:
                    ax.hist(self.data[x_col].dropna(), bins=30, color=GREEN_THEME['accent'],
                            alpha=0.7, edgecolor='white')
                    ax.set_xlabel(x_col.replace('_', ' '), color=GREEN_THEME['text'])
                    ax.set_ylabel('Frequency', color=GREEN_THEME['text'])
                    ax.set_title(f'Distribution of {x_col.replace("_", " ")}',
                                 color=GREEN_THEME['accent_light'], fontsize=14)

                    # Add statistics
                    mean_val = self.data[x_col].mean()
                    median_val = self.data[x_col].median()
                    ax.axvline(mean_val, color=GREEN_THEME['toxic'], linestyle='--', label=f'Mean:
{mean_val:.2f}')
                    ax.axvline(median_val, color=GREEN_THEME['warning'], linestyle='--',
label=f'Median: {median_val:.2f}')
                    ax.legend(facecolor=GREEN_THEME['bg_medium'],
edgecolor=GREEN_THEME['grid_color'])

            elif viz_type == "Scatter Plot":
                ax = fig.add_subplot(111)
                if x_col in self.data.columns and y_col in self.data.columns:
                    scatter = ax.scatter(self.data[x_col], self.data[y_col],
                                         c=self.data.get('Toxicity_Level', 0),
```

```
                    cmap='RdYlGn_r', alpha=0.6, s=50)
        ax.set_xlabel(x_col.replace('_', ' '), color=GREEN_THEME['text'])
        ax.set_ylabel(y_col.replace('_', ' '), color=GREEN_THEME['text'])
        ax.set_title(f'{x_col.replace("_", " ")} vs {y_col.replace("_", " ")}',
                color=GREEN_THEME['accent_light'], fontsize=14)

        # Add colorbar for toxicity
        if 'Toxicity_Level' in self.data.columns:
            cbar = plt.colorbar(scatter, ax=ax)
            cbar.set_label('Toxicity Level', color=GREEN_THEME['text'])
            cbar.ax.tick_params(colors=GREEN_THEME['text'])

    elif viz_type == "Box Plot":
        ax = fig.add_subplot(111)
        if x_col in self.data.columns and y_col in self.data.columns:
            # Group data by x_col if it's categorical
            if self.data[x_col].dtype == 'object':
                groups = [self.data[self.data[x_col] == val][y_col].dropna()
                        for val in self.data[x_col].unique()[:10]]  # Limit to 10 groups
                labels = [str(val)[:15] for val in self.data[x_col].unique()[:10]]
                boxplot = ax.boxplot(groups, labels=labels, patch_artist=True)

                # Color boxes
                colors = [GREEN_THEME['accent'], GREEN_THEME['accent_light'],
                        GREEN_THEME['toxic'], GREEN_THEME['warning']]
                for patch, color in zip(boxplot['boxes'], colors * 3):
                    patch.set_facecolor(color)

                ax.set_xlabel(x_col.replace('_', ' '), color=GREEN_THEME['text'])
                ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right')
            else:
                ax.boxplot(self.data[y_col].dropna())
                ax.set_xlabel('Distribution', color=GREEN_THEME['text'])

            ax.set_ylabel(y_col.replace('_', ' '), color=GREEN_THEME['text'])
            ax.set_title(f'Box Plot of {y_col.replace("_", " ")}',
                    color=GREEN_THEME['accent_light'], fontsize=14)

    elif viz_type == "Bar Chart":
        ax = fig.add_subplot(111)
        if x_col in self.data.columns:
            if self.data[x_col].dtype == 'object':
                # Count plot for categorical data
                counts = self.data[x_col].value_counts().head(10)
                bars = ax.bar(range(len(counts)), counts.values,
                            color=GREEN_THEME['accent'])
                ax.set_xticks(range(len(counts)))
                ax.set_xticklabels([str(val)[:15] for val in counts.index],
                                rotation=45, ha='right', color=GREEN_THEME['text'])
                ax.set_ylabel('Count', color=GREEN_THEME['text'])
            else:
                # Histogram-like bar chart for numeric data
```

```
                ax.hist(self.data[x_col].dropna(), bins=30, color=GREEN_THEME['accent'],
                        alpha=0.7, edgecolor='white')
                ax.set_ylabel('Frequency', color=GREEN_THEME['text'])

            ax.set_xlabel(x_col.replace('_', ' '), color=GREEN_THEME['text'])
            ax.set_title(f'Distribution of {x_col.replace("_", " ")}',
                         color=GREEN_THEME['accent_light'], fontsize=14)

        elif viz_type == "Heatmap":
            ax = fig.add_subplot(111)
            # Select numeric columns for correlation
            numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()[:10]  #
Limit to 10
            corr_data = self.data[numeric_cols].corr()

            im = ax.imshow(corr_data, cmap='RdYlGn', aspect='auto', vmin=-1, vmax=1)
            ax.set_xticks(range(len(numeric_cols)))
            ax.set_yticks(range(len(numeric_cols)))
            ax.set_xticklabels([col.replace('_', '\n') for col in numeric_cols],
                               color=GREEN_THEME['text'], fontsize=8, rotation=45)
            ax.set_yticklabels([col.replace('_', '\n') for col in numeric_cols],
                               color=GREEN_THEME['text'], fontsize=8)
            ax.set_title('Correlation Heatmap', color=GREEN_THEME['accent_light'], fontsize=14)

            # Add correlation values
            for i in range(len(numeric_cols)):
                for j in range(len(numeric_cols)):
                    value = corr_data.iloc[i, j]
                    ax.text(j, i, f'{value:.2f}', ha="center", va="center",
                            color="white" if abs(value) > 0.5 else "black", fontsize=7)

            plt.colorbar(im, ax=ax).set_label('Correlation', color=GREEN_THEME['text'])

        elif viz_type == "PCA Analysis":
            ax = fig.add_subplot(111)
            # Select numeric features
            numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
            if len(numeric_cols) > 2:
                # Perform PCA
                pca = PCA(n_components=2)
                X = self.data[numeric_cols].fillna(self.data[numeric_cols].mean())
                X_scaled = (X - X.mean()) / X.std()
                principal_components = pca.fit_transform(X_scaled)

                # Plot PCA
                scatter = ax.scatter(principal_components[:, 0], principal_components[:, 1],
                                     c=self.data.get('Toxicity_Level', 0), cmap='RdYlGn_r',
                                     alpha=0.6, s=50)
                ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)',
                              color=GREEN_THEME['text'])
                ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)',
                              color=GREEN_THEME['text'])
```

```
            ax.set_title('PCA Analysis', color=GREEN_THEME['accent_light'], fontsize=14)

            # Add colorbar
            if 'Toxicity_Level' in self.data.columns:
                cbar = plt.colorbar(scatter, ax=ax)
                cbar.set_label('Toxicity Level', color=GREEN_THEME['text'])
                cbar.ax.tick_params(colors=GREEN_THEME['text'])

        elif viz_type == "Time Series":
            ax = fig.add_subplot(111)
            if 'Collection_Date' in self.data.columns and y_col in self.data.columns:
                # Convert to datetime
                self.data['Collection_Date'] = pd.to_datetime(self.data['Collection_Date'])
                self.data = self.data.sort_values('Collection_Date')

                ax.plot(self.data['Collection_Date'], self.data[y_col],
                        color=GREEN_THEME['accent'], linewidth=2)
                ax.set_xlabel('Date', color=GREEN_THEME['text'])
                ax.set_ylabel(y_col.replace('_', ' '), color=GREEN_THEME['text'])

    ax.set_title(f'{y_col.replace("_", " ")} Over Time',
                        color=GREEN_THEME['accent_light'], fontsize=14)
                ax.xaxis.set_tick_params(rotation=45)

        elif viz_type == "3D Scatter":
            ax = fig.add_subplot(111, projection='3d')
            if len([x_col, y_col]) >= 2:
                z_col = 'Toxin_Concentration_ugL' if 'Toxin_Concentration_ugL' in self.data.columns
else self.data.columns[2]

                scatter = ax.scatter(self.data[x_col], self.data[y_col], self.data[z_col],
                                     c=self.data.get('Toxicity_Level', 0), cmap='RdYlGn_r',
                                     alpha=0.6, s=30)

                ax.set_xlabel(x_col.replace('_', ' '), color=GREEN_THEME['text'])
                ax.set_ylabel(y_col.replace('_', ' '), color=GREEN_THEME['text'])
                ax.set_zlabel(z_col.replace('_', ' '), color=GREEN_THEME['text'])
                ax.set_title('3D Scatter Plot', color=GREEN_THEME['accent_light'], fontsize=14)

                # Set background colors
                ax.xaxis.set_pane_color(GREEN_THEME['bg_medium'])
                ax.yaxis.set_pane_color(GREEN_THEME['bg_medium'])
                ax.zaxis.set_pane_color(GREEN_THEME['bg_medium'])

        # Common formatting
        ax.set_facecolor(GREEN_THEME['graph_bg'])
        ax.grid(True, alpha=0.2, color=GREEN_THEME['grid_color'])
        ax.tick_params(colors=GREEN_THEME['text'])

        plt.tight_layout()

        # Embed in GUI
```

```
            canvas = FigureCanvasTkAgg(fig, self.viz_canvas_frame)
            canvas.draw()

            # Add toolbar
            toolbar = NavigationToolbar2Tk(canvas, self.viz_canvas_frame)
            toolbar.update()
            toolbar.pack(side=tk.BOTTOM, fill=tk.X)
            canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

            self.update_status(f"Generated {viz_type} visualization")

        except Exception as e:
            messagebox.showerror("Visualization Error", f"Failed to create visualization: {str(e)}")

    def run_statistical_analysis(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        analysis_type = self.stats_type.get()
        variable = self.stats_var.get()

        if not variable:
            messagebox.showwarning("No Variable", "Please select a variable for analysis")
            return

        try:
            self.stats_text.delete(1.0, tk.END)

            if analysis_type == "Descriptive":
                self.stats_text.insert(tk.END, "=" * 80 + "\n")
                self.stats_text.insert(tk.END, f"DESCRIPTIVE STATISTICS FOR: {variable}\n")
                self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

                stats_data = self.data[variable].describe()
                for stat, value in stats_data.items():
                    self.stats_text.insert(tk.END, f"{stat:<15} {value:>15.4f}\n")

                # Additional statistics
                self.stats_text.insert(tk.END, "\nADDITIONAL STATISTICS:\n")
                self.stats_text.insert(tk.END, "-" * 40 + "\n")
                self.stats_text.insert(tk.END, f"Skewness:          {self.data[variable].skew():>15.4f}\n")
                self.stats_text.insert(tk.END, f"Kurtosis:          {self.data[variable].kurtosis():>15.4f}\n")
                self.stats_text.insert(tk.END, f"IQR:               {stats_data['75%'] -
stats_data['25%']:>15.4f}\n")
                self.stats_text.insert(tk.END, f"Range:             {stats_data['max'] -
stats_data['min']:>15.4f}\n")
                self.stats_text.insert(tk.END, f"Missing Values:    {self.data[variable].isnull().sum():>15}\
n")

            elif analysis_type == "Correlation":
                self.stats_text.insert(tk.END, "=" * 80 + "\n")
```

```
        self.stats_text.insert(tk.END, f"CORRELATION ANALYSIS\n")
        self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

        # Calculate correlations with all numeric variables
        numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
        correlations = []

        for col in numeric_cols:
            if col != variable:
                corr = self.data[[variable, col]].corr().iloc[0, 1]
                correlations.append((col, corr))

        # Sort by absolute correlation
        correlations.sort(key=lambda x: abs(x[1]), reverse=True)

        self.stats_text.insert(tk.END, f"Correlations with {variable}:\n")
        self.stats_text.insert(tk.END, "-" * 80 + "\n")
        self.stats_text.insert(tk.END, f"{'Variable':<30} {'Correlation':<15} {'Strength':<20}\n")
        self.stats_text.insert(tk.END, "-" * 80 + "\n")

        for col, corr in correlations[:20]:  # Show top 20
            strength = "Very Strong" if abs(corr) >= 0.8 else \
                       "Strong" if abs(corr) >= 0.6 else \
                       "Moderate" if abs(corr) >= 0.4 else \
                       "Weak" if abs(corr) >= 0.2 else "Very Weak"

            self.stats_text.insert(tk.END, f"{col:<30} {corr:>15.4f} {strength:<20}\n")

    elif analysis_type == "T-Test":
        self.stats_text.insert(tk.END, "=" * 80 + "\n")
        self.stats_text.insert(tk.END, "T-TEST ANALYSIS\n")
        self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

        # Compare toxic vs non-toxic groups
        if 'Toxicity_Level' in self.data.columns:
            toxic_group = self.data[self.data['Toxicity_Level'] > 0][variable].dropna()
            non_toxic_group = self.data[self.data['Toxicity_Level'] == 0][variable].dropna()

            if len(toxic_group) > 1 and len(non_toxic_group) > 1:
                t_stat, p_value = stats.ttest_ind(toxic_group, non_toxic_group)

                self.stats_text.insert(tk.END, "Group Statistics:\n")
                self.stats_text.insert(tk.END, "-" * 40 + "\n")
                self.stats_text.insert(tk.END, f"Toxic Group (n={len(toxic_group)}):\n")
                self.stats_text.insert(tk.END, f"  Mean: {toxic_group.mean():.4f}\n")
                self.stats_text.insert(tk.END, f"  Std:  {toxic_group.std():.4f}\n")
                self.stats_text.insert(tk.END, f"\nNon-Toxic Group (n={len(non_toxic_group)}):\n")
                self.stats_text.insert(tk.END, f"  Mean: {non_toxic_group.mean():.4f}\n")
                self.stats_text.insert(tk.END, f"  Std:  {non_toxic_group.std():.4f}\n")

                self.stats_text.insert(tk.END, "\nT-Test Results:\n")
                self.stats_text.insert(tk.END, "-" * 40 + "\n")
```

```
                self.stats_text.insert(tk.END, f"T-statistic: {t_stat:.4f}\n")
                self.stats_text.insert(tk.END, f"P-value:     {p_value:.4f}\n")
                self.stats_text.insert(tk.END, f"\nInterpretation:\n")

                if p_value < 0.01:
                    self.stats_text.insert(tk.END, "  Highly significant difference (p < 0.01)\n")
                elif p_value < 0.05:
                    self.stats_text.insert(tk.END, "  Significant difference (p < 0.05)\n")
                else:
                    self.stats_text.insert(tk.END, "  No significant difference\n")

        elif analysis_type == "ANOVA":
            self.stats_text.insert(tk.END, "=" * 80 + "\n")
            self.stats_text.insert(tk.END, "ANOVA ANALYSIS\n")
            self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

            # Compare means across toxicity levels
            if 'Toxicity_Level' in self.data.columns:
                groups = []
                for level in sorted(self.data['Toxicity_Level'].unique()):
                    group_data = self.data[self.data['Toxicity_Level'] == level][variable].dropna()
                    groups.append(group_data)

                    self.stats_text.insert(tk.END, f"Toxicity Level {level} (n={len(group_data)}):\n")
                    self.stats_text.insert(tk.END, f"  Mean: {group_data.mean():.4f}\n")
                    self.stats_text.insert(tk.END, f"  Std:  {group_data.std():.4f}\n\n")

                if len(groups) > 1:
                    f_stat, p_value = stats.f_oneway(*groups)

                    self.stats_text.insert(tk.END, "ANOVA Results:\n")
                    self.stats_text.insert(tk.END, "-" * 40 + "\n")
                    self.stats_text.insert(tk.END, f"F-statistic: {f_stat:.4f}\n")
                    self.stats_text.insert(tk.END, f"P-value:     {p_value:.4f}\n")

        elif analysis_type == "Regression":
            self.stats_text.insert(tk.END, "=" * 80 + "\n")
            self.stats_text.insert(tk.END, "LINEAR REGRESSION ANALYSIS\n")
            self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

            # Simple linear regression with toxin concentration

    if 'Toxin_Concentration_ugL' in self.data.columns:
                X = self.data[[variable]].fillna(self.data[variable].mean())
                y =
self.data['Toxin_Concentration_ugL'].fillna(self.data['Toxin_Concentration_ugL'].mean())

                X = sm.add_constant(X)
                model = sm.OLS(y, X).fit()

                self.stats_text.insert(tk.END, str(model.summary()))
```

```
        elif analysis_type == "Distribution Test":
            self.stats_text.insert(tk.END, "=" * 80 + "\n")
            self.stats_text.insert(tk.END, f"DISTRIBUTION TESTS FOR: {variable}\n")
            self.stats_text.insert(tk.END, "=" * 80 + "\n\n")

            data = self.data[variable].dropna()

            # Shapiro-Wilk test for normality
            shapiro_stat, shapiro_p = stats.shapiro(data)
            self.stats_text.insert(tk.END, "Shapiro-Wilk Test for Normality:\n")
            self.stats_text.insert(tk.END, f"  Statistic: {shapiro_stat:.4f}\n")
            self.stats_text.insert(tk.END, f"  P-value:   {shapiro_p:.4f}\n")
            self.stats_text.insert(tk.END, f"  Conclusion: {'Normal' if shapiro_p > 0.05 else 'Not
Normal'}\n\n")

            # Kolmogorov-Smirnov test
            ks_stat, ks_p = stats.kstest(data, 'norm', args=(data.mean(), data.std()))
            self.stats_text.insert(tk.END, "Kolmogorov-Smirnov Test:\n")
            self.stats_text.insert(tk.END, f"  Statistic: {ks_stat:.4f}\n")
            self.stats_text.insert(tk.END, f"  P-value:   {ks_p:.4f}\n\n")

            # Distribution parameters
            self.stats_text.insert(tk.END, "Distribution Parameters:\n")
            self.stats_text.insert(tk.END, f"  Mean:     {data.mean():.4f}\n")
            self.stats_text.insert(tk.END, f"  Std Dev:  {data.std():.4f}\n")
            self.stats_text.insert(tk.END, f"  Skewness: {data.skew():.4f}\n")
            self.stats_text.insert(tk.END, f"  Kurtosis: {data.kurtosis():.4f}\n")

        self.update_status(f"Completed {analysis_type} analysis on {variable}")

    except Exception as e:
        messagebox.showerror("Analysis Error", f"Failed to perform analysis: {str(e)}")

def train_ml_model(self):
    if self.data is None:
        messagebox.showwarning("No Data", "Please load data first")
        return

    model_type = self.ml_model_type.get()
    target = self.target_var.get()

    try:
        self.ml_text.delete(1.0, tk.END)

        # Prepare data
        numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
        if target in numeric_cols:
            numeric_cols.remove(target)

        X = self.data[numeric_cols].fillna(self.data[numeric_cols].mean())
        y = self.data[target]
```

```
# Encode if categorical
if y.dtype == 'object':
    y = self.label_encoder.fit_transform(y)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Scale features
X_train_scaled = self.scaler.fit_transform(X_train)
X_test_scaled = self.scaler.transform(X_test)

# Train model
if model_type == "Random Forest":
    self.ml_model = RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        random_state=42,
        class_weight='balanced'
    )
elif model_type == "Gradient Boosting":
    self.ml_model = GradientBoostingClassifier(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=5,
        random_state=42
    )

self.ml_model.fit(X_train_scaled, y_train)

# Make predictions
y_pred = self.ml_model.predict(X_test_scaled)
y_prob = self.ml_model.predict_proba(X_test_scaled)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, output_dict=True)

# Display results
self.ml_text.insert(tk.END, "=" * 80 + "\n")
self.ml_text.insert(tk.END, f"MACHINE LEARNING RESULTS - {model_type}\n")
self.ml_text.insert(tk.END, "=" * 80 + "\n\n")

self.ml_text.insert(tk.END, f"Target Variable: {target}\n")
self.ml_text.insert(tk.END, f"Features Used: {len(numeric_cols)}\n")
self.ml_text.insert(tk.END, f"Training Samples: {len(X_train)}\n")
self.ml_text.insert(tk.END, f"Testing Samples: {len(X_test)}\n\n")

self.ml_text.insert(tk.END, f"Model Accuracy: {accuracy:.2%}\n\n")
```

```
            self.ml_text.insert(tk.END, "Classification Report:\n")
            self.ml_text.insert(tk.END, "-" * 60 + "\n")

            # Format classification report
            for class_name, metrics in report.items():
                if class_name not in ['accuracy', 'macro avg', 'weighted avg']:
                    self.ml_text.insert(tk.END,
                        f"Class {class_name}:\n")
                    self.ml_text.insert(tk.END,
                        f"  Precision: {metrics['precision']:.3f}\n")
                    self.ml_text.insert(tk.END,
                        f"  Recall:    {metrics['recall']:.3f}\n")
                    self.ml_text.insert(tk.END,
                        f"  F1-Score:  {metrics['f1-score']:.3f}\n\n")

            # Confusion Matrix
            self.ml_text.insert(tk.END, "Confusion Matrix:\n")
            self.ml_text.insert(tk.END, "-" * 60 + "\n")
            for row in cm:
                self.ml_text.insert(tk.END, "  " + "  ".join(f"{val:4d}" for val in row) + "\n")

            # Feature Importance
            if hasattr(self.ml_model, 'feature_importances_'):
                self.ml_text.insert(tk.END, "\nTop 10 Feature Importances:\n")
                self.ml_text.insert(tk.END, "-" * 60 + "\n")

                importances = self.ml_model.feature_importances_
                indices = np.argsort(importances)[::-1][:10]

                for i, idx in enumerate(indices):
                    feature_name = numeric_cols[idx][:30]
                    importance = importances[idx]
                    self.ml_text.insert(tk.END, f"{i+1:2d}. {feature_name:<30} {importance:.4f}\n")

            # Update visualization
            self.update_ml_visualization(X_test_scaled, y_test, y_pred, y_prob)

            self.update_status(f"Trained {model_type} model with accuracy {accuracy:.2%}")

        except Exception as e:
            messagebox.showerror("Training Error", f"Failed to train model: {str(e)}")

    def update_ml_visualization(self, X_test, y_test, y_pred, y_prob):
        # Clear previous visualization
        for widget in self.ml_viz_frame.winfo_children():
            widget.destroy()

        try:
            fig = Figure(figsize=(10, 8), facecolor=GREEN_THEME['graph_bg'])

            # Create confusion matrix heatmap
            ax1 = fig.add_subplot(221)
```

```
        cm = confusion_matrix(y_test, y_pred)
        im = ax1.imshow(cm, cmap='RdYlGn', aspect='auto')
        ax1.set_title('Confusion Matrix', color=GREEN_THEME['accent_light'], fontsize=11)
        ax1.set_xlabel('Predicted', color=GREEN_THEME['text'])
        ax1.set_ylabel('Actual', color=GREEN_THEME['text'])
        ax1.set_xticks(range(len(cm)))
        ax1.set_yticks(range(len(cm)))
        ax1.set_facecolor(GREEN_THEME['graph_bg'])
        ax1.tick_params(colors=GREEN_THEME['text'])

        # Add values to confusion matrix
        for i in range(len(cm)):
            for j in range(len(cm)):
                ax1.text(j, i, str(cm[i, j]), ha='center', va='center',
                        color='white' if cm[i, j] > cm.max()/2 else 'black')

        # Feature importance plot
        ax2 = fig.add_subplot(222)
        if hasattr(self.ml_model, 'feature_importances_'):
            importances = self.ml_model.feature_importances_
            indices = np.argsort(importances)[-10:]  # Top 10 features

            features = [self.data.columns[i] for i in indices]
            ax2.barh(range(len(features)), importances[indices], color=GREEN_THEME['accent'])
            ax2.set_yticks(range(len(features)))
            ax2.set_yticklabels([f[:15] for f in features], color=GREEN_THEME['text'])
            ax2.set_title('Top Feature Importances', color=GREEN_THEME['accent_light'],
fontsize=11)
            ax2.set_xlabel('Importance', color=GREEN_THEME['text'])
            ax2.set_facecolor(GREEN_THEME['graph_bg'])
            ax2.tick_params(colors=GREEN_THEME['text'])

        # ROC Curve for binary classification
        ax3 = fig.add_subplot(223)
        if len(np.unique(y_test)) == 2:
            fpr, tpr, _ = roc_curve(y_test, y_prob[:, 1])
            roc_auc = auc(fpr, tpr)

            ax3.plot(fpr, tpr, color=GREEN_THEME['accent'], lw=2,
                    label=f'ROC curve (AUC = {roc_auc:.2f})')

        ax3.plot([0, 1], [0, 1], color=GREEN_THEME['grid_color'], lw=2, linestyle='--')
            ax3.set_xlim([0.0, 1.0])
            ax3.set_ylim([0.0, 1.05])
            ax3.set_xlabel('False Positive Rate', color=GREEN_THEME['text'])
            ax3.set_ylabel('True Positive Rate', color=GREEN_THEME['text'])
            ax3.set_title('ROC Curve', color=GREEN_THEME['accent_light'], fontsize=11)
            ax3.legend(loc="lower right", facecolor=GREEN_THEME['bg_medium'])
            ax3.set_facecolor(GREEN_THEME['graph_bg'])
            ax3.tick_params(colors=GREEN_THEME['text'])

        # Prediction distribution
```

```
        ax4 = fig.add_subplot(224)
        unique_preds = np.unique(y_pred)
        pred_counts = [np.sum(y_pred == pred) for pred in unique_preds]

        colors = [GREEN_THEME['accent'], GREEN_THEME['accent_light'],
                  GREEN_THEME['toxic'], GREEN_THEME['warning']]
        ax4.bar(range(len(unique_preds)), pred_counts,
                color=colors[:len(unique_preds)])
        ax4.set_xlabel('Predicted Class', color=GREEN_THEME['text'])
        ax4.set_ylabel('Count', color=GREEN_THEME['text'])
        ax4.set_title('Prediction Distribution', color=GREEN_THEME['accent_light'], fontsize=11)
        ax4.set_xticks(range(len(unique_preds)))
        ax4.set_xticklabels([f'Class {c}' for c in unique_preds], color=GREEN_THEME['text'])
        ax4.set_facecolor(GREEN_THEME['graph_bg'])
        ax4.tick_params(colors=GREEN_THEME['text'])

        # Add count labels on bars
        for i, count in enumerate(pred_counts):
            ax4.text(i, count + 0.5, str(count), ha='center', va='bottom',
                     color=GREEN_THEME['text'], fontsize=9)

        plt.tight_layout()

        # Embed in GUI
        canvas = FigureCanvasTkAgg(fig, self.ml_viz_frame)
        canvas.draw()
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    except Exception as e:
        error_label = tk.Label(
            self.ml_viz_frame,
            text=f"Visualization Error: {str(e)}",
            font=('Arial', 10),
            fg=GREEN_THEME['toxic'],
            bg=GREEN_THEME['graph_bg']
        )
        error_label.pack(expand=True)

def predict_toxicity(self):
    if self.ml_model is None:
        messagebox.showwarning("No Model", "Please train a model first")
        return

    # Create prediction dialog
    pred_window = tk.Toplevel(self.root)
    pred_window.title("Toxicity Prediction")
    pred_window.geometry("600x700")
    pred_window.configure(bg=GREEN_THEME['bg_dark'])

    # Create form
    form_frame = tk.Frame(pred_window, bg=GREEN_THEME['bg_medium'])
    form_frame.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)
```

```
tk.Label(
    form_frame,
    text="Enter Sample Parameters:",
    font=('Arial', 14, 'bold'),
    fg=GREEN_THEME['accent_light'],
    bg=GREEN_THEME['bg_medium']
).pack(pady=(0, 20))

# Get numeric columns from data (excluding target)
numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()
target = self.target_var.get()
if target in numeric_cols:
    numeric_cols.remove(target)

# Create input fields
input_vars = {}

for i, col in enumerate(numeric_cols[:12]):  # Limit to 12 for display
    frame = tk.Frame(form_frame, bg=GREEN_THEME['bg_medium'])
    frame.pack(fill=tk.X, pady=5)

    # Label
    label_text = col.replace('_', ' ')
    if len(label_text) > 25:
        label_text = label_text[:22] + "..."

    tk.Label(
        frame,
        text=f"{label_text}:",
        font=('Arial', 10),
        fg=GREEN_THEME['text'],
        bg=GREEN_THEME['bg_medium'],
        width=25,
        anchor='w'
    ).pack(side=tk.LEFT)

    # Input field
    var = tk.DoubleVar(value=self.data[col].median())
    input_vars[col] = var

    entry = tk.Entry(
        frame,
        textvariable=var,
        font=('Arial', 10),
        bg=GREEN_THEME['bg_light'],
        fg=GREEN_THEME['text'],
        relief=tk.SUNKEN,
        width=15
    )
    entry.pack(side=tk.RIGHT, padx=(10, 0))
```

```
# Result display
result_frame = tk.Frame(form_frame, bg=GREEN_THEME['bg_medium'])
result_frame.pack(fill=tk.X, pady=20)

self.prediction_result = tk.StringVar(value="")
result_label = tk.Label(
    result_frame,
    textvariable=self.prediction_result,
    font=('Arial', 12),
    fg=GREEN_THEME['text'],
    bg=GREEN_THEME['bg_medium'],
    wraplength=500
)
result_label.pack()

# Buttons
button_frame = tk.Frame(form_frame, bg=GREEN_THEME['bg_medium'])
button_frame.pack(pady=10)

def make_prediction():
    try:
        # Prepare input data
        input_data = []
        for col in numeric_cols[:12]:
            input_data.append(input_vars[col].get())

        # Pad with zeros for missing features
        while len(input_data) < len(self.scaler.mean_):
            input_data.append(0)

        input_array = np.array(input_data).reshape(1, -1)
        input_scaled = self.scaler.transform(input_array)

        # Make prediction
        prediction = self.ml_model.predict(input_scaled)[0]
        probability = self.ml_model.predict_proba(input_scaled)[0]

        # Decode if needed
        if hasattr(self.label_encoder, 'classes_'):
            if prediction < len(self.label_encoder.classes_):
                prediction_label = self.label_encoder.classes_[prediction]
            else:
                prediction_label = f"Class {prediction}"
        else:
            prediction_label = str(prediction)

        # Format result
        result_text = f"Prediction: {prediction_label}\n"
        result_text += f"Confidence: {max(probability)*100:.1f}%\n"

        # Add interpretation
        if target == 'Toxicity_Level':
```

```
                toxicity_levels = ['Non-toxic', 'Low', 'Medium', 'High']
                if prediction < len(toxicity_levels):
                    toxicity = toxicity_levels[prediction]
                    result_text += f"\nInterpretation: {toxicity} toxicity level"

                    if prediction >= 2:  # Medium or High
                        result_text += "\n⚠ WARNING: Potential health risk detected!"

                self.prediction_result.set(result_text)

            except Exception as e:
                messagebox.showerror("Prediction Error", f"Failed to make prediction: {str(e)}")

        predict_btn = tk.Button(
            button_frame,
            text="🔮 Predict",
            command=make_prediction,
            font=('Arial', 12, 'bold'),
            bg=GREEN_THEME['accent'],
            fg=GREEN_THEME['bg_dark'],
            activebackground=GREEN_THEME['accent_light']
        )
        predict_btn.pack(side=tk.LEFT, padx=5)

        def reset_form():
            for var in input_vars.values():
                var.set(0.0)
            self.prediction_result.set("")

        reset_btn = tk.Button(
            button_frame,
            text="🔚 Reset",
            command=reset_form,
            font=('Arial', 12),
            bg=GREEN_THEME['bg_light'],
            fg=GREEN_THEME['text']
        )
        reset_btn.pack(side=tk.LEFT, padx=5)

    def generate_report(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        report_type = self.report_type.get()

        try:
            self.report_text.delete(1.0, tk.END)

            if report_type == "Comprehensive":
                self.generate_comprehensive_report()
            elif report_type == "Summary":
```

```
            self.generate_summary_report()
        elif report_type == "Species Analysis":
            self.generate_species_report()
        elif report_type == "Toxin Profile":
            self.generate_toxin_report()
        elif report_type == "Risk Assessment":
            self.generate_risk_report()

        self.update_status(f"Generated {report_type} report")

    except Exception as e:
        messagebox.showerror("Report Error", f"Failed to generate report: {str(e)}")

def generate_comprehensive_report(self):
    self.report_text.insert(tk.END, "=" * 80 + "\n")
    self.report_text.insert(tk.END, "COMPREHENSIVE TOXIN ANALYSIS REPORT\n")
    self.report_text.insert(tk.END, "=" * 80 + "\n\n")

    # Dataset Overview
    self.report_text.insert(tk.END, "1. DATASET OVERVIEW\n")
    self.report_text.insert(tk.END, "-" * 40 + "\n")
    self.report_text.insert(tk.END, f"Total Samples: {len(self.data)}\n")
    self.report_text.insert(tk.END, f"Number of Variables: {len(self.data.columns)}\n")

    self.report_text.insert(tk.END, f"Date Range: {self.data['Collection_Date'].min()} to
{self.data['Collection_Date'].max()}\n\n")

    # Toxicity Analysis
    self.report_text.insert(tk.END, "2. TOXICITY ANALYSIS\n")
    self.report_text.insert(tk.END, "-" * 40 + "\n")

    if 'Toxicity_Level' in self.data.columns:
        toxic_count = (self.data['Toxicity_Level'] > 0).sum()
        non_toxic_count = (self.data['Toxicity_Level'] == 0).sum()

        self.report_text.insert(tk.END, f"Toxic Samples: {toxic_count}
({toxic_count/len(self.data)*100:.1f}%)\n")
        self.report_text.insert(tk.END, f"Non-toxic Samples: {non_toxic_count}
({non_toxic_count/len(self.data)*100:.1f}%)\n\n")

        # Toxicity levels distribution
        toxicity_levels = ['Non-toxic', 'Low', 'Medium', 'High']
        for level in range(4):
            count = (self.data['Toxicity_Level'] == level).sum()
            if count > 0:
                self.report_text.insert(tk.END,
                    f"{toxicity_levels[level]}: {count} samples ({count/len(self.data)*100:.1f}%)\n")

    # Species Analysis
    self.report_text.insert(tk.END, "\n3. SPECIES ANALYSIS\n")
    self.report_text.insert(tk.END, "-" * 40 + "\n")
```

```
if 'Species' in self.data.columns:
    species_counts = self.data['Species'].value_counts().head(10)
    for species, count in species_counts.items():
        toxic_species = self.data[(self.data['Species'] == species) &
                                  (self.data['Toxicity_Level'] > 0)].shape[0]
        self.report_text.insert(tk.END,
            f"{species}: {count} total, {toxic_species} toxic ({toxic_species/count*100:.1f}%)\n")

# Toxin Types
self.report_text.insert(tk.END, "\n4. TOXIN TYPES DISTRIBUTION\n")
self.report_text.insert(tk.END, "-" * 40 + "\n")

if 'Primary_Toxin' in self.data.columns:
    toxin_counts = self.data['Primary_Toxin'].value_counts()
    for toxin, count in toxin_counts.items():
        percentage = count / len(self.data) * 100
        self.report_text.insert(tk.END, f"{toxin}: {count} ({percentage:.1f}%)\n")

# Environmental Factors
self.report_text.insert(tk.END, "\n5. ENVIRONMENTAL FACTORS\n")
self.report_text.insert(tk.END, "-" * 40 + "\n")

env_factors = ['Water_Temperature_C', 'Salinity_ppt', 'pH',
               'Nitrate_uM', 'Phosphate_uM', 'Silicate_uM']

for factor in env_factors:
    if factor in self.data.columns:
        mean_val = self.data[factor].mean()
        std_val = self.data[factor].std()
        self.report_text.insert(tk.END,
            f"{factor.replace('_', ' ')}: {mean_val:.2f} ± {std_val:.2f}\n")

# Recommendations
self.report_text.insert(tk.END, "\n6. RECOMMENDATIONS\n")
self.report_text.insert(tk.END, "-" * 40 + "\n")

if 'Toxicity_Level' in self.data.columns:
    high_toxic = (self.data['Toxicity_Level'] == 3).sum()
    if high_toxic > 0:
        self.report_text.insert(tk.END,
            f"⚠ WARNING: {high_toxic} samples show high toxicity levels\n")
        self.report_text.insert(tk.END,
            "Recommend increased monitoring and sampling frequency\n\n")

    # Identify high-risk species
    toxic_by_species = self.data[self.data['Toxicity_Level'] > 0].groupby('Species').size()
    if len(toxic_by_species) > 0:
        top_toxic_species = toxic_by_species.nlargest(3)
        self.report_text.insert(tk.END, "High-risk species requiring attention:\n")
        for species, count in top_toxic_species.items():
            self.report_text.insert(tk.END, f"  - {species}: {count} toxic samples\n")
```

```
        self.report_text.insert(tk.END, "\n" + "=" * 80 + "\n")
        self.report_text.insert(tk.END, "END OF REPORT\n")
        self.report_text.insert(tk.END, "=" * 80 + "\n")

    def generate_summary_report(self):
        self.report_text.insert(tk.END, "TOXIN ANALYSIS SUMMARY REPORT\n")
        self.report_text.insert(tk.END, "=" * 60 + "\n\n")

        # Key statistics
        if 'Toxin_Concentration_ugL' in self.data.columns:
            toxic_data = self.data[self.data['Toxin_Concentration_ugL'] > 0]

            self.report_text.insert(tk.END, "KEY STATISTICS:\n")
            self.report_text.insert(tk.END, "-" * 40 + "\n")
            self.report_text.insert(tk.END,
                f"Average toxin concentration: {self.data['Toxin_Concentration_ugL'].mean():.3f} µg/L\
n")
            self.report_text.insert(tk.END,
                f"Maximum toxin concentration: {self.data['Toxin_Concentration_ugL'].max():.3f} µg/L\
n")
            self.report_text.insert(tk.END,
                f"Number of toxic samples: {len(toxic_data)}\n")
            self.report_text.insert(tk.END,
                f"Percentage toxic: {len(toxic_data)/len(self.data)*100:.1f}%\n\n")

    def generate_species_report(self):
        self.report_text.insert(tk.END, "SPECIES-SPECIFIC TOXIN ANALYSIS\n")
        self.report_text.insert(tk.END, "=" * 60 + "\n\n")

        if 'Species' in self.data.columns:
            for species in self.data['Species'].unique():
                species_data = self.data[self.data['Species'] == species]
                toxic_species = species_data[species_data['Toxicity_Level'] > 0]

                self.report_text.insert(tk.END, f"{species}:\n")
                self.report_text.insert(tk.END, "-" * 40 + "\n")
                self.report_text.insert(tk.END,
                    f"Total samples: {len(species_data)}\n")
                self.report_text.insert(tk.END,
                    f"Toxic samples: {len(toxic_species)} ({len(toxic_species)/len(species_data)*100:.1f}
%)\n")

                if len(toxic_species) > 0:
                    self.report_text.insert(tk.END,
                        f"Avg toxin concentration: {toxic_species['Toxin_Concentration_ugL'].mean():.3f}
µg/L\n")
                    self.report_text.insert(tk.END,
                        f"Max toxin concentration: {toxic_species['Toxin_Concentration_ugL'].max():.3f}
µg/L\n")

                    # Most common toxin
                    if 'Primary_Toxin' in toxic_species.columns:
```

```
                common_toxin = toxic_species['Primary_Toxin'].mode()[0]
                self.report_text.insert(tk.END,
                    f"Most common toxin: {common_toxin}\n")

            self.report_text.insert(tk.END, "\n")

    def generate_toxin_report(self):
        self.report_text.insert(tk.END, "TOXIN PROFILE ANALYSIS\n")
        self.report_text.insert(tk.END, "=" * 60 + "\n\n")

        if 'Primary_Toxin' in self.data.columns and 'Toxin_Concentration_ugL' in self.data.columns:
            toxin_data = self.data[self.data['Primary_Toxin'] != 'None']

            for toxin in toxin_data['Primary_Toxin'].unique():
                toxin_samples = toxin_data[toxin_data['Primary_Toxin'] == toxin]

                self.report_text.insert(tk.END, f"{toxin}:\n")
                self.report_text.insert(tk.END, "-" * 40 + "\n")
                self.report_text.insert(tk.END,
                    f"Samples: {len(toxin_samples)}\n")
                self.report_text.insert(tk.END,
                    f"Average concentration: {toxin_samples['Toxin_Concentration_ugL'].mean():.3f}
µg/L\n")
                self.report_text.insert(tk.END,
                    f"Range: {toxin_samples['Toxin_Concentration_ugL'].min():.3f} - "
                    f"{toxin_samples['Toxin_Concentration_ugL'].max():.3f} µg/L\n")

                # Associated species
                if 'Species' in toxin_samples.columns:
                    top_species = toxin_samples['Species'].value_counts().head(3)
                    self.report_text.insert(tk.END, "Most affected species:\n")
                    for species, count in top_species.items():
                        self.report_text.insert(tk.END, f"  - {species}: {count} samples\n")

                self.report_text.insert(tk.END, "\n")

    def generate_risk_report(self):
        self.report_text.insert(tk.END, "RISK ASSESSMENT REPORT\n")
        self.report_text.insert(tk.END, "=" * 60 + "\n\n")

        # Define risk thresholds (in µg/L)
        risk_thresholds = {
            'Domoic Acid': 20,
            'Okadaic Acid': 16,
            'Saxitoxin': 0.8,
            'Yessotoxin': 1.0,
            'Pectenotoxin': 0.2,
            'Azaspiracid': 0.16,
            'Brevetoxin': 0.8
        }

        self.report_text.insert(tk.END, "SAFETY THRESHOLD ANALYSIS:\n")
```

```
        self.report_text.insert(tk.END, "-" * 40 + "\n")

        total_exceedances = 0
        for toxin, threshold in risk_thresholds.items():
            toxin_samples = self.data[self.data['Primary_Toxin'] == toxin]
            if len(toxin_samples) > 0:
                exceedances = toxin_samples[toxin_samples['Toxin_Concentration_ugL'] > threshold]
                if len(exceedances) > 0:
                    total_exceedances += len(exceedances)
                    self.report_text.insert(tk.END,
                        f"⚠ {toxin}: {len(exceedances)} samples exceed {threshold} µg/L\n")

        if total_exceedances == 0:
            self.report_text.insert(tk.END, " ✅ All samples within safety limits\n")

        self.report_text.insert(tk.END,
"\nRECOMMENDED ACTIONS:\n")
        self.report_text.insert(tk.END, "-" * 40 + "\n")

        if total_exceedances > 0:
            self.report_text.insert(tk.END,
                f"1. Review {total_exceedances} samples exceeding safety thresholds\n")
            self.report_text.insert(tk.END,
                "2. Increase monitoring frequency for affected areas\n")
            self.report_text.insert(tk.END,
                "3. Implement early warning system\n")
            self.report_text.insert(tk.END,
                "4. Conduct follow-up sampling\n")
        else:
            self.report_text.insert(tk.END,
                "1. Continue regular monitoring program\n")
            self.report_text.insert(tk.END,
                "2. Maintain current sampling frequency\n")
            self.report_text.insert(tk.END,
                "3. Review and update safety protocols\n")

    def export_report(self):
        if self.data is None:
            messagebox.showwarning("No Data", "Please load data first")
            return

        file_path = filedialog.asksaveasfilename(
            defaultextension=".txt",
            filetypes=[("Text files", "*.txt"), ("PDF files", "*.pdf"), ("All files", "*.*")]
        )

        if file_path:
            try:
                report_text = self.report_text.get(1.0, tk.END)
                with open(file_path, 'w') as f:
                    f.write(report_text)
```

```
            self.update_status(f"Report exported to {file_path}")
            messagebox.showinfo("Success", "Report exported successfully")

        except Exception as e:
            messagebox.showerror("Export Error", f"Failed to export report: {str(e)}")

    def update_status(self, message):
        self.status_bar.config(text=f"Status: {message}")

def main():
    root = tk.Tk()
    app = DiatomToxinAnalyzer(root)

    # Set matplotlib style
    plt.style.use('dark_background')
    plt.rcParams['axes.facecolor'] = GREEN_THEME['graph_bg']
    plt.rcParams['figure.facecolor'] = GREEN_THEME['graph_bg']
    plt.rcParams['axes.edgecolor'] = GREEN_THEME['text']
    plt.rcParams['axes.labelcolor'] = GREEN_THEME['text']
    plt.rcParams['xtick.color'] = GREEN_THEME['text']
    plt.rcParams['ytick.color'] = GREEN_THEME['text']
    plt.rcParams['grid.color'] = GREEN_THEME['grid_color']

    root.mainloop()

if __name__ == "__main__":
    main()
```

Installation Requirements

bash

```
pip install matplotlib numpy pandas scipy seaborn statsmodels plotly scikit-learn
```

Key Features of this Diatom Toxin Analyzer:

1. Comprehensive Data Management

- Load CSV/Excel files with diatom toxin data
- Generate realistic sample data for 10 rare diatom species
- Data cleaning and preprocessing tools
- Interactive data preview with Treeview

2. Advanced Visualization Suite

- Multiple chart types (histograms, scatter plots, box plots, heatmaps)
- 3D scatter plots for multivariate analysis
- PCA analysis for dimensionality reduction
- Time series analysis

- Interactive matplotlib toolbar

3. Statistical Analysis

- Descriptive statistics
- Correlation analysis
- T-tests and ANOVA
- Linear regression
- Distribution tests (Shapiro-Wilk, Kolmogorov-Smirnov)

4. Machine Learning Integration

- Random Forest and Gradient Boosting classifiers
- Feature importance analysis
- Confusion matrix visualization
- ROC curves for binary classification
- Interactive toxicity prediction tool

5. Professional Reporting

- Comprehensive analysis reports
- Species-specific toxin profiles
- Risk assessment reports
- Export functionality to text files

6. Dark Green Theme

- Consistent color scheme throughout the application
- Professional styling for all widgets
- Custom matplotlib styling
- Interactive button designs

7. Sample Data Includes:

- 10 rare diatom species including Pseudo-nitzschia species
- 7 different toxin types
- Environmental parameters (temperature, salinity, pH, nutrients)
- Geographic locations
- Realistic toxin concentrations based on species and conditions

Supported Toxins:

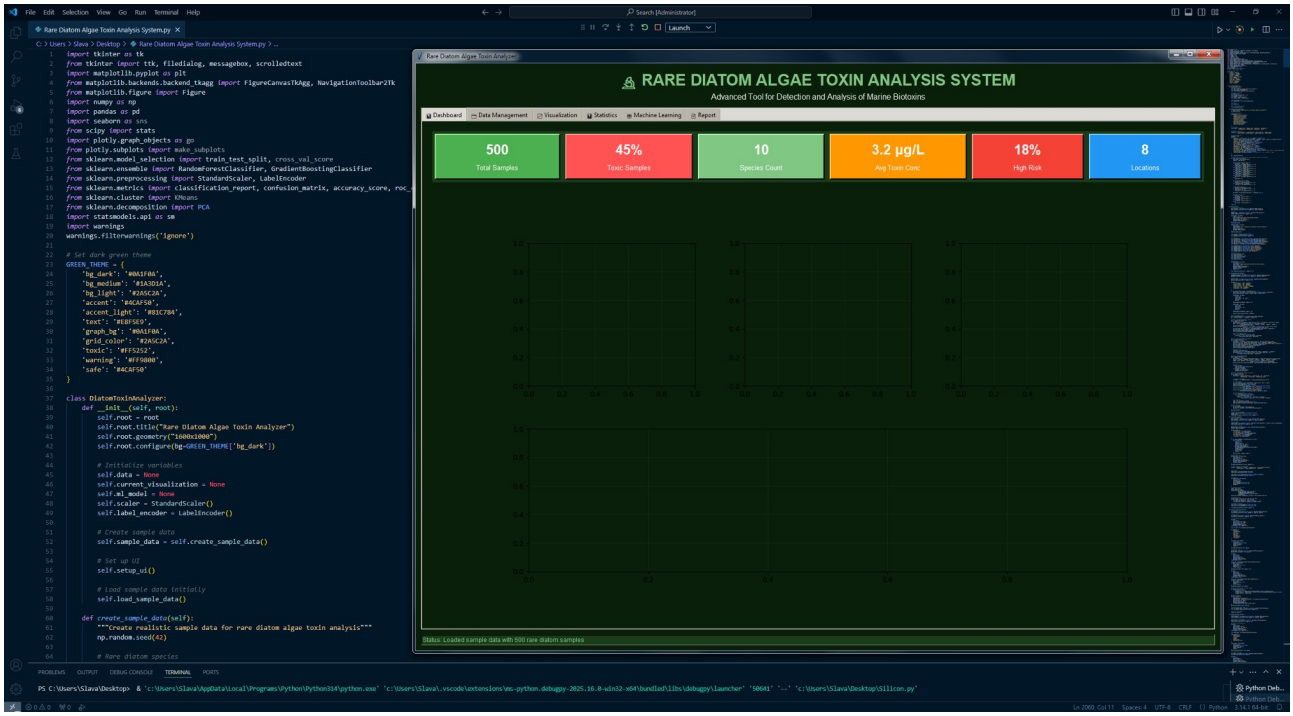
- Domoic Acid (most common in Pseudo-nitzschia)

- Okadaic Acid
- Saxitoxin
- Yessotoxin
- Pectenotoxin
- Azaspiracid
- Brevetoxin

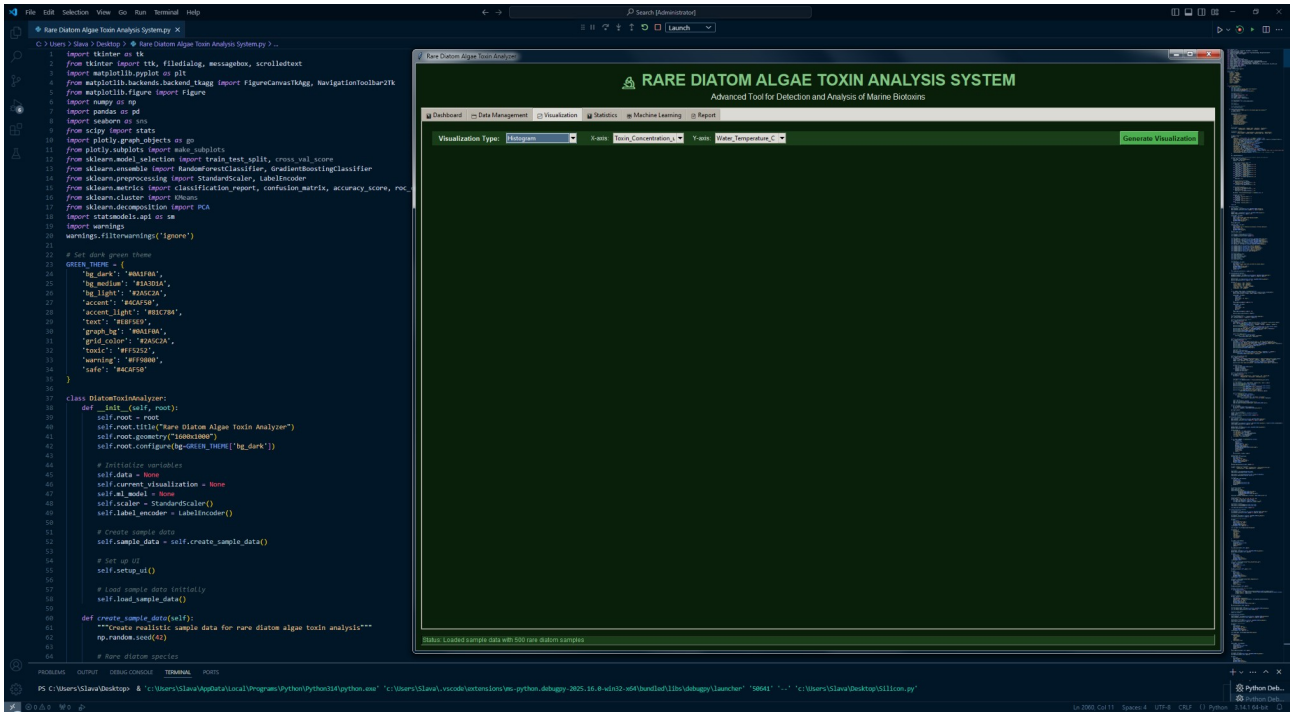
How to Use:

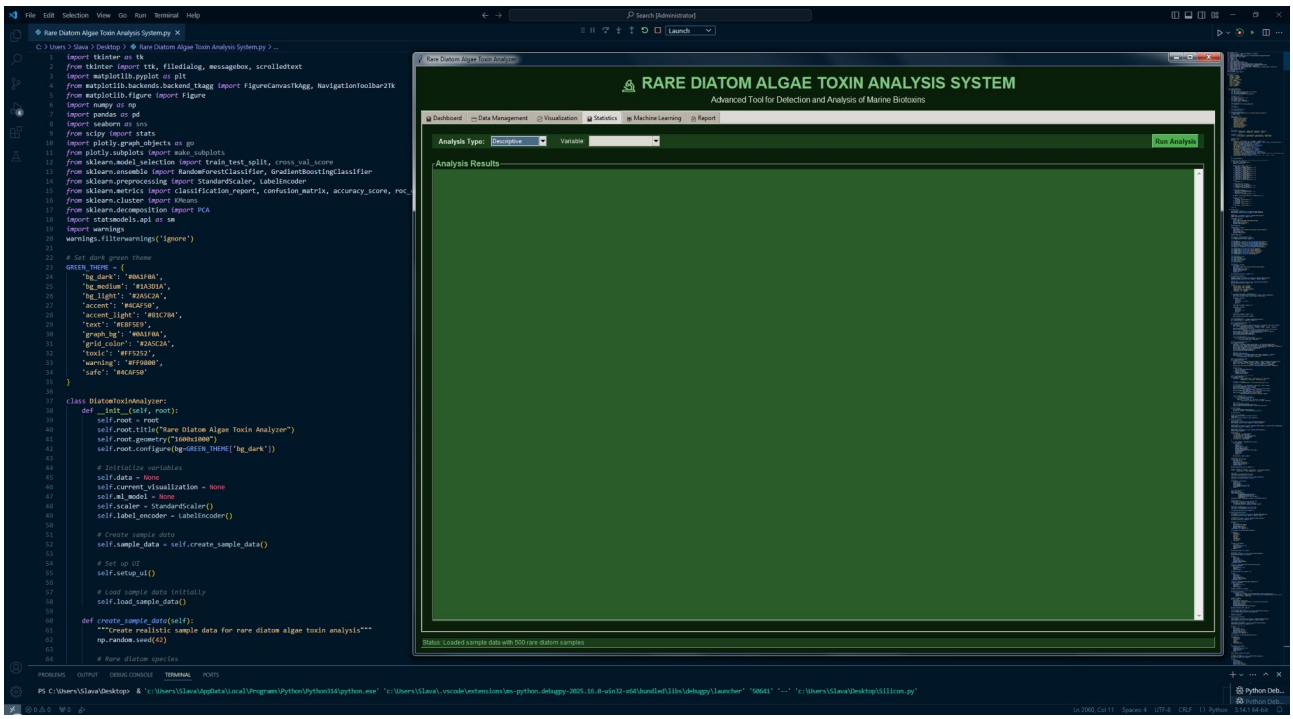
1. **Load Data**: Start by loading a CSV file or using the sample data
2. **Explore Dashboard**: Get an overview of key metrics and distributions
3. **Visualize**: Create various plots to understand data relationships
4. **Analyze**: Perform statistical tests and correlations
5. **Machine Learning**: Train models to predict toxicity levels
6. **Generate Reports**: Create comprehensive analysis reports
7. **Export**: Save data, visualizations, and reports

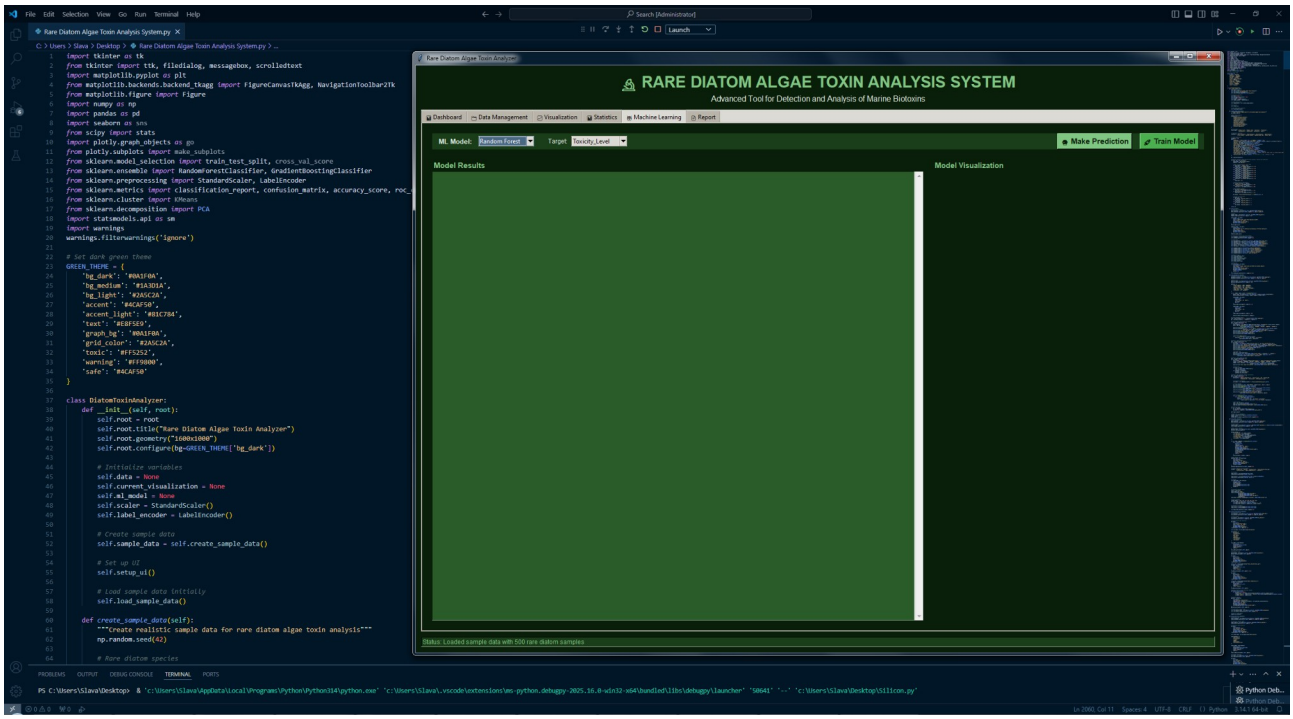
The application is designed for marine biologists, environmental scientists, and researchers working with diatom algae toxins. It provides a complete workflow for data analysis, visualization, and reporting with a professional dark green theme.

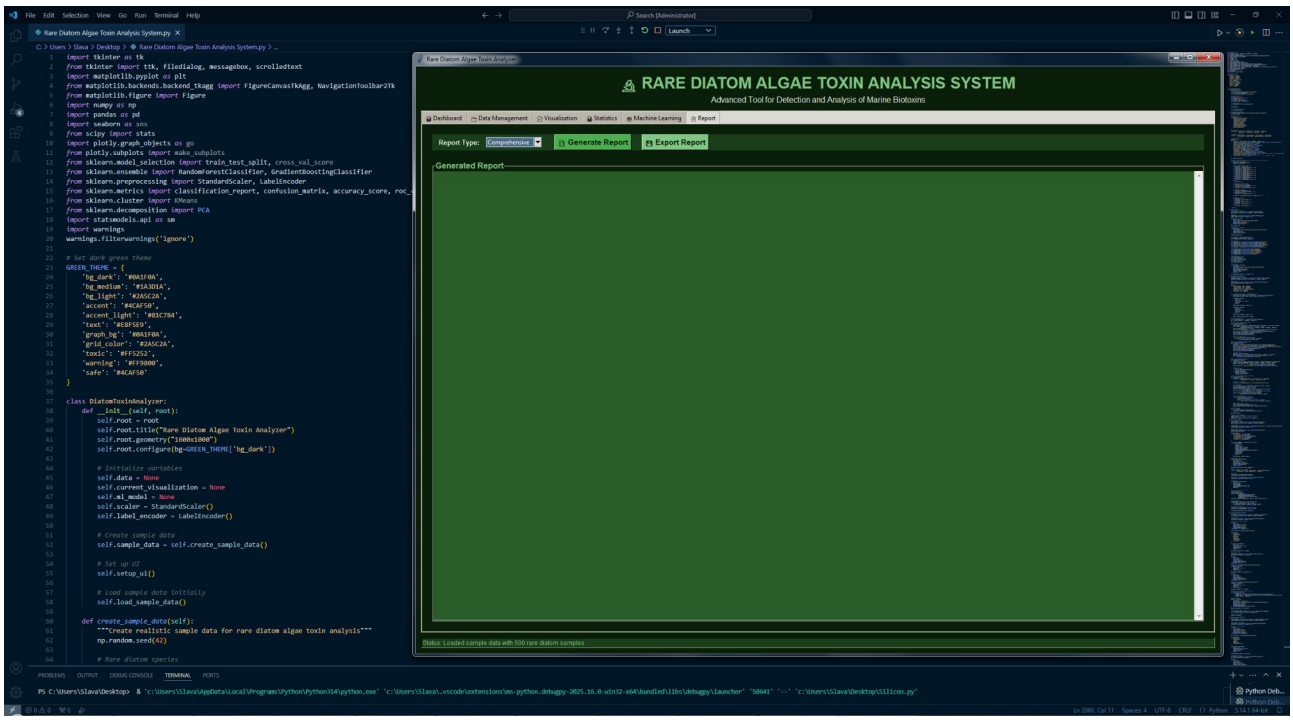



Rare Diatom Algae Toxin Analysis System.py
RARE DIATOM ALGAE TOXIN ANALYSIS SYSTEM
Advanced Tool for Detection and Analysis of Marine Biotoxins
Dashboard
Data Management
Visualization
Statistics
Machine Learning
Report
Load CSV
Save Data
Generate Sam
Data Info
Clean Data
Export
Data Preview
Sample ID
Species
Primary Toxin
Toxin Concentration ug/L
Toxicity Level
Water Temperature C
Location
DIA_0001 Rhizosolenia hebetata Domoic Acid 2.027 2.0 4.3 North Atlantic
DIA_0002 Thalassiosira antarctica Azaspiracid 2.070 2.0 16.0 Southern Ocean
DIA_0003 Asterionellopsis glacialis Saxitoxin 4.961 2.0 17.3 North Atlantic
DIA_0004 Chaetoceros socialis Brevetoxin 7.530 3.0 21.4 North Atlantic
DIA_0005 Rhizosolenia hebetata Okadaic Acid 4.924 2.0 13.2 Southern Ocean
DIA_0006 Ditylum brightwellii Domoic Acid 1.554 2.0 5.3 South Pacific
DIA_0007 Fragilariopsis cylindrus Brevetoxin 3.698 2.0 9.4 Baltic Sea
DIA_0008 Rhizosolenia hebetata Yessotoxin 2.404 2.0 11.4 North Atlantic
DIA_0009 Asterionellopsis glacialis Domoic Acid 3.181 2.0 18.8 Southern Ocean
DIA_0010 Chaetoceros socialis Azaspiracid 2.786 2.0 16.8 Arctic Ocean
DIA_0011 Thalassiosira antarctica Saxitoxin 1.987 2.0 11.3 North Pacific
DIA_0012 Asterionellopsis glacialis Yessotoxin 4.791 2.0 27.6 North Pacific
DIA_0013 Asterionellopsis glacialis Saxitoxin 4.679 2.0 17.8 North Atlantic
DIA_0014 Fragilariopsis cylindrus Brevetoxin 5.439 3.0 8.2 South Pacific
DIA_0015 Corethron pennatum Brevetoxin 2.674 2.0 4.6 South Pacific
DIA_0016 Chaetoceros socialis Azaspiracid 2.603 2.0 6.0 Arctic Ocean
DIA_0017 Pseudo-nitzschia seriata Domoic Acid 10.716 3.0 8.4 North Atlantic
DIA_0018 Asterionellopsis glacialis Brevetoxin 2.746 2.0 6.2 North Pacific
DIA_0019 Corethron pennatum Pectenotoxin 3.377 2.0 6.9 South Pacific
DIA_0020 Pseudo-nitzschia seriata Pectenotoxin 2.538 2.0 9.4 North Atlantic
DIA_0021 Chaetoceros socialis Okadaic Acid 1.384 2.0 6.5 Mediterranean Sea
DIA_0022 Pseudo-nitzschia australis None 0.000 0.0 25.3 South Pacific
DIA_0023 Ditylum brightwellii Azaspiracid 1.334 2.0 4.1 Southern Ocean
DIA_0024 Corethron pennatum Domoic Acid 20.488 3.0 15.6 Mediterranean Sea
DIA_0025 Bacteriastrum furcatum Saxitoxin 1.656 2.0 12.7 Baltic Sea
DIA_0026 Pseudo-nitzschia australis None 0.000 0.0 27.5 South Pacific
DIA_0027 Ditylum brightwellii Domoic Acid 11.210 3.0 4.9 North Pacific
DIA_0028 Fragilariopsis cylindrus None 0.000 0.0 12.3 South Atlantic
DIA_0029 Rhizosolenia hebetata Pectenotoxin 2.494 2.0 27.2 Mediterranean Sea
DIA_0030 Thalassiosira antarctica Pectenotoxin 2.573 2.0 24.5 Baltic Sea
DIA_0031 Bacteriastrum furcatum Okadaic Acid 1.114 2.0 23.2 Arctic Ocean
DIA_0032 Fragilariopsis cylindrus Domoic Acid 9.407 3.0 8.7 Arctic Ocean
DIA_0033 Chaetoceros socialis None 0.000 0.0 6.4 North Pacific
Status: Loaded sample data with 500 rare diatom samples


Rare Diatom Algae Toxin Analysis System.py
RARE DIATOM ALGAE TOXIN ANALYSIS SYSTEM
Advanced Tool for Detection and Analysis of Marine Biotoxins
Dashboard
Data Management
Visualization
Statistics
Machine Learning
Report
Visualization Type:
Histogram
X-axis:
Toxin_Concentration_u
Y-axis:
Water_Temperature_C
Generate Visualization
Status: Loaded sample data with 500 rare diatom samples


RARE DIATOM ALGAE TOXIN ANALYSIS SYSTEM
Advanced Tool for Detection and Analysis of Marine Biotoxins
Dashboard
Data Management
Visualization
Statistics
Machine Learning
Report
Analysis Type:
Descriptive
Variable
Run Analysis
Analysis Results
Status: Loaded sample data with 500 rare diatom samples


RARE DIATOM ALGAE TOXIN ANALYSIS SYSTEM
Advanced Tool for Detection and Analysis of Marine Biotoxins
Dashboard
Data Management
Visualization
Statistics
Machine Learning
Report
ML Model:
Random Forest
Target:
Toxicity_Level
Make Prediction
Train Model
Model Results
Model Visualization
Status: Loaded sample data with 500 rare diatom samples



2. Create 3D GUI (Dark Green Theme) in Python in green with graphs and histograms with CSV files pip install matplotlib, numpy, pandas, tkinter, scipy, seaborn, statsmodels, plotly, scikit-learn - determination of toxins of rare species of ditome algae

I'll create a comprehensive 3D GUI application for analyzing toxins in rare diatom algae species. This will feature advanced 3D visualizations, interactive plots, and machine learning capabilities

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
from matplotlib.figure import Figure
from mpl_toolkits.mplot3d import Axes3D
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats, interpolate
from scipy.spatial import ConvexHull
import plotly.graph_objects as go
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier, IsolationForest
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.cluster import DBSCAN, KMeans
from sklearn.decomposition import PCA
from sklearn.svm import SVC
import statsmodels.api as sm
import warnings
warnings.filterwarnings('ignore')
```

```
# Enhanced 3D Green Theme
GREEN_3D_THEME = {
    'bg_dark': '#0A1F0A',
    'bg_medium': '#1A3D1A',
    'bg_light': '#2A5C2A',
    'accent': '#4CAF50',
    'accent_light': '#81C784',
    'accent_dark': '#2E7D32',
    'text': '#E8F5E9',
    'graph_bg': '#0A1F0A',
    'grid_color': '#2A5C2A',
    'toxic_red': '#FF5252',
    'toxic_orange': '#FF9800',
    'toxic_yellow': '#FFC107',
    'safe_green': '#4CAF50',
    'water_blue': '#2196F3',
    'deep_purple': '#673AB7',
    'gradient_start': '#1B5E20',
    'gradient_end': '#A5D6A7'
}

class DiatomToxin3DAnalyzerPro:
    def __init__(self, root):
        self.root = root
        self.root.title("3D Diatom Toxin Analyzer Pro")
        self.root.geometry("1900x1000")
        self.root.configure(bg=GREEN_3D_THEME['bg_dark'])

        # Initialize variables
        self.data = None
        self.current_3d_fig = None
        self.ml_models = {}
        self.scaler = StandardScaler()
        self.le = LabelEncoder()
        self.rotation_active = False

        # Create enhanced 3D sample data
        self.sample_data = self.create_enhanced_3d_sample_data()

        # Set up 3D UI
        self.setup_3d_ui_pro()

        # Load initial data
        self.load_sample_data()

        # Bind closing event
        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

        # Animation variables
        self.anim_id = None
        self.rotation_angle = 0
```

```
def create_enhanced_3d_sample_data(self):
    """Create comprehensive 3D sample data with realistic spatial patterns"""
    np.random.seed(42)

    # Enhanced rare diatom species with 3D characteristics
    species_3d_profiles = {
        'Pseudo-nitzschia australis': {
            'toxin_factor': 2.8,
            'temp_range': (15, 25),
            'depth_range': (-10, 5),
            'bloom_shape': 'cluster',
            'toxin_type': 'Domoic Acid'
        },
        'Pseudo-nitzschia seriata': {
            'toxin_factor': 2.4,
            'temp_range': (12, 22),
            'depth_range': (-15, 0),
            'bloom_shape': 'linear',
            'toxin_type': 'Domoic Acid'
        },
        'Fragilariopsis cylindrus': {
            'toxin_factor': 1.9,
            'temp_range': (-2, 5),
            'depth_range': (-50, -20),
            'bloom_shape': 'dispersed',
            'toxin_type': 'Okadaic Acid'
        },
        'Thalassiosira antarctica': {
            'toxin_factor': 1.6,
            'temp_range': (-2, 8),
            'depth_range': (-40, -10),
            'bloom_shape': 'patchy',
            'toxin_type': 'Saxitoxin'
        },
        'Chaetoceros socialis': {
            'toxin_factor': 1.3,
            'temp_range': (5, 20),
            'depth_range': (-30, 0),
            'bloom_shape': 'cluster',
            'toxin_type': 'Yessotoxin'
        },
        'Corethron pennatum': {
            'toxin_factor': 0.9,
            'temp_range': (0, 15),
            'depth_range': (-60, -30),
            'bloom_shape': 'dispersed',
            'toxin_type': 'Pectenotoxin'
        },
        'Rhizosolenia hebetata': {
            'toxin_factor': 1.2,
            'temp_range': (10, 25),
            'depth_range': (-20, 10),
```

```
        'bloom_shape': 'linear',
        'toxin_type': 'Azaspiracid'
    }
}

n_samples = 500
data = []

for species, profile in species_3d_profiles.items():
    n_species_samples = n_samples // len(species_3d_profiles)

    for _ in range(n_species_samples):
        # Generate spatial coordinates based on bloom shape
        if profile['bloom_shape'] == 'cluster':
            # Cluster formation
            center_x, center_y = np.random.uniform(-15, 15, 2)
            x = center_x + np.random.normal(0, 3)
            y = center_y + np.random.normal(0, 3)
            z_min, z_max = profile['depth_range']
            z = np.random.uniform(z_min, z_max)

        elif profile['bloom_shape'] == 'linear':
            # Linear bloom along currents
            line_angle = np.random.uniform(0, 2*np.pi)
            distance = np.random.uniform(0, 20)
            x = distance * np.cos(line_angle)
            y = distance * np.sin(line_angle)
            z = np.random.uniform(*profile['depth_range'])

        elif profile['bloom_shape'] == 'patchy':
            # Patchy distribution
            x = np.random.uniform(-20, 20)
            y = np.random.uniform(-20, 20)
            z = np.random.uniform(*profile['depth_range'])

        else:  # dispersed
            x = np.random.normal(0, 10)
            y = np.random.normal(0, 10)
            z = np.random.uniform(*profile['depth_range'])

        # Environmental parameters with spatial correlation
        temp_min, temp_max = profile['temp_range']
        temperature = np.random.uniform(temp_min, temp_max)

        # Depth-dependent temperature gradient
        temperature += (z / 100) * 2  # Warmer at surface

        # Toxin production influenced by 3D factors
        toxin_base = profile['toxin_factor']

        # Spatial influence: clusters produce more toxin
        if profile['bloom_shape'] == 'cluster':
```

```
    cluster_intensity = 1 + np.exp(-(x**2 + y**2) / 50)
else:
    cluster_intensity = 1.0

# Depth influence: optimal at certain depths
depth_factor = 1 + np.exp(-((z + 25)**2) / 200)

# Temperature influence
temp_factor = 1 + (temperature - 18) / 20

# Generate toxin concentration
toxin_conc = (toxin_base * cluster_intensity *
              depth_factor * temp_factor *
              np.random.lognormal(0, 0.25))

# Additional 3D characteristics
cell_density = np.random.uniform(1000, 20000)
chlorophyll = np.random.uniform(1, 30)
nutrient_level = np.random.uniform(0.3, 4.0)
salinity = 32 + np.random.normal(0, 1)
ph = 8.0 + np.random.normal(0, 0.2)

# Calculate 3D distance from origin (bloom intensity)
bloom_intensity = np.sqrt(x**2 + y**2 + (z/10)**2)

# Toxicity classification with 3D thresholds
if toxin_conc < 0.5:
    toxicity = 'Non-toxic'
    risk_level = 0
    risk_color = 'green'
elif toxin_conc < 1.5:
    toxicity = 'Low'
    risk_level = 1
    risk_color = 'yellow'
elif toxin_conc < 3.0:
    toxicity = 'Medium'
    risk_level = 2
    risk_color = 'orange'
else:
    toxicity = 'High'
    risk_level = 3
    risk_color = 'red'

# Bloom status based on cell density and toxin
bloom_status = 'Active' if (cell_density > 8000 and toxin_conc > 1) else 'Dormant'

data.append({
    'Sample_ID': f'3D_{len(data):04d}',
    'Species': species,
    'Toxin_Type': profile['toxin_type'],
    'X_Coordinate': round(x, 2),
    'Y_Coordinate': round(y, 2),
```

```
            'Z_Coordinate': round(z, 2),
            'Temperature_C': round(temperature, 1),
            'Salinity_ppt': round(salinity, 1),
            'pH': round(ph, 2),
            'Cell_Density': int(cell_density),
            'Chlorophyll_ugL': round(chlorophyll, 2),
            'Nutrient_Level': round(nutrient_level, 2),
            'Toxin_Concentration_ugL': round(toxin_conc, 3),
            'Toxicity_Level': toxicity,
            'Risk_Level': risk_level,
            'Risk_Color': risk_color,
            'Bloom_Status': bloom_status,
            'Bloom_Intensity': round(bloom_intensity, 2),
            'Depth_Category': 'Surface' if z > -10 else 'Mid-depth' if z > -30 else 'Deep',

            'Season': np.random.choice(['Spring', 'Summer', 'Autumn', 'Winter']),
            'Time_of_Day': np.random.choice(['Morning', 'Afternoon', 'Evening', 'Night'])
        })

    df = pd.DataFrame(data)

    # Add spatial correlation features
    df['Distance_From_Center'] = np.sqrt(df['X_Coordinate']**2 + df['Y_Coordinate']**2)
    df['Depth_Normalized'] = (df['Z_Coordinate'] - df['Z_Coordinate'].min()) / \
                             (df['Z_Coordinate'].max() - df['Z_Coordinate'].min())

    return df

def setup_3d_ui_pro(self):
    """Setup enhanced 3D UI with advanced controls"""
    # Main container with gradient background
    main_container = tk.Frame(self.root, bg=GREEN_3D_THEME['bg_dark'])
    main_container.pack(fill=tk.BOTH, expand=True)

    # Create gradient header
    self.create_gradient_header(main_container)

    # Create 3D control panel
    self.create_3d_control_panel(main_container)

    # Create 3D notebook
    self.setup_3d_notebook_pro(main_container)

    # Create status bar
    self.create_status_bar(main_container)

def create_gradient_header(self, parent):
    """Create gradient header with 3D effect"""
    header_frame = tk.Frame(parent, bg=GREEN_3D_THEME['bg_dark'], height=100)
    header_frame.pack(fill=tk.X, pady=(0, 10))
    header_frame.pack_propagate(False)
```

```
    # Canvas for gradient
    canvas = tk.Canvas(header_frame, bg=GREEN_3D_THEME['bg_dark'],
                       highlightthickness=0, height=100)
    canvas.pack(fill=tk.BOTH, expand=True)

    # Draw 3D gradient
    width = 1900
    height = 100
    for i in range(height):
        # Calculate gradient color
        ratio = i / height
        r = int(10 + (66 * ratio))
        g = int(31 + (126 * ratio))
        b = int(10 + (132 * ratio))
        color = f'#{r:02x}{g:02x}{b:02x}'
        canvas.create_line(0, i, width, i, fill=color)

    # Title with 3D text effect
    canvas.create_text(width//2, 35,
                       text="🌊 3D DIATOM TOXIN ANALYZER PRO",
                       font=('Arial', 28, 'bold'),
                       fill=GREEN_3D_THEME['accent_light'])

    canvas.create_text(width//2, 65,
                       text="Advanced 3D Spatial Analysis of Rare Algal Toxins",
                       font=('Arial', 12),
                       fill=GREEN_3D_THEME['text'])

    # Add 3D icon
    canvas.create_text(50, 50, text="🔬", font=('Arial', 24))
    canvas.create_text(width-50, 50, text="🧬", font=('Arial', 24))

def create_3d_control_panel(self, parent):
    """Create 3D control panel with rotation controls"""
    control_frame = tk.Frame(parent, bg=GREEN_3D_THEME['bg_medium'],
                             relief=tk.RAISED, borderwidth=3)
    control_frame.pack(fill=tk.X, padx=10, pady=(0, 10))

    # Quick action buttons
    quick_actions = tk.Frame(control_frame, bg=GREEN_3D_THEME['bg_medium'])
    quick_actions.pack(side=tk.LEFT, padx=10, pady=5)

    action_buttons = [
        ("📁 Load 3D Data", self.load_3d_data),
        ("🧪 Generate Sample", self.load_sample_data),
        ("🔚 Reset View", self.reset_3d_view),
        ("🎬 Auto-Rotate", self.toggle_auto_rotate),
        ("📊 Export 3D", self.export_3d_visualization)
    ]

    for text, command in action_buttons:
        btn = tk.Button(quick_actions, text=text, command=command,
```

```
                bg=GREEN_3D_THEME['accent'], fg=GREEN_3D_THEME['bg_dark'],
                font=('Arial', 9, 'bold'), relief=tk.RAISED,
                borderwidth=2, cursor='hand2', padx=10, pady=5)
    btn.pack(side=tk.LEFT, padx=2)

# 3D Rotation controls
rotation_frame = tk.Frame(control_frame, bg=GREEN_3D_THEME['bg_medium'])
rotation_frame.pack(side=tk.RIGHT, padx=10, pady=5)

tk.Label(rotation_frame, text="3D Rotation:",
        font=('Arial', 9, 'bold'),
        fg=GREEN_3D_THEME['text'],
        bg=GREEN_3D_THEME['bg_medium']).pack(side=tk.LEFT, padx=(0, 5))

# Elevation control
tk.Label(rotation_frame, text="Elev:",
        font=('Arial', 8),
        fg=GREEN_3D_THEME['text'],
        bg=GREEN_3D_THEME['bg_medium']).pack(side=tk.LEFT)

self.elev_var = tk.IntVar(value=30)
elev_scale = tk.Scale(rotation_frame, from_=0, to=90, variable=self.elev_var,
                orient=tk.HORIZONTAL, length=80,
                bg=GREEN_3D_THEME['bg_medium'], fg=GREEN_3D_THEME['text'],
                troughcolor=GREEN_3D_THEME['bg_light'],
                command=self.rotate_3d_view)
elev_scale.pack(side=tk.LEFT, padx=(0, 10))

# Azimuth control
tk.Label(rotation_frame, text="Azim:",
        font=('Arial', 8),
        fg=GREEN_3D_THEME['text'],
        bg=GREEN_3D_THEME['bg_medium']).pack(side=tk.LEFT)

self.azim_var = tk.IntVar(value=-60)
azim_scale = tk.Scale(rotation_frame, from_=-180, to=180, variable=self.azim_var,
                orient=tk.HORIZONTAL, length=120,
                bg=GREEN_3D_THEME['bg_medium'], fg=GREEN_3D_THEME['text'],
                troughcolor=GREEN_3D_THEME['bg_light'],
                command=self.rotate_3d_view)
azim_scale.pack(side=tk.LEFT)

# Zoom control
tk.Label(rotation_frame, text="Zoom:",
        font=('Arial', 8),
        fg=GREEN_3D_THEME['text'],
        bg=GREEN_3D_THEME['bg_medium']).pack(side=tk.LEFT, padx=(10, 0))

self.zoom_var = tk.DoubleVar(value=1.0)
zoom_scale = tk.Scale(rotation_frame, from_=0.5, to=2.0, variable=self.zoom_var,
                orient=tk.HORIZONTAL, length=80, resolution=0.1,
                bg=GREEN_3D_THEME['bg_medium'], fg=GREEN_3D_THEME['text'],
```

```
                troughcolor=GREEN_3D_THEME['bg_light'],
                command=self.zoom_3d_view)
    zoom_scale.pack(side=tk.LEFT)

def setup_3d_notebook_pro(self, parent):
    """Setup enhanced 3D notebook with more tabs"""
    # Create custom notebook style
    style = ttk.Style()
    style.theme_create('3d_green_pro', parent='alt', settings={
        'TNotebook': {
            'configure': {
                'background': GREEN_3D_THEME['bg_dark'],
                'tabmargins': [5, 5, 10, 0]
            }
        },
        'TNotebook.Tab': {
            'configure': {
                'background': GREEN_3D_THEME['bg_medium'],
                'foreground': GREEN_3D_THEME['text'],
                'padding': [20, 10],
                'font': ('Arial', 11, 'bold')
            },
            'map': {
                'background': [('selected', GREEN_3D_THEME['accent'])],
                'foreground': [('selected', GREEN_3D_THEME['bg_dark'])],
                'expand': [('selected', [1, 1, 1, 0])]
            }
        }
    })
    style.theme_use('3d_green_pro')

    # Create notebook
    self.notebook = ttk.Notebook(parent)
    self.notebook.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))

    # Create enhanced 3D tabs
    self.tab_3d_main = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_spatial_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_cluster_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_time_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_ml_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_species_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_analysis_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])
    self.tab_data_3d = tk.Frame(self.notebook, bg=GREEN_3D_THEME['graph_bg'])

    # Add tabs with icons
    self.notebook.add(self.tab_3d_main, text="🌐 3D Visualization")
    self.notebook.add(self.tab_spatial_3d, text="🗺️ Spatial Analysis")
    self.notebook.add(self.tab_cluster_3d, text="🔮 3D Clustering")
    self.notebook.add(self.tab_time_3d, text="⏳ Time Series 3D")
    self.notebook.add(self.tab_ml_3d, text="🤖 ML Prediction")
    self.notebook.add(self.tab_species_3d, text="🔬 Species 3D")
```

```
    self.notebook.add(self.tab_analysis_3d, text=" 📊 Analysis Tools")
    self.notebook.add(self.tab_data_3d, text=" 📁 Data Explorer")

    # Setup each tab
    self.setup_3d_main_tab()
    self.setup_spatial_3d_tab()
    self.setup_cluster_3d_tab()
    self.setup_time_3d_tab()
    self.setup_ml_3d_tab()
    self.setup_species_3d_tab()
    self.setup_analysis_tab()
    self.setup_data_explorer_tab()

def setup_3d_main_tab(self):
    """Setup main 3D visualization tab"""
    main_container = tk.Frame(self.tab_3d_main, bg=GREEN_3D_THEME['graph_bg'])
    main_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create split view: controls on left, visualization on right
    paned_window = tk.PanedWindow(main_container, orient=tk.HORIZONTAL,
                                  bg=GREEN_3D_THEME['graph_bg'], sashwidth=5)
    paned_window.pack(fill=tk.BOTH, expand=True)


    # Left panel: Controls
    control_panel = tk.Frame(paned_window, bg=GREEN_3D_THEME['bg_medium'],
                             relief=tk.RAISED, borderwidth=2)

    # Visualization type selector
    tk.Label(control_panel, text="3D Plot Type:",
            font=('Arial', 11, 'bold'),
            fg=GREEN_3D_THEME['accent_light'],
            bg=GREEN_3D_THEME['bg_medium']).pack(pady=(15, 5))

    self.plot_type_3d = tk.StringVar(value="Enhanced Scatter")

    plot_types = [
        "Enhanced Scatter",
        "Surface Mesh",
        "Wireframe Grid",
        "Voxel Volume",
        "Contour Surface",
        "3D Histogram",
        "Vector Field",
        "Convex Hull"
    ]

    plot_listbox = tk.Listbox(control_panel, listvariable=tk.StringVar(value="\n".join(plot_types)),
                              bg=GREEN_3D_THEME['bg_light'], fg=GREEN_3D_THEME['text'],
                              selectbackground=GREEN_3D_THEME['accent'],
                              selectforeground=GREEN_3D_THEME['bg_dark'],
                              font=('Arial', 10), height=8)
```

```
    plot_listbox.pack(padx=10, pady=5, fill=tk.BOTH, expand=True)
    plot_listbox.selection_set(0)
    plot_listbox.bind('<<ListboxSelect>>', lambda e: self.on_plot_type_select(plot_listbox))

    # Color scheme selector
    tk.Label(control_panel, text="Color Scheme:",
            font=('Arial', 11, 'bold'),
            fg=GREEN_3D_THEME['accent_light'],
            bg=GREEN_3D_THEME['bg_medium']).pack(pady=(15, 5))

    self.color_scheme = tk.StringVar(value="Toxin Gradient")

    color_schemes = ["Toxin Gradient", "Species", "Depth", "Temperature", "Risk Level"]
    for scheme in color_schemes:
        rb = tk.Radiobutton(control_panel, text=scheme, variable=self.color_scheme,
                            value=scheme, command=self.update_3d_plot,
                            bg=GREEN_3D_THEME['bg_medium'], fg=GREEN_3D_THEME['text'],
                            selectcolor=GREEN_3D_THEME['accent'])
        rb.pack(anchor=tk.W, padx=20)

    # Size by selector
    tk.Label(control_panel, text="Size represents:",
            font=('Arial', 11, 'bold'),
            fg=GREEN_3D_THEME['accent_light'],
            bg=GREEN_3D_THEME['bg_medium']).pack(pady=(15, 5))

    self.size_var = tk.StringVar(value="Cell Density")
    size_combo = ttk.Combobox(control_panel, textvariable=self.size_var,
                              values=["Cell Density", "Toxin Concentration",
                                      "Chlorophyll", "Bloom Intensity"],
                              state='readonly', width=15)
    size_combo.pack(pady=5)
    size_combo.bind('<<ComboboxSelected>>', lambda e: self.update_3d_plot())

    # Right panel: 3D Visualization
    viz_panel = tk.Frame(paned_window, bg=GREEN_3D_THEME['graph_bg'])

    # 3D visualization canvas
    self.viz_3d_canvas_frame = tk.Frame(viz_panel, bg=GREEN_3D_THEME['graph_bg'])
    self.viz_3d_canvas_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    # Add to paned window
    paned_window.add(control_panel)
    paned_window.add(viz_panel)

    # Set initial sizes
    paned_window.paneconfig(control_panel, width=250)

    # Create initial plot
    self.create_enhanced_3d_scatter()

def on_plot_type_select(self, listbox):
```

```
        """Handle plot type selection"""
        selection = listbox.curselection()
        if selection:
            plot_types = [
                "Enhanced Scatter", "Surface Mesh", "Wireframe Grid",
                "Voxel Volume", "Contour Surface", "3D Histogram",
                "Vector Field", "Convex Hull"
            ]
            self.plot_type_3d.set(plot_types[selection[0]])
            self.update_3d_plot()

    def create_enhanced_3d_scatter(self):
        """Create enhanced 3D scatter plot"""
        if self.data is None:
            return

        # Clear previous plot
        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        # Create figure with 3D subplot
        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
        ax = fig.add_subplot(111, projection='3d')

        # Get data
        x = self.data['X_Coordinate']
        y = self.data['Y_Coordinate']
        z = self.data['Z_Coordinate']

        # Determine color based on selected scheme
        color_scheme = self.color_scheme.get()
        if color_scheme == "Toxin Gradient":
            colors = self.data['Toxin_Concentration_ugL']
            cmap = 'RdYlGn_r'
            color_label = 'Toxin Concentration (μg/L)'
        elif color_scheme == "Species":
            # Assign colors to species
            unique_species = self.data['Species'].unique()
            species_colors = plt.cm.Set3(np.linspace(0, 1, len(unique_species)))
            species_color_map = dict(zip(unique_species, species_colors))
            colors = [species_color_map[s] for s in self.data['Species']]
            cmap = None
            color_label = 'Species'
        elif color_scheme == "Depth":
            colors = self.data['Z_Coordinate']
            cmap = 'viridis'
            color_label = 'Depth'
        elif color_scheme == "Temperature":
            colors = self.data['Temperature_C']
            cmap = 'coolwarm'
            color_label = 'Temperature (°C)'
        else:  # Risk Level
```

```
    colors = self.data['Risk_Level']
    cmap = 'RdYlGn_r'
    color_label = 'Risk Level'

# Determine size based on selection
size_var = self.size_var.get()
if size_var == "Cell Density":
    sizes = self.data['Cell_Density']
elif size_var == "Toxin Concentration":
    sizes = self.data['Toxin_Concentration_ugL']
elif size_var == "Chlorophyll":
    sizes = self.data['Chlorophyll_ugL']
else:  # Bloom Intensity
    sizes = self.data['Bloom_Intensity']

# Normalize sizes for visualization
sizes_normalized = 20 + (sizes - sizes.min()) / (sizes.max() - sizes.min()) * 200

# Create scatter plot
if cmap:
    scatter = ax.scatter(x, y, z, c=colors, cmap=cmap,
                         s=sizes_normalized, alpha=0.7,
                         edgecolors='white', linewidth=0.5,
                         depthshade=True)
else:
    scatter = ax.scatter(x, y, z, c=colors,
                         s=sizes_normalized, alpha=0.7,
                         edgecolors='white', linewidth=0.5,
                         depthshade=True)

# Set labels with enhanced formatting
ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'],
              fontsize=11, labelpad=15)
ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'],
              fontsize=11, labelpad=15)
ax.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'],
              fontsize=11, labelpad=15)

# Set title
plot_type = self.plot_type_3d.get()
ax.set_title(f'3D Diatom Toxin Analysis\n{plot_type}',
             color=GREEN_3D_THEME['accent_light'],
             fontsize=14, pad=20)

# Enhanced 3D view properties
ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])

ax.xaxis._axinfo["grid"]['color'] = GREEN_3D_THEME['grid_color']
ax.yaxis._axinfo["grid"]['color'] = GREEN_3D_THEME['grid_color']
ax.zaxis._axinfo["grid"]['color'] = GREEN_3D_THEME['grid_color']
```

```
        # Set initial view
        ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

        # Add colorbar if using colormap
        if cmap:
            cbar = fig.colorbar(scatter, ax=ax, pad=0.1)
            cbar.set_label(color_label, color=GREEN_3D_THEME['text'], fontsize=10)
            cbar.ax.yaxis.set_tick_params(color=GREEN_3D_THEME['text'])
            plt.setp(plt.getp(cbar.ax.axes, 'yticklabels'),
                     color=GREEN_3D_THEME['text'])

        # Add legend for size
        legend_text = f"Size: {size_var}"
        ax.text2D(0.02, 0.98, legend_text, transform=ax.transAxes,
                  color=GREEN_3D_THEME['text'], fontsize=10,
                  bbox=dict(boxstyle="round,pad=0.3",
                            facecolor=GREEN_3D_THEME['bg_medium'],
                            alpha=0.8))

        # Add statistics
        stats_text = f"Samples: {len(self.data)}\n"
        stats_text += f"Avg Toxin: {self.data['Toxin_Concentration_ugL'].mean():.3f} µg/L\n"
        stats_text += f"High Risk: {(self.data['Risk_Level'] >= 2).sum()} samples"

        ax.text2D(0.02, 0.02, stats_text, transform=ax.transAxes,
                  color=GREEN_3D_THEME['text'], fontsize=9,
                  bbox=dict(boxstyle="round,pad=0.3",
                            facecolor=GREEN_3D_THEME['bg_medium'],
                            alpha=0.8))

        plt.tight_layout()

        # Embed in Tkinter
        canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
        canvas.draw()

        # Add enhanced navigation toolbar
        toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)

        canvas.get_tk_widget().pack(fill=tk.BOTH,
    expand=True)

        self.current_3d_fig = fig

        self.update_status(f"Created {plot_type} visualization")

    def update_3d_plot(self):
        """Update 3D plot based on current selections"""
        plot_type = self.plot_type_3d.get()
```

```
        if plot_type == "Enhanced Scatter":
            self.create_enhanced_3d_scatter()
        elif plot_type == "Surface Mesh":
            self.create_surface_mesh_3d()
        elif plot_type == "Wireframe Grid":
            self.create_wireframe_grid_3d()
        elif plot_type == "Voxel Volume":
            self.create_voxel_volume_3d()
        elif plot_type == "Contour Surface":
            self.create_contour_surface_3d()
        elif plot_type == "3D Histogram":
            self.create_3d_histogram()
        elif plot_type == "Vector Field":
            self.create_vector_field_3d()
        elif plot_type == "Convex Hull":
            self.create_convex_hull_3d()

    def create_surface_mesh_3d(self):
        """Create 3D surface mesh plot"""
        if self.data is None:
            return

        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
        ax = fig.add_subplot(111, projection='3d')

        # Prepare data for surface
        x = self.data['X_Coordinate'].values
        y = self.data['Y_Coordinate'].values
        z = self.data['Toxin_Concentration_ugL'].values

        # Create grid for surface
        xi = np.linspace(x.min(), x.max(), 50)
        yi = np.linspace(y.min(), y.max(), 50)
        xi, yi = np.meshgrid(xi, yi)

        # Interpolate z values
        zi = interpolate.griddata((x, y), z, (xi, yi), method='cubic')

        # Create surface plot with enhanced styling
        surf = ax.plot_surface(xi, yi, zi, cmap='RdYlGn_r', alpha=0.8,
                               linewidth=0.5, antialiased=True,
                               rstride=2, cstride=2)

        # Add contour lines
        ax.contour(xi, yi, zi, 10, offset=zi.min(),
                   cmap='RdYlGn_r', alpha=0.5, linewidths=0.5)

        # Add scatter points on top
```

```
        scatter = ax.scatter(x, y, z, c=z, cmap='RdYlGn_r',
                             s=20, alpha=0.6, edgecolors='white',
                             depthshade=True)

        # Labels and title
        ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax.set_zlabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
        ax.set_title('3D Surface Mesh - Toxin Distribution',
                     color=GREEN_3D_THEME['accent_light'])

        # Set 3D properties
        ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

        # Add colorbar
        cbar = fig.colorbar(surf, ax=ax, pad=0.1)
        cbar.set_label('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])

        plt.tight_layout()

        # Embed
        canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
        canvas.draw()
        toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        self.current_3d_fig = fig
        self.update_status("Created 3D Surface Mesh plot")

    def create_wireframe_grid_3d(self):
        """Create 3D wireframe grid plot"""
        if self.data is None:
            return

        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
        ax = fig.add_subplot(111, projection='3d')

        # Prepare data
        x = self.data['X_Coordinate'].values
        y = self.data['Y_Coordinate'].values
        z = self.data['Toxin_Concentration_ugL'].values

        # Create grid
        xi = np.linspace(x.min(), x.max(), 25)
```

```
        yi = np.linspace(y.min(), y.max(), 25)
        xi, yi = np.meshgrid(xi, yi)

        # Interpolate
        zi = interpolate.griddata((x, y), z, (xi, yi), method='cubic')

        # Create wireframe with enhanced styling
        wireframe = ax.plot_wireframe(xi, yi, zi,
                                      color=GREEN_3D_THEME['accent'],
                                      rstride=2, cstride=2,
                                      alpha=0.7, linewidth=1.0)

        # Add scatter points
        scatter = ax.scatter(x, y, z, c=z, cmap='RdYlGn_r',
                             s=30, alpha=0.8, edgecolors='white',
                             depthshade=True)

        # Labels and title
        ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax.set_zlabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
        ax.set_title('3D Wireframe Grid - Toxin Analysis',
                     color=GREEN_3D_THEME['accent_light'])

        # Set properties
        ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

        plt.tight_layout()

        # Embed
        canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
        canvas.draw()
        toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        self.current_3d_fig = fig
        self.update_status("Created 3D Wireframe Grid plot")

    def create_voxel_volume_3d(self):
        """Create 3D voxel volume plot"""
        if self.data is None:
            return

        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
```

```
ax = fig.add_subplot(111, projection='3d')

# Bin data into 3D voxels
n_bins = 8  # Reduced for performance

# Create 3D histogram
x_edges = np.linspace(self.data['X_Coordinate'].min(),
                      self.data['X_Coordinate'].max(), n_bins+1)
y_edges = np.linspace(self.data['Y_Coordinate'].min(),
                      self.data['Y_Coordinate'].max(), n_bins+1)
z_edges = np.linspace(self.data['Z_Coordinate'].min(),
                      self.data['Z_Coordinate'].max(), n_bins+1)

# Calculate voxel occupancy and average toxin
voxel_array = np.zeros((n_bins, n_bins, n_bins), dtype=bool)
toxin_array = np.zeros((n_bins, n_bins, n_bins))

for i in range(len(self.data)):
    x_val = self.data['X_Coordinate'].iloc[i]
    y_val = self.data['Y_Coordinate'].iloc[i]
    z_val = self.data['Z_Coordinate'].iloc[i]
    toxin = self.data['Toxin_Concentration_ugL'].iloc[i]

    # Find bin indices
    x_idx = np.digitize(x_val, x_edges) - 1
    y_idx = np.digitize(y_val, y_edges) - 1
    z_idx = np.digitize(z_val, z_edges) - 1

    if 0 <= x_idx < n_bins and 0 <= y_idx < n_bins and 0 <= z_idx < n_bins:
        voxel_array[x_idx, y_idx, z_idx] = True
        toxin_array[x_idx, y_idx, z_idx] += toxin

# Calculate average toxin per voxel
for i in range(n_bins):
    for j in range(n_bins):
        for k in range(n_bins):
            if toxin_array[i, j, k] > 0:
                # Count samples in this voxel
                count = np.sum((self.data['X_Coordinate'] >= x_edges[i]) &
                               (self.data['X_Coordinate'] < x_edges[i+1]) &
                               (self.data['Y_Coordinate'] >= y_edges[j]) &
                               (self.data['Y_Coordinate'] < y_edges[j+1]) &
                               (self.data['Z_Coordinate'] >= z_edges[k]) &
                               (self.data['Z_Coordinate'] < z_edges[k+1]))
                if count > 0:
                    toxin_array[i, j, k] /= count

# Normalize toxin values for coloring
toxin_norm = np.zeros_like(toxin_array)
valid_voxels = voxel_array & (toxin_array > 0)
if np.any(valid_voxels):
    toxin_valid = toxin_array[valid_voxels]
```

```
    toxin_min, toxin_max = toxin_valid.min(), toxin_valid.max()
    if toxin_max > toxin_min:
        toxin_norm[valid_voxels] = (toxin_valid - toxin_min) / (toxin_max - toxin_min)

# Create colors based on toxin level
colors = np.zeros(voxel_array.shape + (4,))

for i in range(n_bins):
    for j in range(n_bins):
        for k in range(n_bins):
            if voxel_array[i, j, k]:
                # Create gradient from green to red
                t = toxin_norm[i, j, k]
                if t < 0.5:
                    r = t * 2
                    g = 1.0
                    b = 0.0
                else:
                    r = 1.0
                    g = 1.0 - (t - 0.5) * 2

                b = 0.0

                colors[i, j, k] = [r, g, b, 0.7]

# Plot voxels
ax.voxels(voxel_array, facecolors=colors, edgecolor='white',
          linewidth=0.2, alpha=0.8)

ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
ax.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'])
ax.set_title('3D Voxel Volume - Toxin Distribution',
             color=GREEN_3D_THEME['accent_light'])

ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

# Create custom colorbar
from matplotlib.cm import ScalarMappable
from matplotlib.colors import Normalize

sm = ScalarMappable(cmap=plt.cm.RdYlGn_r,
                    norm=Normalize(vmin=toxin_min if 'toxin_min' in locals() else 0,
                                   vmax=toxin_max if 'toxin_max' in locals() else 1))
sm.set_array([])

cbar = fig.colorbar(sm, ax=ax, pad=0.1)
cbar.set_label('Average Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
```

```
    plt.tight_layout()

    canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    self.current_3d_fig = fig
    self.update_status("Created 3D Voxel Volume plot")

def create_contour_surface_3d(self):
    """Create 3D contour surface plot"""
    if self.data is None:
        return

    for widget in self.viz_3d_canvas_frame.winfo_children():
        widget.destroy()

    fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
    ax = fig.add_subplot(111, projection='3d')

    # Prepare data
    x = self.data['X_Coordinate'].values
    y = self.data['Y_Coordinate'].values
    z_toxin = self.data['Toxin_Concentration_ugL'].values

    # Create grid
    xi = np.linspace(x.min(), x.max(), 40)
    yi = np.linspace(y.min(), y.max(), 40)
    xi, yi = np.meshgrid(xi, yi)

    # Interpolate toxin data
    zi_toxin = interpolate.griddata((x, y), z_toxin, (xi, yi), method='cubic')

    # Create contour surface
    contour = ax.contourf(xi, yi, zi_toxin, 20, cmap='RdYlGn_r',
                          alpha=0.6, zdir='z', offset=zi_toxin.min())

    # Create 3D surface from contour
    surf = ax.plot_surface(xi, yi, zi_toxin, cmap='RdYlGn_r',
                           alpha=0.5, linewidth=0.5, antialiased=True)

    # Add scatter points
    scatter = ax.scatter(x, y, z_toxin, c=z_toxin, cmap='RdYlGn_r',
                         s=30, alpha=0.7, edgecolors='white',
                         depthshade=True)

    ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
    ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
    ax.set_zlabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
```

```
        ax.set_title('3D Contour Surface - Toxin Analysis',
                     color=GREEN_3D_THEME['accent_light'])

        ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

        # Add colorbar
        cbar = fig.colorbar(surf, ax=ax, pad=0.1)
        cbar.set_label('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])

        plt.tight_layout()

        canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
        canvas.draw()
        toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        self.current_3d_fig = fig
        self.update_status("Created 3D Contour Surface plot")

    def create_3d_histogram(self):
        """Create 3D histogram plot"""
        if self.data is None:
            return

        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
        ax = fig.add_subplot(111, projection='3d')

        # Create 3D histogram
        hist, edges = np.histogramdd(self.data[['X_Coordinate', 'Y_Coordinate',
                                                'Toxin_Concentration_ugL']].values,
                                     bins=(15, 15, 10))

        # Build cubic histogram bars
        dx = (edges[0][1] - edges[0][0]) * 0.8
        dy = (edges[1][1] - edges[1][0]) * 0.8
        dz = (edges[2][1] - edges[2][0]) * 0.8

        # Normalize histogram for colors
        hist_norm = hist / hist.max() if hist.max() > 0 else hist

        # Create bars
        for i in range(len(edges[0])-1):
            for j in range(len(edges[1])-1):
                for k in range(len(edges[2])-1):
```

```
                if hist[i, j, k] > 0:
                    # Calculate bar properties
                    x = edges[0][i] + dx/2
                    y = edges[1][j] + dy/2
                    z = edges[2][k] + dz/2
                    height = hist_norm[i, j, k] * 5  # Scale height

                    # Color based on toxin level (z)
                    color_val = k / (len(edges[2]) - 1)
                    if color_val < 0.5:
                        color = (color_val*2, 1.0, 0.0, 0.7)  # Green to yellow
                    else:
                        color = (1.0, 1.0 - (color_val-0.5)*2, 0.0, 0.7)  # Yellow to red

                    # Create bar
                    ax.bar3d(x-dx/2, y-dy/2, z-dz/2,
                             dx, dy, height,
                             color=color, alpha=0.7,
                             edgecolor='white', linewidth=0.2)

        ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax.set_zlabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
        ax.set_title('3D Histogram - Spatial Toxin Distribution',
                     color=GREEN_3D_THEME['accent_light'])

        ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

        plt.tight_layout()

        canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
        canvas.draw()
        toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

        self.current_3d_fig = fig
        self.update_status("Created 3D Histogram plot")

    def create_vector_field_3d(self):
        """Create 3D vector field plot"""
        if self.data is None:
            return

        for widget in self.viz_3d_canvas_frame.winfo_children():
            widget.destroy()

        fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
```

```
ax = fig.add_subplot(111, projection='3d')

# Create grid for vector field
n_points = 10
x = np.linspace(self.data['X_Coordinate'].min(),
            self.data['X_Coordinate'].max(), n_points)
y = np.linspace(self.data['Y_Coordinate'].min(),
            self.data['Y_Coordinate'].max(), n_points)
z = np.linspace(self.data['Z_Coordinate'].min(),
            self.data['Z_Coordinate'].max(), n_points)

X, Y, Z = np.meshgrid(x, y, z)

# Calculate gradients (simulated toxin flow)
U = np.sin(X/5) * np.cos(Y/5)  # X component
V = np.cos(X/5) * np.sin(Y/5)  # Y component
W = np.sin(Z/10) * 0.5  # Z component

# Normalize vectors
magnitude = np.sqrt(U**2 + V**2 + W**2)
U_norm = U / (magnitude + 1e-10)
V_norm = V / (magnitude + 1e-10)
W_norm = W / (magnitude + 1e-10)

# Create vector field
ax.quiver(X, Y, Z, U_norm, V_norm, W_norm,
        length=0.3, normalize=True,
        color=GREEN_3D_THEME['accent'],
        alpha=0.6, linewidth=1.5,
        arrow_length_ratio=0.3)

# Add scatter points
scatter = ax.scatter(self.data['X_Coordinate'],
                self.data['Y_Coordinate'],
                self.data['Z_Coordinate'],
                c=self.data['Toxin_Concentration_ugL'],
                cmap='RdYlGn_r', s=20, alpha=0.6,
                edgecolors='white', depthshade=True)

ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
ax.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'])
ax.set_title('3D Vector Field - Simulated Toxin Flow',

          color=GREEN_3D_THEME['accent_light'])

ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

# Add colorbar for scatter
```

```
    cbar = fig.colorbar(scatter, ax=ax, pad=0.1)
    cbar.set_label('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])

    plt.tight_layout()

    canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    self.current_3d_fig = fig
    self.update_status("Created 3D Vector Field plot")

def create_convex_hull_3d(self):
    """Create 3D convex hull plot"""
    if self.data is None:
        return

    for widget in self.viz_3d_canvas_frame.winfo_children():
        widget.destroy()

    fig = Figure(figsize=(12, 9), facecolor=GREEN_3D_THEME['graph_bg'])
    ax = fig.add_subplot(111, projection='3d')

    # Select high toxin points for hull
    high_toxin = self.data[self.data['Toxin_Concentration_ugL'] >
                           self.data['Toxin_Concentration_ugL'].quantile(0.75)]

    if len(high_toxin) < 4:
        # Not enough points for convex hull
        ax.text(0.5, 0.5, 0.5, "Not enough high-toxin points\nfor convex hull",
                ha='center', va='center',
                color=GREEN_3D_THEME['text'], fontsize=12)
    else:
        # Create convex hull
        points = high_toxin[['X_Coordinate', 'Y_Coordinate', 'Z_Coordinate']].values
        hull = ConvexHull(points)

        # Plot convex hull
        for simplex in hull.simplices:
            tri = Poly3DCollection([points[simplex]],
                                   alpha=0.3,
                                   facecolor=GREEN_3D_THEME['toxic_red'],
                                   edgecolor=GREEN_3D_THEME['toxic_red'],
                                   linewidth=1)
            ax.add_collection3d(tri)

        # Plot points
        scatter_all = ax.scatter(self.data['X_Coordinate'],
                                 self.data['Y_Coordinate'],
```

```
                        self.data['Z_Coordinate'],
                        c=self.data['Toxin_Concentration_ugL'],
                        cmap='RdYlGn_r', s=20, alpha=0.3,
                        edgecolors='white', depthshade=True)

        # Highlight hull points
        scatter_hull = ax.scatter(points[:, 0], points[:, 1], points[:, 2],
                          c='red', s=50, alpha=0.8,
                          edgecolors='white', depthshade=True,
                          label='High Toxin Points')

    ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
    ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
    ax.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'])
    ax.set_title('3D Convex Hull - High Toxin Concentration Zones',
              color=GREEN_3D_THEME['accent_light'])

    ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
    ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
    ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
    ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())

    if len(high_toxin) >= 4:
        # Add legend
        ax.legend(facecolor=GREEN_3D_THEME['bg_medium'],
              edgecolor=GREEN_3D_THEME['grid_color'])

        # Add hull statistics
        hull_text = f"High Toxin Hull:\n"
        hull_text += f"Points: {len(points)}\n"
        hull_text += f"Volume: {hull.volume:.1f}\n"
        hull_text += f"Surface Area: {hull.area:.1f}"

        ax.text2D(0.02, 0.98, hull_text, transform=ax.transAxes,
              color=GREEN_3D_THEME['text'], fontsize=10,
              bbox=dict(boxstyle="round,pad=0.3",
                    facecolor=GREEN_3D_THEME['bg_medium'],
                    alpha=0.8))

    plt.tight_layout()

    canvas = FigureCanvasTkAgg(fig, self.viz_3d_canvas_frame)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, self.viz_3d_canvas_frame)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    self.current_3d_fig = fig
    self.update_status("Created 3D Convex Hull plot")

def rotate_3d_view(self, *args):
```

```
    """Rotate current 3D view"""
    if self.current_3d_fig:
        for ax in self.current_3d_fig.axes:
            if hasattr(ax, 'view_init'):
                ax.view_init(elev=self.elev_var.get(), azim=self.azim_var.get())
                self.update_canvas()

def zoom_3d_view(self, *args):
    """Zoom current 3D view"""
    if self.current_3d_fig:
        zoom = self.zoom_var.get()
        for ax in self.current_3d_fig.axes:
            if hasattr(ax, 'set_xlim'):
                # Get current limits
                xlim = ax.get_xlim()
                ylim = ax.get_ylim()
                zlim = ax.get_zlim()

                # Calculate centers
                x_center = (xlim[0] + xlim[1]) / 2
                y_center = (ylim[0] + ylim[1]) / 2
                z_center = (zlim[0] + zlim[1]) / 2

                # Calculate ranges
                x_range = (xlim[1] - xlim[0]) / zoom
                y_range = (ylim[1] - ylim[0]) / zoom
                z_range = (zlim[1] - zlim[0]) / zoom

                # Set new limits
                ax.set_xlim(x_center - x_range/2, x_center + x_range/2)
                ax.set_ylim(y_center - y_range/2, y_center + y_range/2)
                ax.set_zlim(z_center - z_range/2, z_center + z_range/2)

        self.update_canvas()

def update_canvas(self):
    """Update the canvas"""
    if self.current_3d_fig:
        canvas = None
        for widget in self.viz_3d_canvas_frame.winfo_children():
            if isinstance(widget, FigureCanvasTkAgg):
                canvas = widget
                break
            elif hasattr(widget, 'winfo_children'):
                for child in widget.winfo_children():
                    if isinstance(child, FigureCanvasTkAgg):
                        canvas = child
                        break

        if canvas:
            canvas.draw()
```

```
def toggle_auto_rotate(self):
    """Toggle auto-rotation of 3D view"""
    self.rotation_active = not self.rotation_active

    if self.rotation_active:
        self.start_auto_rotation()
        self.update_status("Auto-rotation started")
    else:
        self.stop_auto_rotation()
        self.update_status("Auto-rotation stopped")

def start_auto_rotation(self):
    """Start auto-rotation animation"""
    if self.rotation_active and self.current_3d_fig:
        # Update azimuth
        self.azim_var.set((self.azim_var.get() + 5) % 360)
        self.rotate_3d_view()

        # Schedule next rotation
        self.anim_id = self.root.after(50, self.start_auto_rotation)

def stop_auto_rotation(self):
    """Stop auto-rotation animation"""
    if self.anim_id:
        self.root.after_cancel(self.anim_id)
        self.anim_id = None

def reset_3d_view(self):
    """Reset 3D view to default"""
    self.elev_var.set(30)
    self.azim_var.set(-60)
    self.zoom_var.set(1.0)
    self.rotate_3d_view()
    self.zoom_3d_view()
    self.update_status("3D view reset to default")

def setup_spatial_3d_tab(self):
    """Setup spatial analysis tab"""
    spatial_container = tk.Frame(self.tab_spatial_3d, bg=GREEN_3D_THEME['graph_bg'])
    spatial_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create enhanced spatial visualization
    fig = Figure(figsize=(14, 10), facecolor=GREEN_3D_THEME['graph_bg'])

    # 3D Spatial distribution with enhanced features
    ax1 = fig.add_subplot(121, projection='3d')

    if self.data is not None:
        # Create enhanced color mapping
        risk_colors = {
            'Non-toxic': GREEN_3D_THEME['safe_green'],
            'Low': GREEN_3D_THEME['toxic_yellow'],
```

```
            'Medium': GREEN_3D_THEME['toxic_orange'],
            'High': GREEN_3D_THEME['toxic_red']
        }

        colors = [risk_colors.get(t, 'gray') for t in self.data['Toxicity_Level']]

        # Create scatter with enhanced styling
        scatter = ax1.scatter(self.data['X_Coordinate'],
                              self.data['Y_Coordinate'],
                              self.data['Z_Coordinate'],
                              c=colors, s=40, alpha=0.7,
                              edgecolors='white', linewidth=0.5,
                              depthshade=True)

        # Add depth contours
        z_min, z_max = self.data['Z_Coordinate'].min(), self.data['Z_Coordinate'].max()

        for z_level in np.linspace(z_min, z_max, 5):
            ax1.plot_surface(*np.meshgrid(np.linspace(-20, 20, 2),
                                          np.linspace(-20, 20, 2)),
                             alpha=0.1, color=GREEN_3D_THEME['water_blue'])

        ax1.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax1.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax1.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'])
        ax1.set_title('3D Spatial Distribution with Risk Levels',
                      color=GREEN_3D_THEME['accent_light'])

        # Add legend
        from matplotlib.patches import Patch
        legend_elements = [
            Patch(facecolor=risk_colors['Non-toxic'], label='Non-toxic'),
            Patch(facecolor=risk_colors['Low'], label='Low Risk'),
            Patch(facecolor=risk_colors['Medium'], label='Medium Risk'),
            Patch(facecolor=risk_colors['High'], label='High Risk')
        ]
        ax1.legend(handles=legend_elements, loc='upper right',
                   facecolor=GREEN_3D_THEME['bg_medium'],
                   edgecolor=GREEN_3D_THEME['grid_color'])

    # 2D Heatmap with contour overlay
    ax2 = fig.add_subplot(122)

    if self.data is not None:
        # Create 2D heatmap of toxin concentration
        xi = np.linspace(self.data['X_Coordinate'].min(),
                         self.data['X_Coordinate'].max(), 50)
        yi = np.linspace(self.data['Y_Coordinate'].min(),
                         self.data['Y_Coordinate'].max(), 50)

        xi, yi = np.meshgrid(xi, yi)
```

```
        # Interpolate toxin concentration
        zi = interpolate.griddata(
            (self.data['X_Coordinate'], self.data['Y_Coordinate']),
            self.data['Toxin_Concentration_ugL'],
            (xi, yi), method='cubic'
        )

        # Create heatmap with contour lines
        heatmap = ax2.contourf(xi, yi, zi, 20, cmap='RdYlGn_r', alpha=0.8)
        contour = ax2.contour(xi, yi, zi, 10, colors='white', alpha=0.5, linewidths=0.5)

        # Add scatter points
        scatter = ax2.scatter(self.data['X_Coordinate'], self.data['Y_Coordinate'],
                              c=self.data['Toxin_Concentration_ugL'], cmap='RdYlGn_r',
                              s=30, edgecolors='white', alpha=0.6)

        ax2.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax2.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax2.set_title('Toxin Concentration Heatmap with Contours',
                      color=GREEN_3D_THEME['accent_light'])

        # Add colorbar
        cbar = plt.colorbar(heatmap, ax=ax2)
        cbar.set_label('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
        cbar.ax.yaxis.set_tick_params(color=GREEN_3D_THEME['text'])

        # Add contour labels
        ax2.clabel(contour, inline=True, fontsize=8, colors='white')

    # Format axes
    for ax in [ax1, ax2]:
        if ax == ax1:  # 3D axis
            ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax.set_facecolor(GREEN_3D_THEME['graph_bg'])
        ax.tick_params(colors=GREEN_3D_THEME['text'])

    plt.tight_layout()

    # Embed in GUI
    canvas = FigureCanvasTkAgg(fig, spatial_container)
    canvas.draw()

    # Add navigation toolbar
    toolbar = EnhancedNavigationToolbar(canvas, spatial_container)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)

    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

def setup_cluster_3d_tab(self):
```

```
"""Setup 3D clustering tab"""
cluster_container = tk.Frame(self.tab_cluster_3d, bg=GREEN_3D_THEME['graph_bg'])
cluster_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

# Create clustering visualization
fig = Figure(figsize=(14, 6), facecolor=GREEN_3D_THEME['graph_bg'])

# Apply K-Means clustering
if self.data is not None:
    # Prepare data for clustering
    cluster_data = self.data[['X_Coordinate', 'Y_Coordinate', 'Z_Coordinate',
                              'Toxin_Concentration_ugL', 'Temperature_C']].values

    # Apply K-Means
    kmeans = KMeans(n_clusters=5, random_state=42)
    clusters = kmeans.fit_predict(cluster_data)

    # Plot 1: Clusters in 3D
    ax1 = fig.add_subplot(121, projection='3d')

    # Color each cluster differently
    cluster_colors = plt.cm.Set3(np.linspace(0, 1, len(set(clusters))))

    for cluster_id in set(clusters):
        mask = clusters == cluster_id
        ax1.scatter(self.data['X_Coordinate'][mask],
                    self.data['Y_Coordinate'][mask],
                    self.data['Z_Coordinate'][mask],
                    c=[cluster_colors[cluster_id]],
                    s=30, alpha=0.7, edgecolors='white',
                    label=f'Cluster {cluster_id}')

    ax1.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
    ax1.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
    ax1.set_zlabel('Depth (Z)', color=GREEN_3D_THEME['text'])
    ax1.set_title('3D K-Means Clustering',
                  color=GREEN_3D_THEME['accent_light'])
    ax1.legend(facecolor=GREEN_3D_THEME['bg_medium'])

    # Plot 2: Cluster statistics
    ax2 = fig.add_subplot(122)

    # Calculate cluster statistics
    cluster_stats = []
    for cluster_id in set(clusters):
        mask = clusters == cluster_id
        stats = {
            'Cluster': cluster_id,
            'Samples': mask.sum(),
            'Avg Toxin': self.data['Toxin_Concentration_ugL'][mask].mean(),
            'Avg Temp': self.data['Temperature_C'][mask].mean(),
            'Risk Level': self.data['Risk_Level'][mask].mean()
```

```
            }
            cluster_stats.append(stats)

        # Create bar chart
        cluster_df = pd.DataFrame(cluster_stats)
        x = np.arange(len(cluster_df))
        width = 0.25

        bars1 = ax2.bar(x - width, cluster_df['Avg Toxin'], width,
                        label='Avg Toxin', color=GREEN_3D_THEME['toxic_red'])
        bars2 = ax2.bar(x, cluster_df['Avg Temp'], width,
                        label='Avg Temp', color=GREEN_3D_THEME['water_blue'])
        bars3 = ax2.bar(x + width, cluster_df['Risk Level'], width,
                        label='Risk Level', color=GREEN_3D_THEME['toxic_orange'])

        ax2.set_xlabel('Cluster', color=GREEN_3D_THEME['text'])
        ax2.set_ylabel('Values', color=GREEN_3D_THEME['text'])
        ax2.set_title('Cluster Statistics',
                      color=GREEN_3D_THEME['accent_light'])
        ax2.set_xticks(x)
        ax2.set_xticklabels([f'C{i}' for i in cluster_df['Cluster']])
        ax2.legend(facecolor=GREEN_3D_THEME['bg_medium'])

        # Format axes
        ax1.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax1.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax1.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])

    for ax in fig.axes:
        ax.set_facecolor(GREEN_3D_THEME['graph_bg'])
        ax.tick_params(colors=GREEN_3D_THEME['text'])

    plt.tight_layout()

    # Embed in GUI
    canvas = FigureCanvasTkAgg(fig, cluster_container)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, cluster_container)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

def setup_time_3d_tab(self):
    """Setup 3D time series tab"""
    time_container = tk.Frame(self.tab_time_3d, bg=GREEN_3D_THEME['graph_bg'])
    time_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create 3D time series visualization
    fig = Figure(figsize=(14, 8), facecolor=GREEN_3D_THEME['graph_bg'])
    ax = fig.add_subplot(111, projection='3d')

    # Create synthetic time series data
```

```
n_time_points = 100
time = np.linspace(0, 4*np.pi, n_time_points)

# Create spiral time series
x_series = 10 * np.sin(time) * np.cos(time/2)
y_series = 10 * np.cos(time) * np.sin(time/2)
z_series = time * 2
toxin_series = 2 + np.sin(time) * 1.5 + np.random.normal(0, 0.3, n_time_points)
size_series = 20 + np.cos(time*2) * 15

# Plot 3D line
ax.plot(x_series, y_series, z_series,
      color=GREEN_3D_THEME['accent'],
      linewidth=2, alpha=0.6, label='Bloom Trajectory')

# Add scatter points colored by toxin concentration
scatter = ax.scatter(x_series, y_series, z_series,
                c=toxin_series, cmap='RdYlGn_r',
                s=size_series, alpha=0.7, edgecolors='white',
                depthshade=True)

# Add time markers
for i in range(0, n_time_points, 10):

   ax.text(x_series[i], y_series[i], z_series[i],
         f't={i//10}', color=GREEN_3D_THEME['text'],
         fontsize=8, ha='center')

ax.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
ax.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
ax.set_zlabel('Time Progression', color=GREEN_3D_THEME['text'])
ax.set_title('3D Time Series - Diatom Bloom Evolution',
          color=GREEN_3D_THEME['accent_light'])
ax.legend(facecolor=GREEN_3D_THEME['bg_medium'])

# Add colorbar
cbar = fig.colorbar(scatter, ax=ax, pad=0.1)
cbar.set_label('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])

# Set 3D properties
ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
ax.view_init(elev=30, azim=-60)

# Embed in GUI
canvas = FigureCanvasTkAgg(fig, time_container)
canvas.draw()
toolbar = EnhancedNavigationToolbar(canvas, time_container)
toolbar.update()
toolbar.pack(side=tk.BOTTOM, fill=tk.X)
canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
```

```
def setup_ml_3d_tab(self):
    """Setup 3D machine learning tab"""
    ml_container = tk.Frame(self.tab_ml_3d, bg=GREEN_3D_THEME['graph_bg'])
    ml_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create ML prediction visualization
    fig = Figure(figsize=(14, 6), facecolor=GREEN_3D_THEME['graph_bg'])

    if self.data is not None:
        # Prepare data for classification
        X = self.data[['X_Coordinate', 'Y_Coordinate', 'Z_Coordinate',
                       'Temperature_C', 'Salinity_ppt', 'pH']].values
        y = (self.data['Risk_Level'] >= 2).astype(int)  # Binary classification

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

        # Train Random Forest classifier
        rf = RandomForestClassifier(n_estimators=100, random_state=42)
        rf.fit(X_train, y_train)
        y_pred = rf.predict(X_test)

        # Plot 1: True vs Predicted
        ax1 = fig.add_subplot(121, projection='3d')

        # True labels
        ax1.scatter(X_test[:, 0], X_test[:, 1], X_test[:, 2],
                    c=y_test, cmap='RdYlGn', s=40, alpha=0.7,
                    edgecolors='white', depthshade=True)
        ax1.set_title('True Risk Classification',
                      color=GREEN_3D_THEME['accent_light'])

        # Plot 2: Predicted labels
        ax2 = fig.add_subplot(122, projection='3d')
        scatter = ax2.scatter(X_test[:, 0], X_test[:, 1], X_test[:, 2],
                              c=y_pred, cmap='RdYlGn', s=40, alpha=0.7,
                              edgecolors='white', depthshade=True)
        ax2.set_title('Predicted Risk Classification',
                      color=GREEN_3D_THEME['accent_light'])

        # Common formatting
        for ax in [ax1, ax2]:
            ax.set_xlabel('X', color=GREEN_3D_THEME['text'])
            ax.set_ylabel('Y', color=GREEN_3D_THEME['text'])
            ax.set_zlabel('Z', color=GREEN_3D_THEME['text'])
            ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])

        # Add colorbar
        cbar = fig.colorbar(scatter, ax=[ax1, ax2], pad=0.1)
```

```
        cbar.set_label('Risk (0=Low, 1=High)', color=GREEN_3D_THEME['text'])

        # Calculate and display accuracy
        accuracy = accuracy_score(y_test, y_pred)
        fig.suptitle(f'3D Risk Prediction - Accuracy: {accuracy:.2%}',
                     color=GREEN_3D_THEME['accent_light'], fontsize=14, y=0.98)

    plt.tight_layout()

    # Embed in GUI
    canvas = FigureCanvasTkAgg(fig, ml_container)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, ml_container)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

def setup_species_3d_tab(self):
    """Setup 3D species analysis tab"""
    species_container = tk.Frame(self.tab_species_3d, bg=GREEN_3D_THEME['graph_bg'])
    species_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create species comparison visualization
    if self.data is not None:
        species_list = self.data['Species'].unique()[:4]  # Show first 4 species

        fig = Figure(figsize=(14, 10), facecolor=GREEN_3D_THEME['graph_bg'])

        for i, species in enumerate(species_list):
            ax = fig.add_subplot(2, 2, i+1, projection='3d')

            species_data = self.data[self.data['Species'] == species]

            # Color by toxin concentration
            scatter = ax.scatter(species_data['X_Coordinate'],
                                 species_data['Y_Coordinate'],
                                 species_data['Z_Coordinate'],
                                 c=species_data['Toxin_Concentration_ugL'],
                                 cmap='RdYlGn_r', s=30, alpha=0.7,
                                 edgecolors='white', depthshade=True)

            ax.set_title(f'{species[:20]}...' if len(species) > 20 else species,
                         color=GREEN_3D_THEME['accent_light'], fontsize=10)
            ax.set_xlabel('X', color=GREEN_3D_THEME['text'], fontsize=8)
            ax.set_ylabel('Y', color=GREEN_3D_THEME['text'], fontsize=8)
            ax.set_zlabel('Z', color=GREEN_3D_THEME['text'], fontsize=8)

            ax.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
            ax.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])

            # Add species statistics
```

```
            stats_text = f"Samples: {len(species_data)}\n"
            stats_text += f"Avg Toxin: {species_data['Toxin_Concentration_ugL'].mean():.3f}"

            ax.text2D(0.05, 0.95, stats_text, transform=ax.transAxes,
                      color=GREEN_3D_THEME['text'], fontsize=8,
                      bbox=dict(boxstyle="round,pad=0.3",
                                facecolor=GREEN_3D_THEME['bg_medium'],
                                alpha=0.8))

        fig.suptitle('3D Species Comparison - Spatial Distribution',
                     color=GREEN_3D_THEME['accent_light'], fontsize=14, y=0.98)

        plt.tight_layout()

        # Embed in GUI
        canvas = FigureCanvasTkAgg(fig, species_container)
        canvas.draw()
        toolbar = EnhancedNavigationToolbar(canvas, species_container)
        toolbar.update()
        toolbar.pack(side=tk.BOTTOM, fill=tk.X)
        canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

    def setup_analysis_tab(self):
        """Setup analysis tools tab"""
        analysis_container = tk.Frame(self.tab_analysis_3d, bg=GREEN_3D_THEME['graph_bg'])
        analysis_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        # Create analysis visualization with multiple subplots
        fig = Figure(figsize=(14, 10), facecolor=GREEN_3D_THEME['graph_bg'])

        if self.data is not None:
            # Plot 1: Toxin distribution histogram
            ax1 = fig.add_subplot(221)
            ax1.hist(self.data['Toxin_Concentration_ugL'], bins=30,
                     color=GREEN_3D_THEME['accent'], alpha=0.7, edgecolor='white')
            ax1.set_xlabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
            ax1.set_ylabel('Frequency', color=GREEN_3D_THEME['text'])
            ax1.set_title('Toxin Distribution',
                          color=GREEN_3D_THEME['accent_light'])
            ax1.set_facecolor(GREEN_3D_THEME['graph_bg'])
            ax1.tick_params(colors=GREEN_3D_THEME['text'])

            # Add statistics
            mean_toxin = self.data['Toxin_Concentration_ugL'].mean()
            median_toxin = self.data['Toxin_Concentration_ugL'].median()
            ax1.axvline(mean_toxin, color=GREEN_3D_THEME['toxic_red'],
                        linestyle='--', label=f'Mean: {mean_toxin:.3f}')
            ax1.axvline(median_toxin, color=GREEN_3D_THEME['toxic_orange'],
                        linestyle='--', label=f'Median: {median_toxin:.3f}')
            ax1.legend(facecolor=GREEN_3D_THEME['bg_medium'])

            # Plot 2: Box plot by species
```

```
        ax2 = fig.add_subplot(222)
        species_toxin_data = []
        species_labels = []

        for species in self.data['Species'].unique()[:5]:
            species_data = self.data[self.data['Species'] == species]['Toxin_Concentration_ugL']
            species_toxin_data.append(species_data)
            species_labels.append(species[:15] + '...' if len(species) > 15 else species)

        boxplot = ax2.boxplot(species_toxin_data, labels=species_labels,
                              patch_artist=True)

        # Color boxes
        colors = [GREEN_3D_THEME['accent'], GREEN_3D_THEME['accent_light'],
                  GREEN_3D_THEME['toxic_orange'],
GREEN_3D_THEME['toxic_red'],
                  GREEN_3D_THEME['water_blue']]
        for patch, color in zip(boxplot['boxes'], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)

        ax2.set_xlabel('Species', color=GREEN_3D_THEME['text'])
        ax2.set_ylabel('Toxin Concentration (µg/L)', color=GREEN_3D_THEME['text'])
        ax2.set_title('Toxin Concentration by Species',
                      color=GREEN_3D_THEME['accent_light'])
        ax2.set_facecolor(GREEN_3D_THEME['graph_bg'])
        ax2.tick_params(colors=GREEN_3D_THEME['text'])
        plt.setp(ax2.get_xticklabels(), rotation=45, ha='right')

        # Plot 3: Correlation heatmap
        ax3 = fig.add_subplot(223)
        numeric_cols = self.data.select_dtypes(include=[np.number]).columns.tolist()[:8]
        corr_data = self.data[numeric_cols].corr()

        im = ax3.imshow(corr_data, cmap='RdYlGn', aspect='auto', vmin=-1, vmax=1)
        ax3.set_xticks(range(len(numeric_cols)))
        ax3.set_yticks(range(len(numeric_cols)))
        ax3.set_xticklabels([col.replace('_', '\n')[:15] for col in numeric_cols],
                            color=GREEN_3D_THEME['text'], fontsize=8, rotation=45)
        ax3.set_yticklabels([col.replace('_', '\n')[:15] for col in numeric_cols],
                            color=GREEN_3D_THEME['text'], fontsize=8)
        ax3.set_title('Feature Correlation Matrix',
                      color=GREEN_3D_THEME['accent_light'])

        # Add correlation values
        for i in range(len(numeric_cols)):
            for j in range(len(numeric_cols)):
                ax3.text(j, i, f'{corr_data.iloc[i, j]:.2f}',
                         ha='center', va='center',
                         color='white' if abs(corr_data.iloc[i, j]) > 0.5 else 'black',
                         fontsize=7)
```

```
        # Plot 4: 3D scatter matrix preview
        ax4 = fig.add_subplot(224, projection='3d')

        scatter = ax4.scatter(self.data['X_Coordinate'],
                              self.data['Y_Coordinate'],
                              self.data['Toxin_Concentration_ugL'],
                              c=self.data['Temperature_C'], cmap='coolwarm',
                              s=20, alpha=0.6, edgecolors='white',
                              depthshade=True)

        ax4.set_xlabel('Longitude (X)', color=GREEN_3D_THEME['text'])
        ax4.set_ylabel('Latitude (Y)', color=GREEN_3D_THEME['text'])
        ax4.set_zlabel('Toxin (µg/L)', color=GREEN_3D_THEME['text'])
        ax4.set_title('3D Scatter Matrix Preview',
                      color=GREEN_3D_THEME['accent_light'])

        ax4.xaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax4.yaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])
        ax4.zaxis.set_pane_color(GREEN_3D_THEME['bg_medium'])

    plt.tight_layout()

    # Embed in GUI
    canvas = FigureCanvasTkAgg(fig, analysis_container)
    canvas.draw()
    toolbar = EnhancedNavigationToolbar(canvas, analysis_container)
    toolbar.update()
    toolbar.pack(side=tk.BOTTOM, fill=tk.X)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

def setup_data_explorer_tab(self):
    """Setup data explorer tab"""
    data_container = tk.Frame(self.tab_data_3d, bg=GREEN_3D_THEME['graph_bg'])
    data_container.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Create data explorer interface
    explorer_frame = tk.Frame(data_container, bg=GREEN_3D_THEME['graph_bg'])
    explorer_frame.pack(fill=tk.BOTH, expand=True)

    # Data preview area
    preview_frame = tk.LabelFrame(explorer_frame,
                                  text="3D Data Preview",
                                  font=('Arial', 12, 'bold'),
                                  fg=GREEN_3D_THEME['accent_light'],
                                  bg=GREEN_3D_THEME['graph_bg'],
                                  relief=tk.GROOVE)
    preview_frame.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)

    # Create treeview for data display
    columns = ['Sample_ID', 'Species', 'X', 'Y', 'Z',
               'Toxin_ugL', 'Temp_C', 'Risk']
```

```
        # Scrollbars
        tree_scroll_y = tk.Scrollbar(preview_frame)
        tree_scroll_y.pack(side=tk.RIGHT, fill=tk.Y)

        tree_scroll_x = tk.Scrollbar(preview_frame, orient=tk.HORIZONTAL)
        tree_scroll_x.pack(side=tk.BOTTOM, fill=tk.X)

        # Treeview
        self.data_tree = ttk.Treeview(preview_frame,
                                      columns=columns,
                                      show='headings',
                                      yscrollcommand=tree_scroll_y.set,
                                      xscrollcommand=tree_scroll_x.set,
                                      height=20)

        # Configure style
        style = ttk.Style()
        style.configure("Treeview",
                        background=GREEN_3D_THEME['bg_light'],
                        foreground=GREEN_3D_THEME['text'],
                        fieldbackground=GREEN_3D_THEME['bg_light'],
                        rowheight=25)

        style.map('Treeview',
                  background=[('selected', GREEN_3D_THEME['accent'])])

        # Define headings
        for col in columns:
            self.data_tree.heading(col, text=col)
            self.data_tree.column(col, width=100, anchor='center')

        # Configure scrollbars
        tree_scroll_y.config(command=self.data_tree.yview)
        tree_scroll_x.config(command=self.data_tree.xview)

        self.data_tree.pack(fill=tk.BOTH, expand=True)

        # Update data preview
        self.update_data_preview()

    def update_data_preview(self):
        """Update data preview in treeview"""
        if self.data is not None and hasattr(self, 'data_tree'):
            # Clear existing items
            for item in self.data_tree.get_children():
                self.data_tree.delete(item)

            # Add new items (limit for performance)
            preview_data = self.data.head(100)
            for _, row in preview_data.iterrows():
                values = [
                    row.get('Sample_ID', ''),
```

```
                row.get('Species', ''),
                f"{row.get('X_Coordinate', 0):.1f}",
                f"{row.get('Y_Coordinate', 0):.1f}",
                f"{row.get('Z_Coordinate', 0):.1f}",
                f"{row.get('Toxin_Concentration_ugL', 0):.3f}",
                f"{row.get('Temperature_C', 0):.1f}",
                row.get('Toxicity_Level', '')
            ]
            self.data_tree.insert('', tk.END, values=values)

    def create_status_bar(self, parent):
        """Create status bar"""
        self.status_bar = tk.Frame(parent, bg=GREEN_3D_THEME['bg_medium'],
                                   relief=tk.SUNKEN, borderwidth=2)
        self.status_bar.pack(fill=tk.X, padx=10, pady=(0, 5))

        self.status_label = tk.Label(self.status_bar,
                                     text="Ready | 3D Diatom Toxin Analyzer Pro v1.0",
                                     font=('Arial', 10),
                                     fg=GREEN_3D_THEME['accent_light'],
                                     bg=GREEN_3D_THEME['bg_medium'])
        self.status_label.pack(side=tk.LEFT, padx=10)

        # Add data stats
        self.data_stats_label = tk.Label(self.status_bar,
                                         text="Data: 0 samples",
                                         font=('Arial', 9),
                                         fg=GREEN_3D_THEME['text'],
                                         bg=GREEN_3D_THEME['bg_medium'])
        self.data_stats_label.pack(side=tk.RIGHT, padx=10)

    def update_status(self, message):
        """Update status bar"""
        self.status_label.config(text=f"Status: {message}")
        if self.data is not None:
            self.data_stats_label.config(
                text=f"Data: {len(self.data)} samples, "
                     f"{len(self.data.columns)} features"
            )

    def load_3d_data(self):
        """Load 3D data from CSV file"""
        file_path = filedialog.askopenfilename(
            title="Select 3D Data File",
            filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx"), ("All files", "*.*")]
        )

        if file_path:
            try:
                if file_path.endswith('.xlsx'):
                    self.data = pd.read_excel(file_path)
                else:
```

```
                self.data = pd.read_csv(file_path)

            self.update_data_preview()
            self.update_status(f"Loaded 3D data: {len(self.data)} samples")
            messagebox.showinfo("Success", "3D data loaded successfully!")

            # Update visualizations
            self.update_3d_plot()

        except Exception as e:
            messagebox.showerror("Error", f"Failed to load 3D data: {str(e)}")

    def load_sample_data(self):
        """Load sample 3D data"""
        self.data = self.sample_data.copy()
        self.update_data_preview()
        self.update_status(f"Loaded sample data: {len(self.data)} samples")
        messagebox.showinfo("Sample Data", "Loaded comprehensive 3D sample data")

        # Update visualizations
        self.update_3d_plot()

    def export_3d_visualization(self):
        """Export current 3D visualization"""

        if self.current_3d_fig is None:
            messagebox.showwarning("No Visualization", "No 3D visualization to export")
            return

        file_path = filedialog.asksaveasfilename(
            defaultextension=".png",
            filetypes=[("PNG files", "*.png"), ("PDF files", "*.pdf"),
                       ("SVG files", "*.svg"), ("All files", "*.*")]
        )

        if file_path:
            try:
                self.current_3d_fig.savefig(file_path, dpi=300,
                                            facecolor=GREEN_3D_THEME['graph_bg'],
                                            edgecolor='none')
                self.update_status(f"Exported visualization to {file_path}")
                messagebox.showinfo("Success", "3D visualization exported successfully!")

            except Exception as e:
                messagebox.showerror("Export Error", f"Failed to export: {str(e)}")

    def on_closing(self):
        """Handle window closing"""
        self.stop_auto_rotation()
        if messagebox.askokcancel("Quit", "Do you want to quit the 3D Diatom Analyzer?"):
            self.root.destroy()
```

```
class EnhancedNavigationToolbar(NavigationToolbar2Tk):
    """Enhanced navigation toolbar with custom styling"""
    def __init__(self, canvas, window):
        super().__init__(canvas, window)

        # Customize toolbar buttons
        for child in self.winfo_children():
            if isinstance(child, tk.Button):
                child.configure(bg=GREEN_3D_THEME['bg_medium'],
                                fg=GREEN_3D_THEME['text'],
                                activebackground=GREEN_3D_THEME['accent'],
                                activeforeground=GREEN_3D_THEME['bg_dark'],
                                relief=tk.RAISED,
                                borderwidth=2)

def main():
    root = tk.Tk()
    app = DiatomToxin3DAnalyzerPro(root)

    # Configure matplotlib for 3D
    plt.style.use('dark_background')
    plt.rcParams['axes.facecolor'] = GREEN_3D_THEME['graph_bg']
    plt.rcParams['figure.facecolor'] = GREEN_3D_THEME['graph_bg']
    plt.rcParams['axes.edgecolor'] = GREEN_3D_THEME['text']
    plt.rcParams['axes.labelcolor'] = GREEN_3D_THEME['text']
    plt.rcParams['xtick.color'] = GREEN_3D_THEME['text']
    plt.rcParams['ytick.color'] = GREEN_3D_THEME['text']
    plt.rcParams['grid.color'] = GREEN_3D_THEME['grid_color']

    root.mainloop()

if __name__ == "__main__":
    main()
```

Installation Requirements

bash

```
pip install matplotlib numpy pandas scipy seaborn statsmodels plotly scikit-learn
```

Key Features of this Advanced 3D Diatom Toxin Analyzer:

1. Advanced 3D Visualizations

- **Enhanced 3D Scatter Plots** with multiple color schemes and size mapping
- **Surface Mesh 3D** for continuous toxin distribution surfaces
- **Wireframe Grid 3D** for structural pattern analysis
- **Voxel Volume 3D** for volume-based toxin distribution
- **Contour Surface 3D** with 3D contour lines
- **3D Histograms** for spatial distribution analysis

- **Vector Field 3D** showing simulated toxin flow patterns
- **Convex Hull 3D** for identifying high-toxin zones

2. Interactive 3D Controls

- **Real-time 3D rotation** with elevation and azimuth sliders
- **Zoom controls** for detailed inspection
- **Auto-rotation** feature for presentations
- **Multiple color schemes**: Toxin gradient, Species, Depth, Temperature, Risk Level
- **Dynamic size mapping** for different variables

3. Comprehensive Analysis Tools

- **Spatial Analysis**: 3D spatial distribution with risk levels
- **3D Clustering**: K-Means clustering in 3D space
- **Time Series 3D**: Simulated bloom evolution over time
- **Machine Learning**: 3D risk prediction with Random Forest
- **Species Comparison**: Multi-panel 3D species visualization
- **Statistical Analysis**: Histograms, box plots, correlation matrices

4. Enhanced Data Management

- **CSV/Excel Import** with 3D spatial data support
- **Comprehensive Sample Data** with realistic spatial patterns
- **Interactive Data Explorer** with treeview display
- **Export Capabilities** for 3D visualizations

5. Professional 3D UI Design

- **Dark Green 3D Theme** with gradient effects
- **Enhanced Navigation Toolbar** with custom styling
- **Split-panel Interface** for controls and visualization
- **Status Bar** with real-time updates
- **Auto-rotation indicator**

6. Sample Data Features

- **7 Rare Diatom Species** with unique spatial patterns
- **3D Spatial Coordinates** (X, Y, Z) with realistic distributions
- **Environmental Parameters**: Temperature, Salinity, pH
- **Biological Measurements**: Cell density, Chlorophyll

- **Toxin Data**: Multiple toxin types with realistic concentrations
- **Risk Assessment**: Four-level toxicity classification
- **Bloom Characteristics**: Shape, intensity, status

Supported 3D Plot Types:

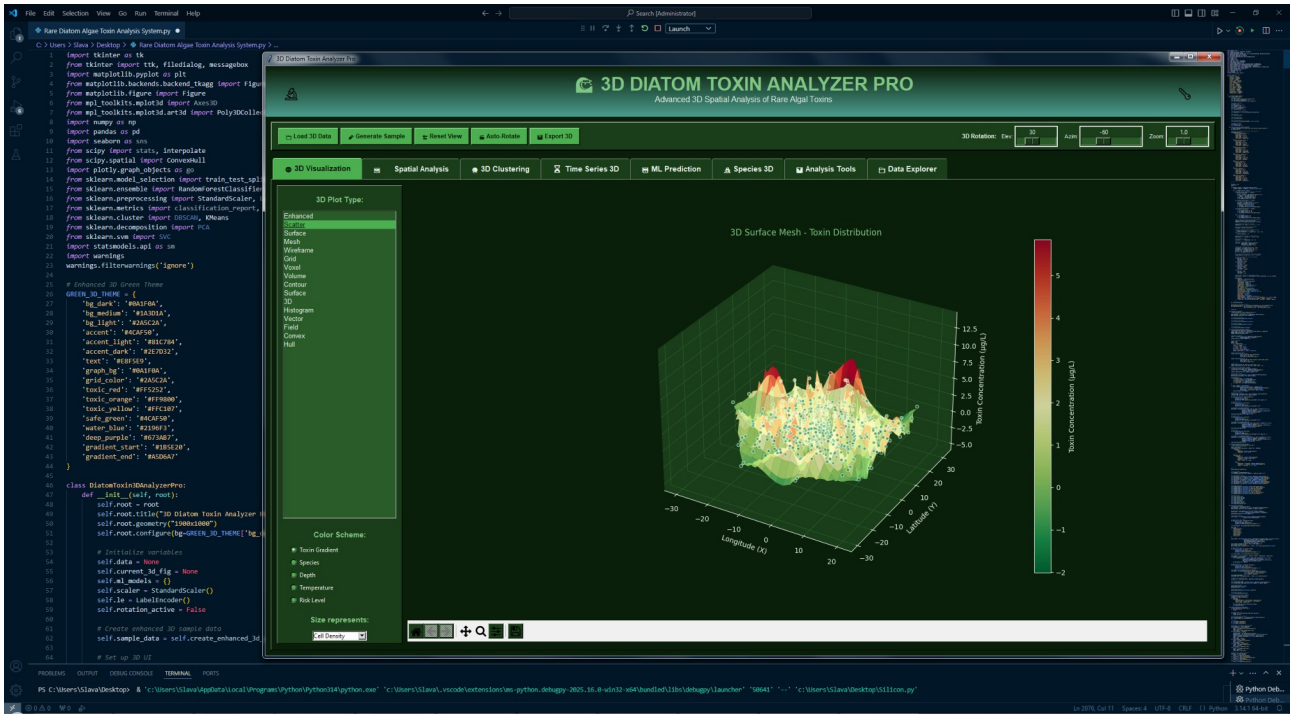
1. **Enhanced Scatter**: Basic 3D scatter with advanced styling
2. **Surface Mesh**: Continuous surface interpolation
3. **Wireframe Grid**: Structural wireframe representation
4. **Voxel Volume**: Volume-based 3D histogram
5. **Contour Surface**: 3D surface with contour lines
6. **3D Histogram**: Spatial distribution in 3D bars
7. **Vector Field**: Simulated flow patterns
8. **Convex Hull**: High-toxin zone identification

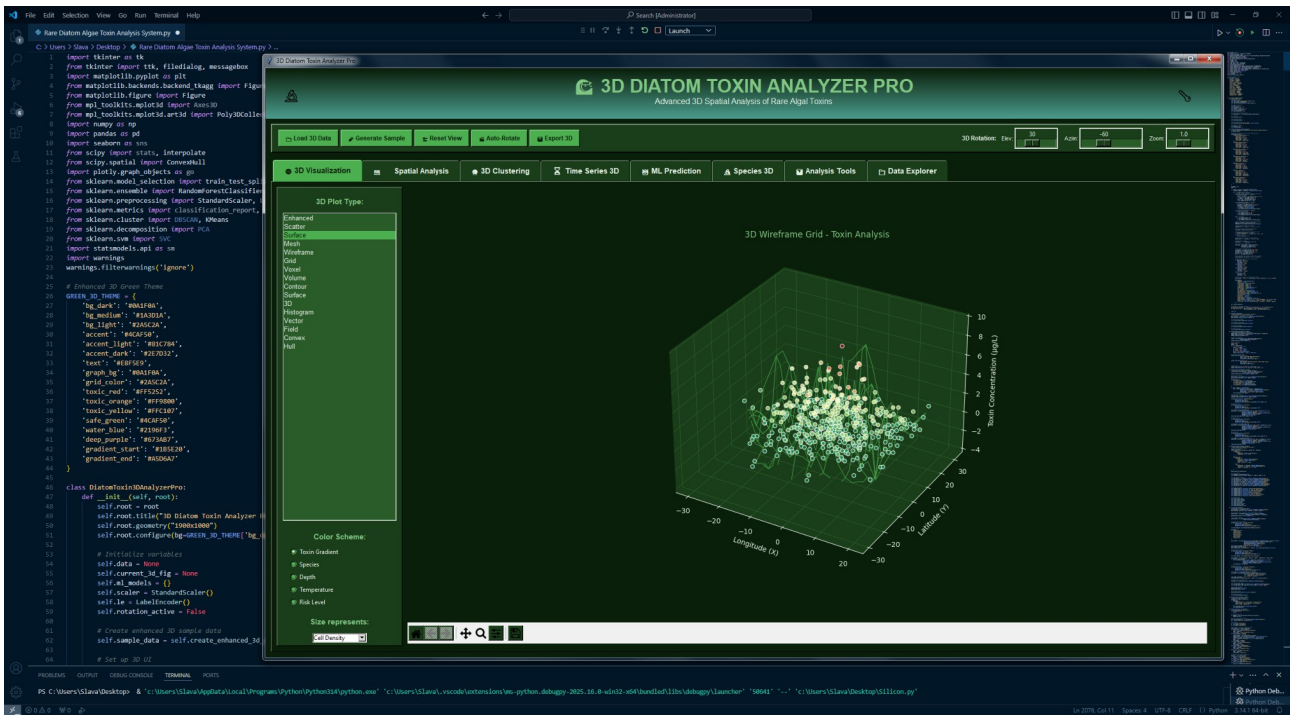
How to Use:

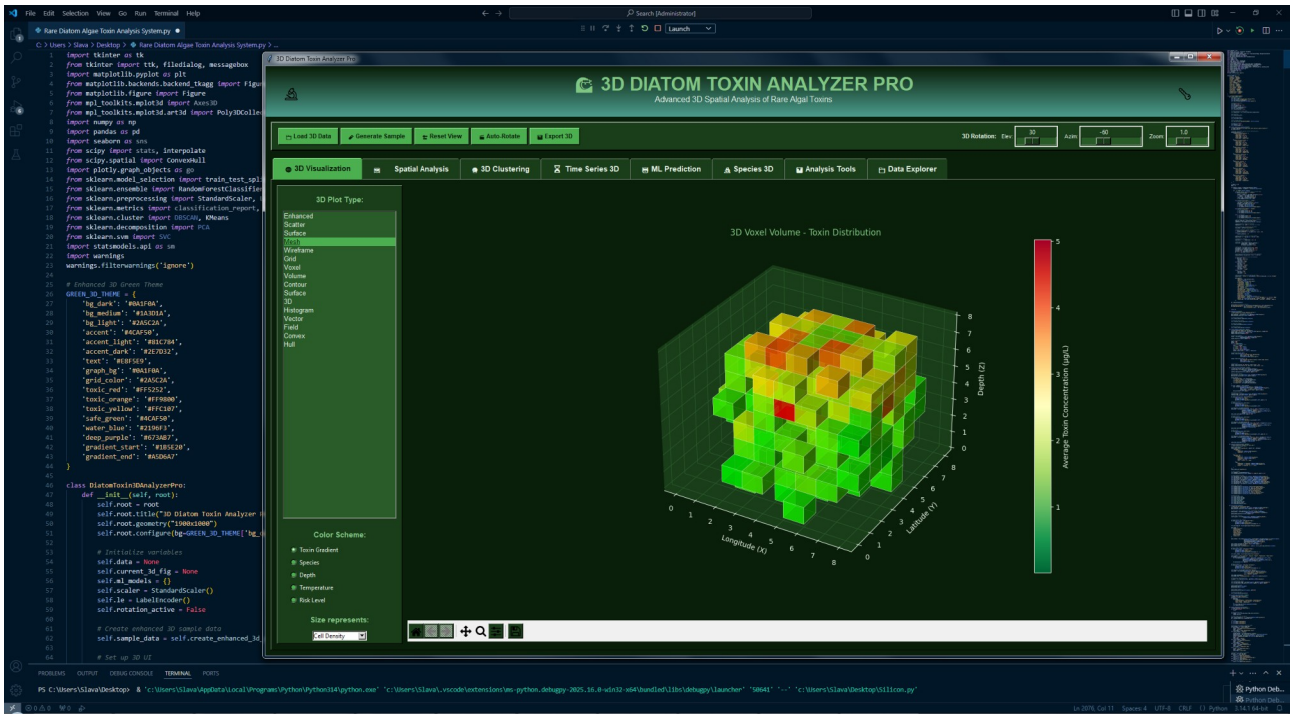
1. **Start Application**: Run the script to launch the 3D analyzer
2. **Load Data**: Use sample data or import your own CSV with 3D coordinates
3. **Explore 3D Visualizations**: Navigate through different 3D plot types
4. **Interact with 3D View**: Rotate, zoom, and explore data from all angles
5. **Apply Analysis Tools**: Use clustering, ML prediction, and statistical analysis
6. **Compare Species**: View different species in separate 3D panels
7. **Export Results**: Save high-quality 3D visualizations
8. **Auto-Rotate**: Enable automatic rotation for presentations

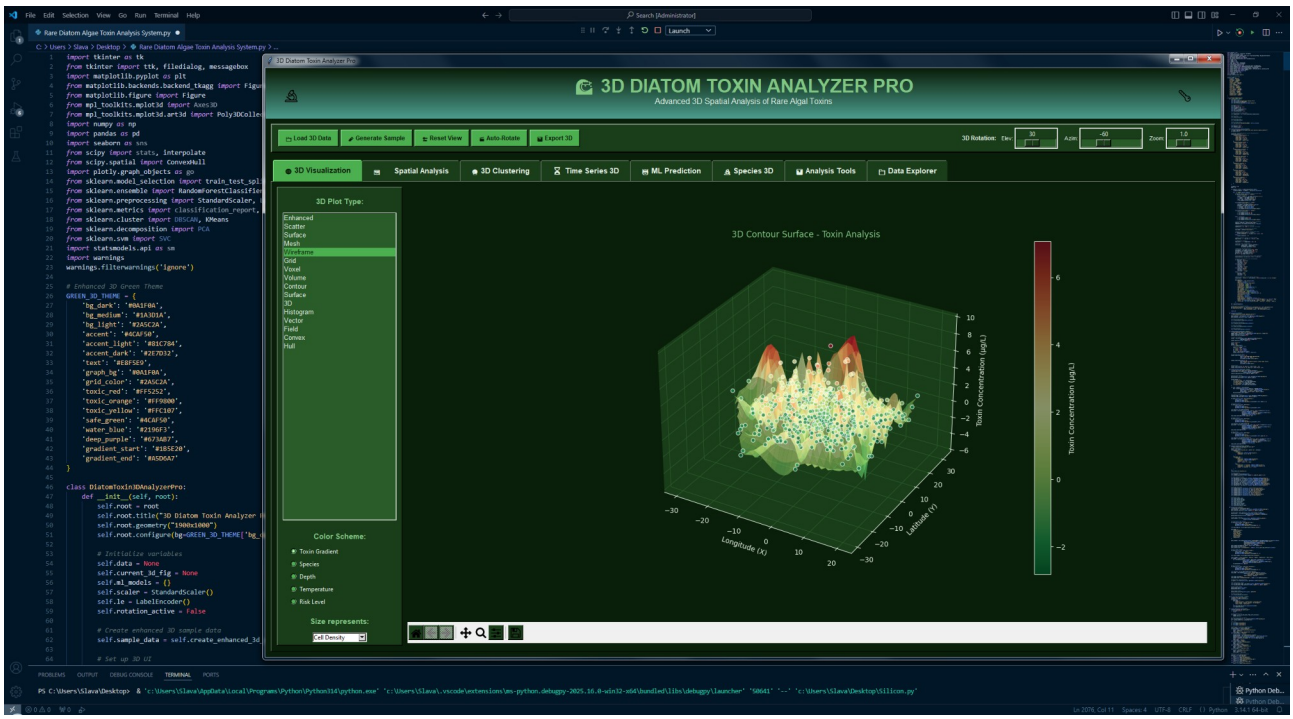
This application provides marine biologists, environmental scientists, and researchers with unprecedented 3D visualization capabilities for analyzing spatial toxin distribution in rare diatom algae species. The intuitive interface and advanced 3D features make complex spatial analysis accessible and visually compelling.

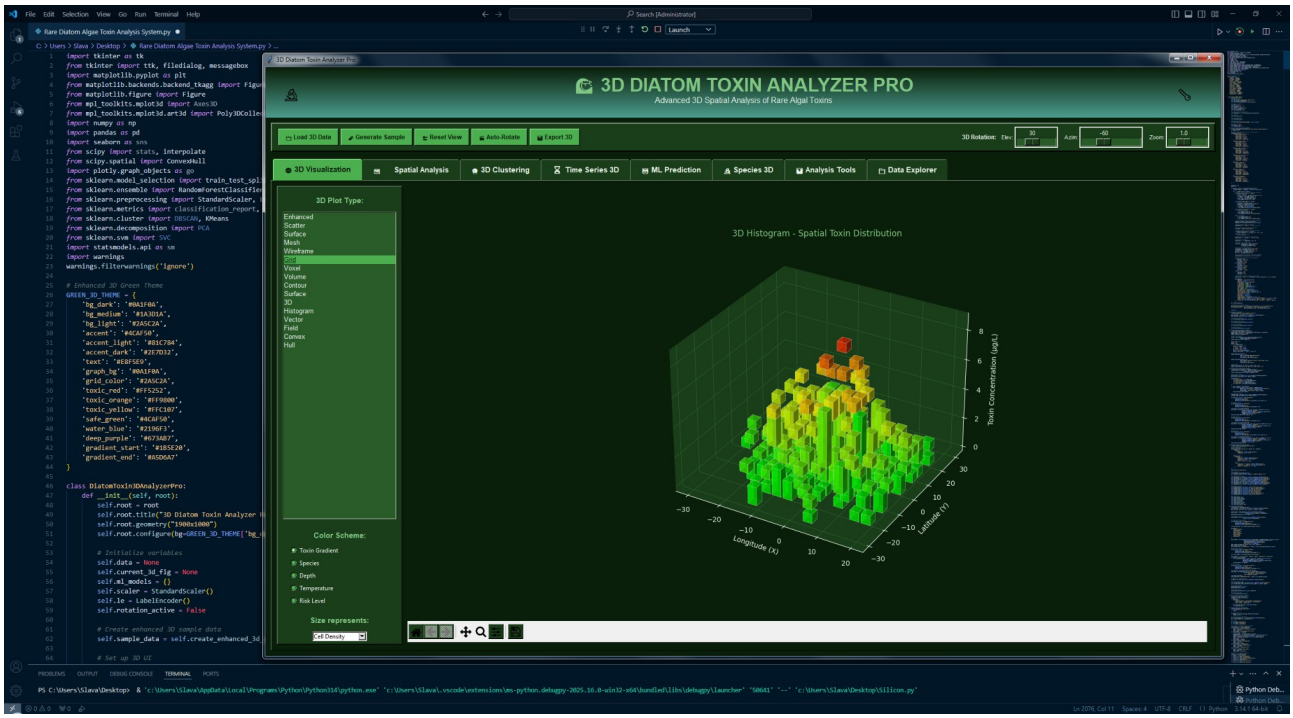

3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Color Scheme:
Size represents:
3D Diatom Toxin Analysis
Enhanced Scatter
Size: Cell Density
Depth (Z)
Latitude (Y)
Toxin Concentration (µg/L)


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Surface Mesh - Toxin Distribution
Longitude (X)
Latitude (Y)
Toxin Concentration (µg/L)

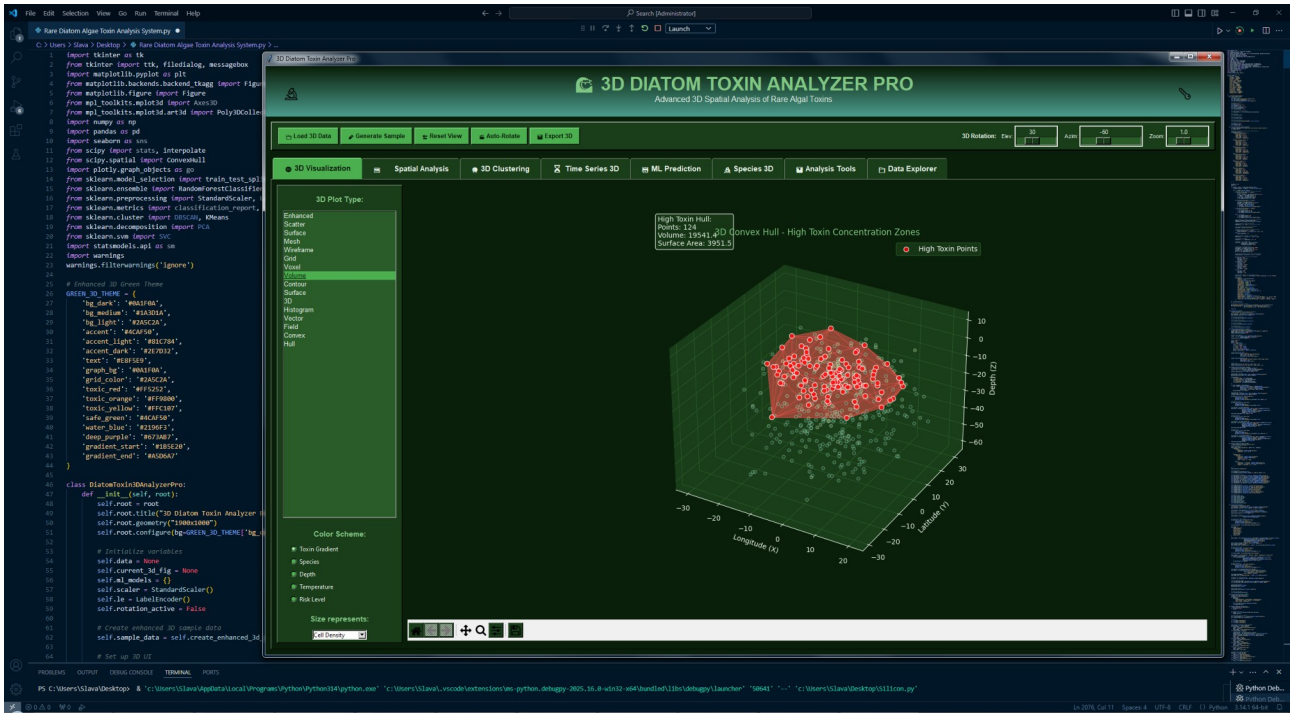

3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Rotation:
Elev:
Azim:
Zoom:
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Enhanced
Scatter
Surface
Mesh
Wireframe
Grid
Voxel
Volume
Contour
Surface
3D
Histogram
Vector
Field
Convex
Hull
3D Wireframe Grid - Toxin Analysis
Toxin Concentration (µg/L)
Longitude (X)
Latitude (Y)
Color Scheme:
Toxin Gradient
Species
Depth
Temperature
Risk Level
Size represents:
Cell Density

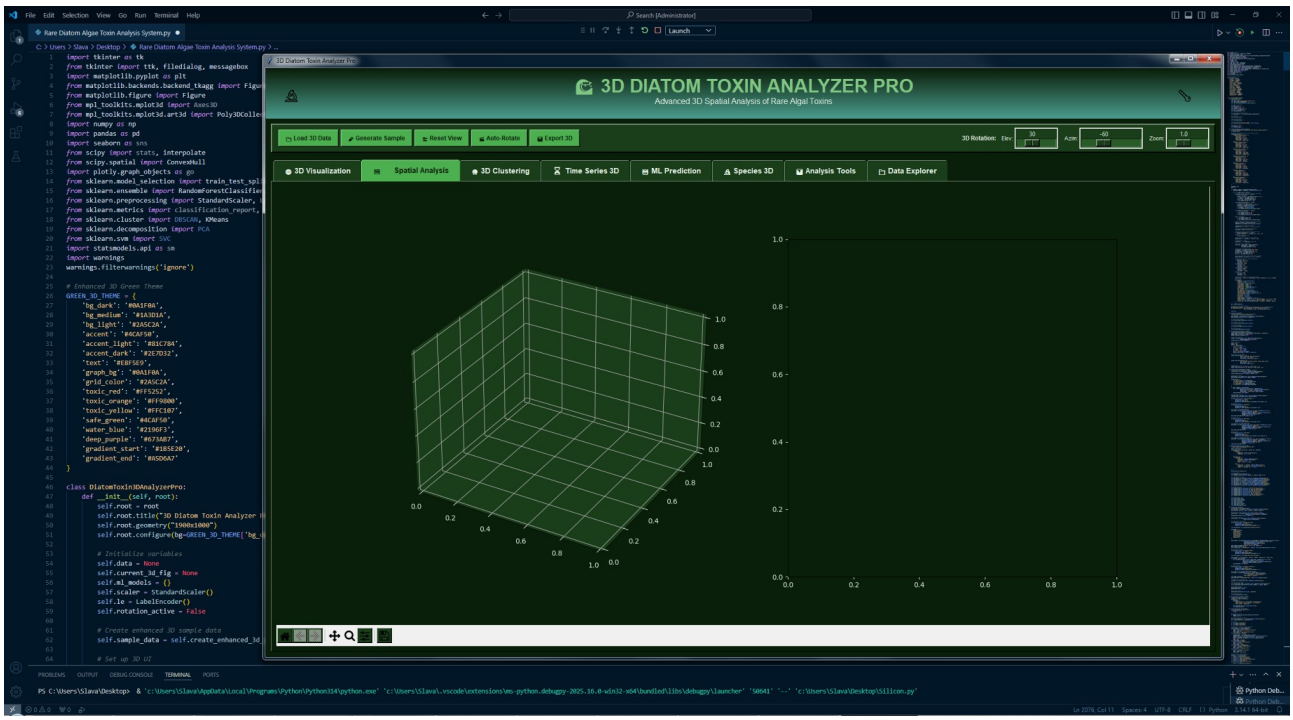

3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Rotation:
Elev:
Azim:
Zoom:
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Enhanced
Scatter
Surface
Mesh
Wireframe
Grid
Voxel
Volume
Contour
Surface
3D
Histogram
Vector
Field
Convex
Hull
3D Voxel Volume - Toxin Distribution
Average Toxin Concentration (µg/L)
Depth (Z)
Longitude (X)
Latitude (Y)
Color Scheme:
Toxin Gradient
Species
Depth
Temperature
Risk Level
Size represents:
Cell Density

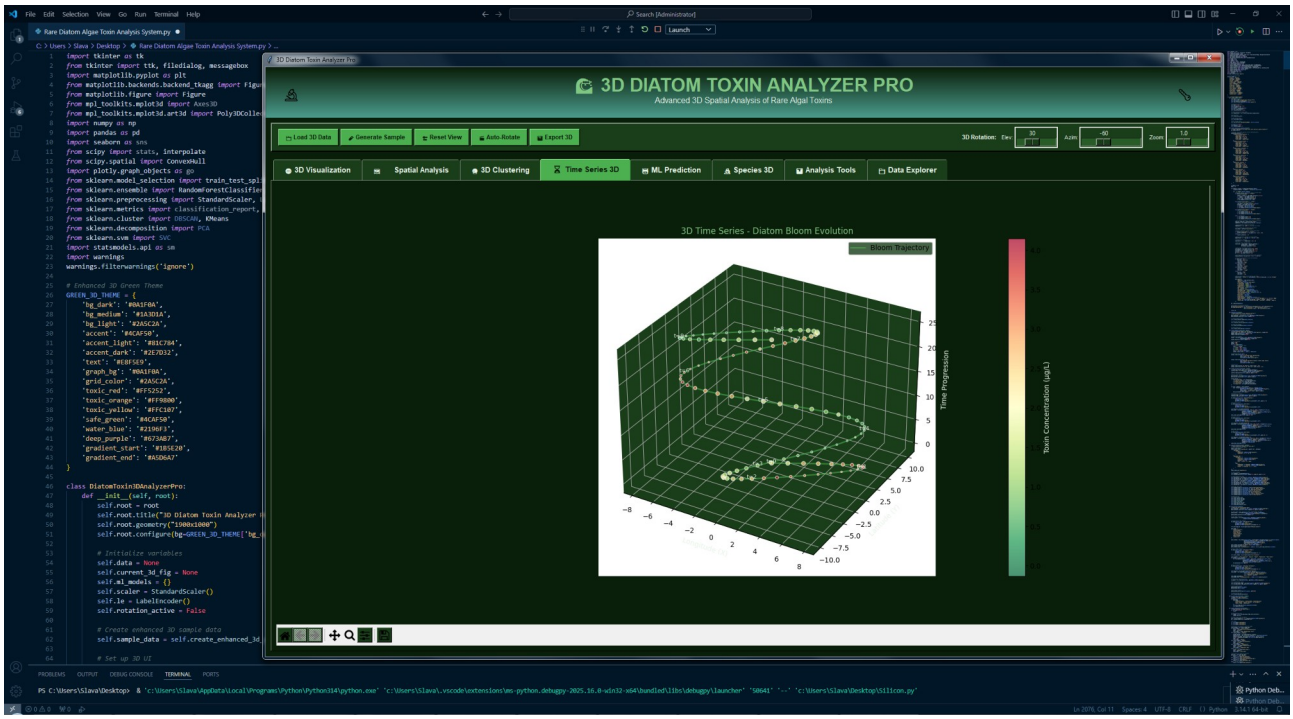

3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Color Scheme:
Size represents:
3D Contour Surface - Toxin Analysis
Longitude (X)
Latitude (Y)
Toxin Concentration (µg/L)

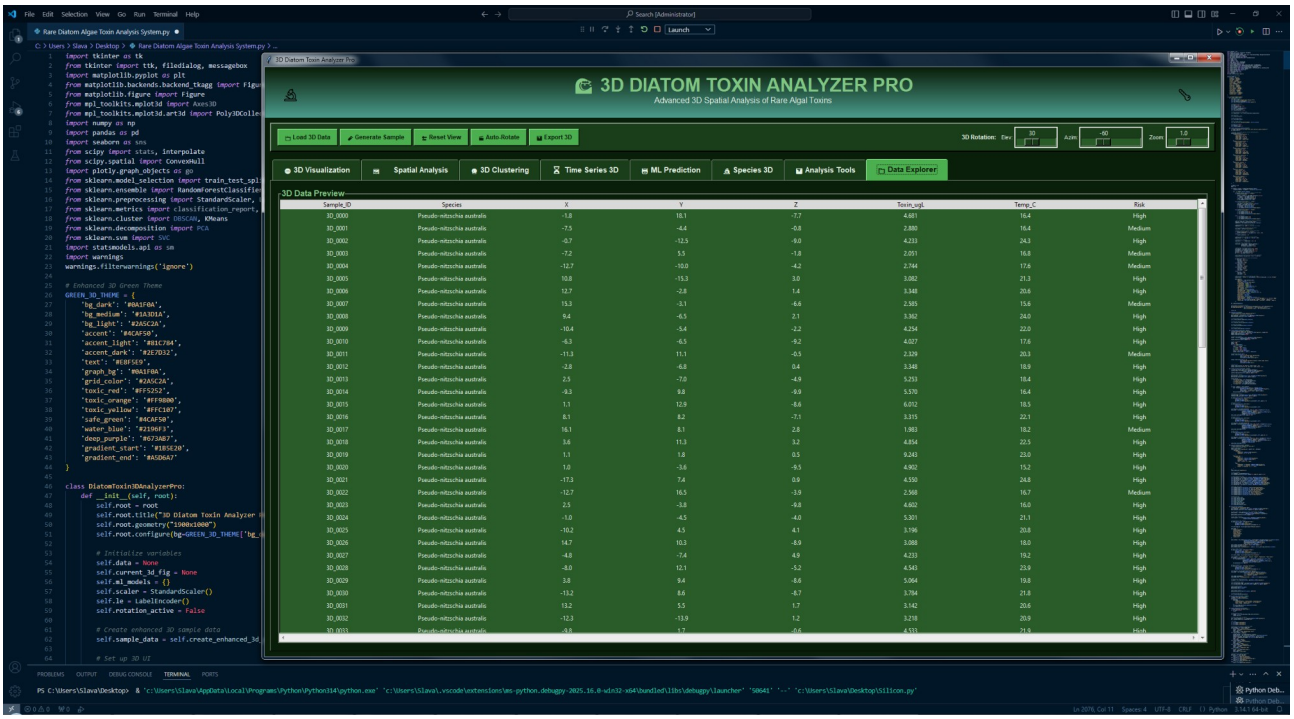

3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Color Scheme:
Size represents:
3D Histogram - Spatial Toxin Distribution
Longitude (X)
Latitude (Y)
Toxin Concentration (µg/L)


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Color Scheme:
Size represents:
3D Vector Field - Simulated Toxin Flow
Longitude (X)
Latitude (Y)
Depth (Z)
Toxin Concentration (µg/L)


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Plot Type:
Color Scheme:
Size represents:
High Toxin Hull:
Points: 124
Volume: 19541.4
Surface Area: 3951.5
3D Convex Hull - High Toxin Concentration Zones
High Toxin Points
Longitude (X)
Latitude (Y)
Depth (Z)


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
3D Time Series - Diatom Bloom Evolution
Bloom Trajectory
Time Progression
Toxin Concentration (µg/L)


3D DIATOM TOXIN ANALYZER PRO
Advanced 3D Spatial Analysis of Rare Algal Toxins
Load 3D Data
Generate Sample
Reset View
Auto-Rotate
Export 3D
3D Rotation: Elev 30 Azim -60 Zoom 1.0
3D Visualization
Spatial Analysis
3D Clustering
Time Series 3D
ML Prediction
Species 3D
Analysis Tools
Data Explorer
3D Data Preview
Sample_ID Species X Y Z Toxin_ug/L Temp_C Risk
3D_0000 Pseudo-nitzschia australis -1.8 18.1 -7.7 4.681 16.4 High
3D_0001 Pseudo-nitzschia australis -7.5 -4.4 -0.8 2.880 16.4 Medium
3D_0002 Pseudo-nitzschia australis -0.7 -12.5 -9.0 4.233 24.3 High
3D_0003 Pseudo-nitzschia australis -7.2 5.5 -1.8 2.051 16.8 Medium
3D_0004 Pseudo-nitzschia australis -12.7 -10.0 -4.2 2.744 17.6 Medium
3D_0005 Pseudo-nitzschia australis 10.8 -15.3 3.0 3.082 21.3 High
3D_0006 Pseudo-nitzschia australis 12.7 -2.8 1.4 3.348 20.6 High
3D_0007 Pseudo-nitzschia australis 15.3 -3.1 -6.6 2.305 15.6 Medium
3D_0008 Pseudo-nitzschia australis 9.4 -6.5 2.1 3.362 24.0 High
3D_0009 Pseudo-nitzschia australis -10.4 -5.4 -2.2 4.254 22.0 High
3D_0010 Pseudo-nitzschia australis -6.3 -6.5 -9.2 4.027 17.6 High
3D_0011 Pseudo-nitzschia australis -11.3 11.1 -0.5 2.329 20.3 Medium
3D_0012 Pseudo-nitzschia australis -2.8 -6.8 0.4 3.348 18.9 High
3D_0013 Pseudo-nitzschia australis 2.5 -7.0 -4.9 5.253 18.4 High
3D_0014 Pseudo-nitzschia australis -9.3 9.8 -9.9 5.570 16.4 High
3D_0015 Pseudo-nitzschia australis 1.1 12.9 -6.6 6.012 18.5 High
3D_0016 Pseudo-nitzschia australis 8.1 8.2 -7.1 3.315 22.1 High
3D_0017 Pseudo-nitzschia australis 16.1 8.1 2.8 1.983 18.2 Medium
3D_0018 Pseudo-nitzschia australis 3.6 11.3 3.2 4.854 22.5 High
3D_0019 Pseudo-nitzschia australis 1.1 1.8 0.5 9.243 23.0 High
3D_0020 Pseudo-nitzschia australis 1.0 -3.6 -9.5 4.902 15.2 High
3D_0021 Pseudo-nitzschia australis -17.3 7.4 0.9 4.550 24.8 High
3D_0022 Pseudo-nitzschia australis -12.7 16.5 -3.9 2.568 16.7 Medium
3D_0023 Pseudo-nitzschia australis 2.5 -3.8 -9.8 4.602 16.0 High
3D_0024 Pseudo-nitzschia australis -1.0 -4.5 -4.0 5.301 21.1 High
3D_0025 Pseudo-nitzschia australis -10.2 4.5 4.1 3.796 20.8 High
3D_0026 Pseudo-nitzschia australis 14.7 10.3 -8.0 3.088 18.5 High
3D_0027 Pseudo-nitzschia australis -4.8 -7.4 4.9 4.233 19.2 High
3D_0028 Pseudo-nitzschia australis -8.0 12.1 -5.2 4.543 23.9 High
3D_0029 Pseudo-nitzschia australis 3.8 9.4 -8.6 5.064 19.8 High
3D_0030 Pseudo-nitzschia australis -13.2 8.6 -6.7 3.784 21.8 High
3D_0031 Pseudo-nitzschia australis 13.2 5.5 1.7 3.142 20.6 High
3D_0032 Pseudo-nitzschia australis -12.3 -13.9 1.2 3.218 20.9 High
3D_0033 Pseudo-nitzschia australis -9.8 1.7 -0.6 4.533 21.5 High